

Warsaw University of Technology

FACULTY OF
POWER AND AERONAUTICAL ENGINEERING



Institute of Aeronautics and Applied Mechanics

Master's diploma thesis

in the field of study Automation and Robotics
and specialisation Robotics

HYBRID CLASSIFICATION SYSTEM FOR HAND GESTURE RECOGNITION
USING sEMG SIGNALS

Beidemaryam Awoke Bishaw

student record book number 323700

thesis supervisor
Professor Rzymkowski Cezary

WARSAW 2023

HYBRID CLASSIFICATION SYSTEM FOR HAND GESTURE RECOGNITION USING sEMG SIGNALS

Abstract. Grasping challenges in robotics and prosthetics require precise control mechanisms. Visual perception and EMG signal analysis enhance these abilities, but manual EMG evaluation is time-consuming. Leveraging AI can improve accuracy in detecting and classifying EMG signals for enhanced grasping control.

In this study, a comprehensive evaluation of three distinct classification systems for hand grip recognition using surface EMG (sEMG) signals was conducted, based on the results obtained from two separate databases. These systems include a standalone convolutional neural network (CNN), a hybrid CNN-Support Vector Machine (SVM) system, and a hybrid CNN-k-nearest neighbors (KNN) system. From the extensive experimentation on benchmark sEMG datasets revealed significant performance differences among the systems. The standalone CNN achieved solid accuracy, averaging 89% for Database 1 and 91% for Database 2, albeit with limited interpretability. In contrast, the hybrid CNN-SVM system outshone, merging CNN feature learning with SVM's decision boundaries, maintaining high accuracy (91% for Database 1 and 93% for Database 2) and superior interpretability. The hybrid CNN-KNN system, harnessing nearest neighbor concepts, offered flexibility and intuitive classification, with impressive results (average accuracy: 92% for Database 1 and 93% for Database 2). Notably, in Database 1, both hybrid systems surpassed the standalone CNN, while in Database 2, all three exhibited noteworthy accuracy, with the hybrid systems maintaining their superior performance.

This comparative analysis provides valuable insights into the effectiveness of these classification systems for hand grip recognition using sEMG signals and highlights the significance of considering the specific dataset when selecting the most suitable system. The findings offer guidance to researchers and practitioners in choosing the optimal system for their applications, contributing to advancements in human-machine interaction and paving the way for more accurate and efficient interfaces in fields like rehabilitation, virtual reality, and robotics.

Keywords: Grasping, Robotics, Prosthetics, Hand Grip Recognition, Electromyography(EMG) signals, Convolutional Neural Network(CNN), Support Vector Machine(SVM), K-Nearest Neighbors(kNN), Human-machine Interaction, Rehabilitation, Virtual Reality.

HYBRYDOWY SYSTEM KLASYFIKACJI ROZPOZNAWANIA GESTÓW DŁONI Z WYKORZYSTANIEM SYGNAŁÓW sEMG

Streszczenie. Chwyatanie stanowi istotne wyzwanie dla robotyków i protetyków i wymaga określenia precyzyjnych mechanizmów kontrolnych. Percepcja wzrokowa i analiza sygnału EMG zwiększają te zdolności, ale ręczna ocena EMG jest czasochłonna. Wykorzystanie sztucznej inteligencji może poprawić dokładność wykrywania i klasyfikacji sygnałów EMG w celu zwiększenia kontroli chwytania.

W tym badaniu przeprowadzono kompleksową ocenę trzech różnych systemów klasyfikacji rozpoznawania uścisku dłoni przy użyciu sygnałów powierzchniowych EMG (sEMG), w oparciu o wyniki uzyskane z dwóch oddzielnych baz danych. Systemy te obejmują samodzielną splotową sieć neuronową (CNN), hybrydowy system CNN-Support Vector Machine (SVM) i hybrydowy system CNN-k-najbliższych sąsiadów (KNN). Szeroko zakrojone eksperymenty na wzorcowych zbiorach danych sEMG wykazały znaczne różnice w wydajności pomiędzy systemami. Samodzielna sieć CNN osiągnęła dobrą dokładność, średnio 89% dla Bazy Danych 1 i 91% dla Bazy Danych 2, aczkolwiek z ograniczoną możliwością interpretacji. Natomiast hybrydowy system CNN-SVM wypadł lepiej, łącząc wysoką dokładność (91% dla bazy danych 1 i 93% dla bazy danych 2) z doskonałą interpretowalnością. Hybrydowy system CNN-KNN, wykorzystujący koncepcje najbliższego sąsiada, zapewnił elastyczność i intuicyjną klasyfikację dając jeszcze lepsze wyniki (średnia dokładność: 92% dla Bazy Danych 1 i 93% dla Bazy Danych 2). Warto zauważyć że dla Bazy Danych 1 oba systemy hybrydowe przewyższyły samodzielną sieć CNN, podczas gdy dla Bazy Danych 2 wszystkie trzy wykazały godną uwagi dokładność, a systemy hybrydowe zachowały swoją doskonałą wydajność.

Zaprezentowana w pracy analiza porównawcza dostarcza cennych informacji na temat skuteczności tych systemów klasyfikacji w zakresie rozpoznawania mechanizmów chwytania przy użyciu sygnałów sEMG i podkreśla znaczenie uwzględnienia konkretnego zbioru danych przy wyborze najbardziej odpowiedniego systemu. Odkrycia oferują badaczom i praktykom wskazówki dotyczące wyboru optymalnego systemu do ich zastosowań, przyczyniając się do postępu w interakcji człowiek-maszyna i torując drogę do dokładniejszych i wydajniejszych interfejsów w takich dziedzinach, jak rehabilitacja, rzeczywistość wirtualna i robotyka.

Słowa kluczowe: Chwyatanie, robotyka, protetyka, rozpoznawanie gestów dłoni, elektromiografia – sygnały EMG, konwolucyjna sieć neuronowa (CNN), maszyna wektorów nośnych (SVM), k-najbliższych sąsiadów (kNN), interakcja człowiek-maszyna, rehabilitacja, rzeczywistość wirtualna.

Contents

1. Introduction	9
1.1. Motivation	10
1.2. The goal of the Thesis	10
2. General Theory	11
2.1. Muscular System	11
2.1.1. Structure of Muscle	11
2.1.2. Types of Muscles	11
2.1.3. Action Potential	12
2.1.4. Muscle Contraction and Relaxation	13
2.2. Electromyography (EMG)	14
2.2.1. Measurement of Electromyography	14
2.2.2. Noises on EMG Signal	15
2.2.3. Features Extraction Methods of EMG Signal	16
2.3. Machine Learning	20
2.3.1. Supervised Learning	20
2.3.2. Unsupervised Learning	22
2.3.3. Reinforcement Learning	22
2.4. Artificial Neural Network (ANN)	23
2.4.1. Single-Layer Artificial Neural Network	23
2.4.2. Activation Function	24
2.4.3. Multiple Layer Neural Network (MLPs)	28
2.4.4. Deep Learning Algorithms	29
3. State of the Art	33
3.1. Deep Learning	33
3.1.1. Convolutional Neural Network (CNN)	33
3.1.2. Parallel Block CNN	35
3.1.3. Feedforward Neural Network(FFNN)	36
3.1.4. Extreme Learning Machine (ELM)	36
3.1.5. RNN	37
3.1.6. LSTM	39
3.2. Traditional Machine Learning	39
3.2.1. SVM	39
3.2.2. KNN	41
3.2.3. Decision Tree and Random Forest	42
3.2.4. Native Bayes Classifier	43
3.2.5. Extreme Value Machine (EVM)	44
3.3. Hybrid Systems	45
3.3.1. CNN-SVM	45
3.3.2. CNN-RNN	45
3.3.3. CNN-LSTM	47

3.3.4. HMM-SVM	48
4. Materials and Methods	50
4.1. Data Acquisition	50
4.1.1. Feature Extraction	52
4.1.2. Machine Learning Model Selection	53
4.1.3. Standalone SVM	53
4.1.4. Standalone KNN	55
4.1.5. Standalone CNN	56
4.1.6. Hybrid System	58
5. Implementation of Experiment	60
5.1. Dataset Description	60
5.1.1. Characteristics of the Dataset	60
5.1.2. Instrumentation	61
5.1.3. Hardware and Sensor Setup	61
5.1.4. Experimental Protocol	61
5.1.5. Hand Movements	62
5.1.6. Databases	62
5.1.7. Data Format	63
5.1.8. Data Dimensions	63
5.2. Evaluation Metrics	63
5.3. Procedure Followed	65
5.3.1. Define Schema of Dataset	65
5.3.2. Preprocess and Standardize Dataset	67
5.3.3. Define and Create CNN Model	67
5.3.4. Split Data into Folds	68
5.3.5. Train CNN Model	69
5.3.6. Feature Extraction from CNN	70
5.3.7. Classification with SVM and KNN	70
5.3.8. Evaluation of Models Performance	72
6. Results and Discussion	73
6.1. Database 1	73
6.1.1. CNN	73
6.1.2. CNN-SVM	74
6.1.3. CNN-KNN	75
6.1.4. Confusion Matrices	76
6.2. Database 2	76
6.2.1. CNN	76
6.2.2. CNN-SVM	78
6.2.3. CNN-KNN	79
6.2.4. Confusion Matrices	80
6.3. Comprehensive Model Performance Analysis by Grip Category	81
6.4. Overall Model Performance Comparison	83

7. Summary	84
References	85
List of Symbols and Abbreviations	92
List of Figures	94
List of Tables	95
List of Appendices	95

1. Introduction

Throughout history, mankind has continually strived to comprehend the intricacies of its own locomotion. At the core of the movement system lies the muscular system, a vital element that has piqued the curiosity of researchers and scholars alike. Electrical signals generated within the muscles have played an integral role in uncovering the underlying principles of muscle function, offering a window into the body's inner workings and its remarkable capacity for motion and control [1].

In recent years, the convergence of technology and biology has given rise to the fascinating field of hand grip recognition, using signals known as surface electromyography (sEMG). This innovative approach has garnered significant attention across various fields, including robotics, virtual reality, and rehabilitation. sEMG signals, collected from the skin's surface through special sensors, hold the key to understanding the activity and functions of our muscles. These signals find applications in clinical medicine, biomedical engineering, and advanced prosthetics, establishing sEMG as a powerful tool for capturing and interpreting neuromuscular activity [2].

This thesis delves deep into the world of hand grip recognition using sEMG signals, undertaking a comprehensive comparison of three distinct classification systems. These systems encompass a standalone convolutional neural network (CNN), a hybrid CNN combined with support vector machines (CNN-SVM), and a hybrid CNN integrated with k-nearest neighbors (CNN-kNN). Each of these approaches brings unique strengths that significantly impact their ability to accurately classify hand grips.

The standalone CNN relies on deep learning techniques to automatically extract meaningful patterns from raw sEMG data, enabling it to recognize intricate grips effectively. In contrast, the hybrid CNN-SVM system cleverly combines the feature extraction capabilities of CNNs with the discriminating power of SVMs, resulting in improved classification accuracy and interpretability. Similarly, the hybrid CNN-kNN system capitalizes on the concept of nearest neighbors, offering flexibility and precision in the classification process [3].

Through rigorous experiments and evaluations using benchmark sEMG datasets, this study scrutinizes the performance of these three systems. Metrics like classification accuracy, precision, recall, and F1-score provide valuable insights into the strengths and limitations of each approach. This analysis guides researchers and practitioners in selecting the most suitable method for sEMG-based hand grip recognition.

The results of this thesis not only enrich our understanding of hand grip recognition systems but also have tangible implications in the realm of human-machine interaction. These findings stand to facilitate the development of highly accurate and efficient interfaces, particularly beneficial in fields such as rehabilitation, virtual reality, and robotics. Ultimately, this work carries the potential to elevate the quality of life for individuals grappling with impaired motor function, marking a significant stride towards bridging the divide between biology and technology.

1.1. Motivation

The motivation behind this thesis lies in addressing the need for accurate and efficient hand grip recognition systems using surface electromyography (sEMG) signals. With the growing demand for intuitive human-computer interaction, particularly in fields like robotics, virtual reality, and rehabilitation, sEMG signals offer a non-invasive and natural approach. However, achieving high classification accuracy and robustness remains a challenge due to the complex nature of sEMG signals. To overcome these challenges, this research focuses on comparing and evaluating three classification systems: standalone convolutional neural network (CNN), CNN combined with support vector machines (CNN-SVM), and CNN integrated with k-nearest neighbors (CNN-kNN). By investigating the strengths and limitations of each approach, we aim to improve the accuracy, interpretability, and efficiency of hand grip recognition using sEMG signals.

1.2. The goal of the Thesis

The goal of this thesis is to individually compare and evaluate three different classification systems for hand grip recognition using sEMG signals: a standalone convolutional neural network (CNN), a CNN combined with support vector machines (CNN-SVM), and a CNN integrated with k-nearest neighbors (CNN-kNN). Each system offers distinct advantages and characteristics that influence their performance in accurately classifying hand grips. By conducting rigorous experimentation and evaluation using UCI Machine Learning Repository sEMG datasets, we aim to assess the performance of each system in terms of classification accuracy, precision, recall, and F1-score. The findings will provide valuable insights into the strengths and limitations of each approach, enabling researchers and practitioners to make informed decisions when selecting the most suitable system for hand grip recognition using sEMG signals. Additionally, this research contributes to the advancement of the field by enhancing the understanding of hand grip recognition systems and their practical application in areas such as rehabilitation, virtual reality, and robotics, ultimately improving human-machine interaction and the quality of life for individuals with impaired motor function.

2. General Theory

In this section, the fundamental concepts of muscle function will be discussed. Additionally, electromyography, a technique for measuring electrical signals from muscles, and the exploration of artificial neural networks, computational tools for comprehending and managing complex information, will be covered.

2.1. Muscular System

Muscle is categorized as one of the four main tissue types in the body, consisting of specialized fibers. When individuals typically envision muscles, they usually refer to the ones visible beneath the skin, particularly in the limbs. These muscles are known as skeletal muscles since they primarily facilitate movement of the skeleton. However, in addition to skeletal muscles, there are two other types of muscles: smooth muscles and cardiac muscles. The human body boasts over 600 muscles that play a substantial role in contributing to its overall weight [4].

2.1.1. Structure of Muscle

As shown in Figure 2.1, a muscle is composed of multiple muscle tissues grouped together and enveloped by a tough connective tissue called epimysium, which resembles cartilage. Within the epimysium, there are bundles of nerve cells running in elongated fibers known as fascicles. Each fascicle is enclosed by its own protective layer called perimysium, which facilitates the passage of nerves and blood to the individual muscle fibers. Additionally, each fiber is further enveloped by another protective layer called endomysium. This arrangement forms a basic pattern of bundled fibers separated by protective layers within a muscle, enabling different parts of the muscle to contract independently. The protective layers surrounding the bundles allow them to glide past each other during contraction. The epimysium connects to tendons, which in turn attach to the periosteum, a connective tissue surrounding the bones. This connection to two bones enables the muscle to cause movement in the skeleton when it contracts. Another type of muscle surrounds various organs, and the epimysium connects to other connective tissues, exerting forces on the organs and controlling functions such as circulation and food processing [5].

2.1.2. Types of Muscles

As shown in Figure 2.2, muscle has three primary types: skeletal, smooth, and cardiac muscles.

A. Skeletal Muscle

The musculoskeletal system, also referred to as the locomotors system, encompasses the specialized tissue that connects to bones and enables movement. Specifically, skeletal muscles, when combined with bones, constitute this system. Typically, skeletal muscles are organized in opposing pairs, such as the biceps and triceps located on the front and back of the upper arm. These muscles fall under our conscious

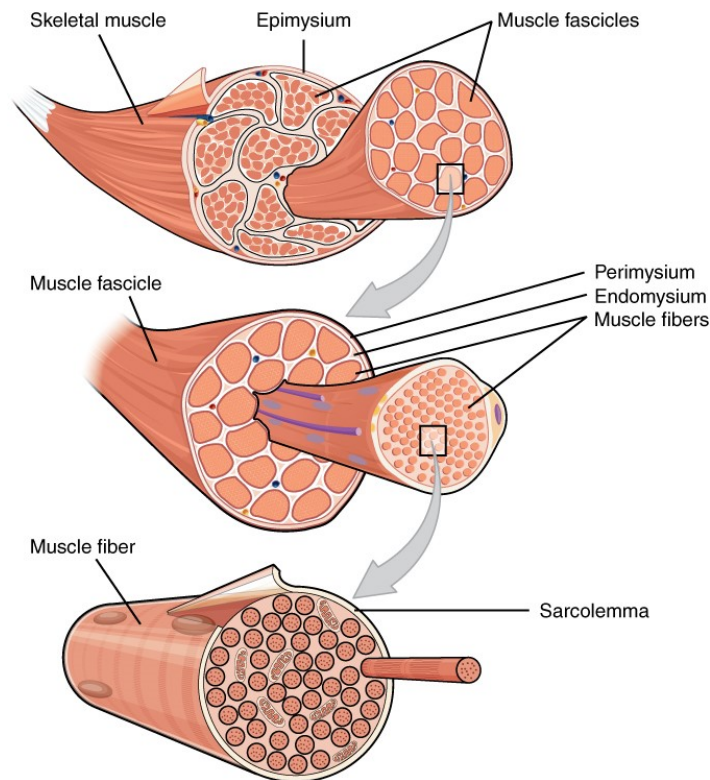


Figure 2.1. Connective Tissue Layers [5]

control, thus earning the designation of voluntary muscles. Another term used to describe them is "striated muscles," as their tissue exhibits a striped appearance when observed through a microscope [6].

B. Cardiac Muscle

Cardiac muscles are located within the walls of the heart. Their role is to assist in pumping blood throughout the cardiovascular system. Unlike skeletal muscles, cardiac muscles are not under conscious control. Instead, they receive signals from the heart itself, dictating when to contract [6].

C. Smooth Muscle

Smooth muscles are found lining the inner walls of organs like the bladder, stomach, and intestines. They serve crucial functions in various body systems, including the female and male reproductive systems, urinary system, and respiratory system. Unlike skeletal muscles, smooth muscles operate involuntarily, without conscious thought. They perform vital tasks such as facilitating the movement of waste through the intestines and aiding in the expansion of the lungs during breathing [6].

2.1.3. Action Potential

Action potentials serve as the fundamental units of communication in the nervous system, enabling the transmission of electrical signals over long distances and facilitating various physiological processes such as muscle contraction, sensory perception, and information processing in the brain. An action potential is a rapid and transient change in the electrical potential across a cell membrane. As shown in Figure 2.3, it is initiated

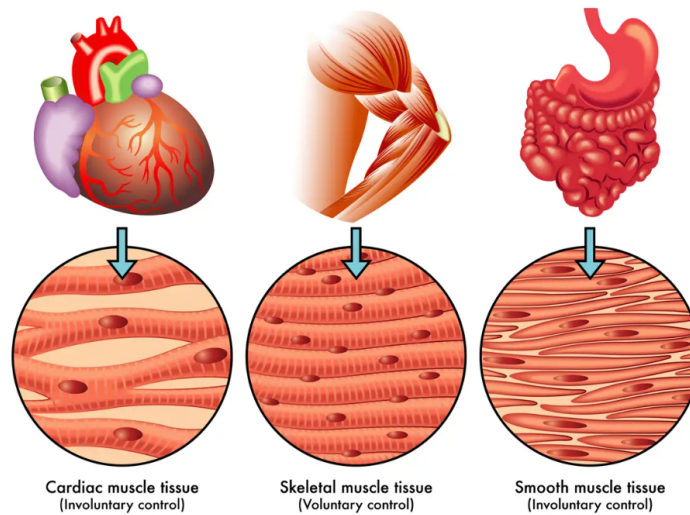


Figure 2.2. Types of Muscle Tissue [7]

by a stimulus and involves depolarization, where the membrane potential becomes less negative, followed by repolarization. During depolarization, voltage-gated sodium channels open, allowing sodium ions to enter the cell and causing a rise in membrane potential. Repolarization occurs as voltage-gated potassium channels open, leading to the outflow of potassium ions and the restoration of the membrane potential. Action potentials serve as essential electrical signals in the nervous system, enabling the transmission of information and facilitating various physiological processes [8].

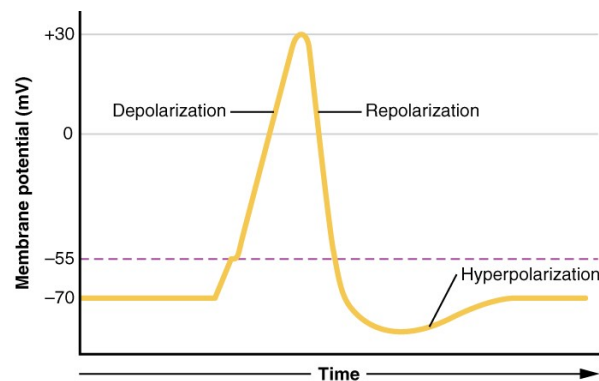


Figure 2.3. Action Potential [8]

2.1.4. Muscle Contraction and Relaxation

Muscle contraction is initiated by a signal from the nervous system, leading to the release of calcium ions that enable the formation of cross-bridges between actin and myosin filaments. ATP provides energy for this process, causing the muscle to contract and generate force. Muscle relaxation occurs when the nerve signal stops, and calcium ions are pumped back into storage. This leads to the detachment of cross-bridges and the reshielding of actin-binding sites. As a result, the muscle fiber returns to its original length through passive elasticity. The sequence of events involves depolarization of the muscle

fiber membrane, release of calcium ions, sustained contraction with the availability of ATP, and subsequent repolarization and relaxation as calcium ions are removed from the sarcoplasm [9].

2.2. Electromyography (EMG)

Electromyography (EMG) is a diagnostic technique used to assess and study the electrical activity of muscles. It involves the measurement and analysis of the electrical signals produced by muscle fibers during rest and contraction. EMG provides valuable insights into muscle function, neuromuscular disorders, and the interaction between nerves and muscles [10].

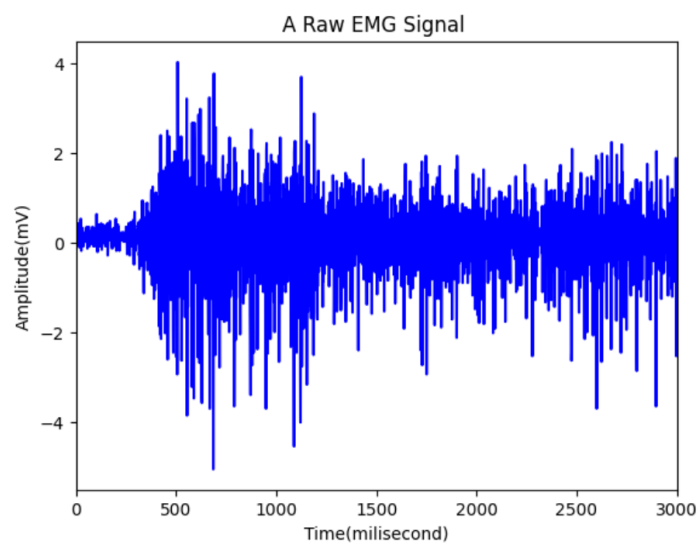


Figure 2.4. Sample Raw EMG Signal

2.2.1. Measurement of Electromyography

During an EMG procedure, small electrodes are placed on the skin above the muscles being examined. These electrodes detect and record the electrical signals generated by the muscle fibers. The signals are then amplified, filtered, and displayed on a monitor or recorded for further analysis. EMG can provide information about muscle activity, including the timing, strength, and coordination of muscle contractions. It helps in the diagnosis of various conditions such as muscle disorders, nerve injuries, neuropathies, and neuromuscular diseases [11].

Types of EMG Signal Measurements

As discussed in section 2.2, electromyography (EMG) is a valuable technique used to study muscle activity. It encompasses two primary methods: surface EMG (sEMG) and intramuscular EMG (iEMG).

- A. **Surface EMG (sEMG):** As depicted in Figure 2.5, sEMG involves placing electrodes on the skin surface, providing a non-invasive assessment of overall muscle activity.

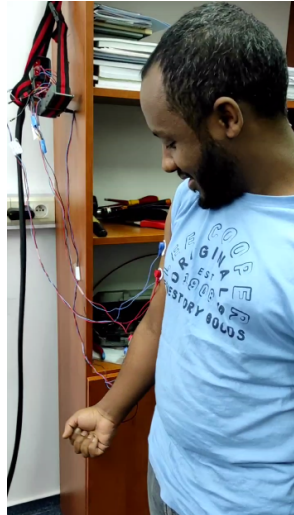


Figure 2.5. Measurement of Electromyography[10]

- B. **Intramuscular EMG (iEMG):** As shown in Figure 2.6, iEMG involves inserting a fine needle electrode directly into the muscle, allowing for a more detailed examination of specific muscle fibers [12].

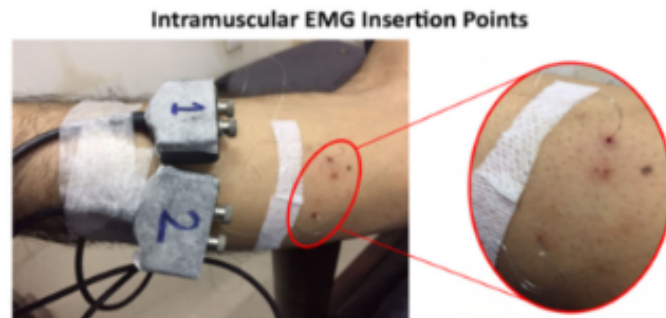


Figure 2.6. Intramuscular Electrode Insertion Sites on the Forearm [13]

2.2.2. Noises on EMG Signal

The EMG signal has an amplitude range of 0 to 10 mV before amplification. As the EMG signal travels through various tissues, it picks up noise, as shown in Figure 2.4. Understanding the characteristics of this electrical noise is crucial. The types of electrical noise that can affect EMG signals can be classified as follows [14]:

A. **Inherent instability of signal:**

The inherent instability of the signal refers to the fluctuations or variations present in the EMG signal that are not related to any noise sources but are inherent properties of the physiological signal itself. These fluctuations may arise from factors such as small changes in electrode-skin impedance, muscle fatigue, or natural variability in muscle activity. EMG signal amplitude is inherently random due to motor unit firing rates, typically within the frequency range of 0 to 20 Hz. Removing this unwanted noise is crucial for accurate analysis [14].

B. **Motion artifact:**

Motion artifact distorts data by introducing irregularities, mainly originating from the electrode interface, muscle and cable movement. Proper circuitry design and setup can help to reduce motion artefact [14], [15].

C. **Inherent noise in electronics equipment:**

Noise generated by electronics equipment like amplifiers is unavoidable and can only be minimized by using high-quality components [15].

D. **Muscle Cross-Talk:**

In some cases, adjacent muscles can create interference in the EMG signal. This occurs when the EMG electrodes pick up signals from nearby muscles in addition to the targeted muscle[16].

E. **Ambient noise:**

Ambient noise arises from electromagnetic radiation in our surroundings and can have amplitudes much higher than the EMG signal [17].

2.2.3. Features Extraction Methods of EMG Signal

EMG signal features can be categorized into three domains: time domain, frequency domain, and time-frequency domain features. Each domain offers unique insights into the characteristics and patterns of EMG signals [18].

I. **Time-domain Features:**

The time domain is a widely used method for feature extraction in EMG classification due to its simplicity and efficiency. Unlike other domains, it requires no signal transformation, making it easy and quick to compute. This approach leverages the signal's amplitude to extract features, providing valuable information about the signal's amplitude, frequency, and duration.

A. **Mean Absolute Value (MAV):**

The MAV is a simple and commonly used time-domain feature. It calculates the average absolute amplitude of the EMG signal within a specific time window. The formula for MAV is as follows:

$$MVN = \frac{1}{N} \sum_{n=1}^N |X_n| \quad (2.1)$$

In equation (2.1), N is the number of samples in the time window and X_n represents the EMG amplitude at sample n . MAV provides a measure of the overall magnitude of the EMG signal within the selected time window, which can be useful in quantifying muscle activation levels.

B. **Root Mean Square (RMS):**

The RMS is another frequently used time-domain feature. It calculates the square root of the average of the squared values of the EMG signal within a time window.

The formula for RMS is as follows:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{n=1}^N X_n^2} \quad (2.2)$$

RMS is more sensitive to high amplitudes and less sensitive to low amplitudes compared to MAV. It provides information about the signal's energy and is often used to assess muscle fatigue.

C. Zero Crossing (ZC):

Zero crossing is a simple time-domain feature that counts the number of times the EMG signal crosses the zero baseline within a time window. It can provide insights into the frequency and rhythmicity of muscle activation. The equation for ZC is given by:

$$\text{ZC} = \sum_{i=1}^{N-1} f(x) \quad (2.3)$$

where $f(x)$ is defined as:

$$f(x) = \begin{cases} 1, & \text{if } (x_i > 0 \text{ and } x_{i+1} < 0) \text{ or } (x_i < 0 \text{ and } x_{i+1} > 0) \text{ and } |x_i - x_{i+1}| \geq \epsilon, \\ 0, & \text{otherwise.} \end{cases}$$

D. Integrated Absolute Value (IAV):

IAV is a time-domain feature that calculates the total area under the absolute value of the EMG signal within a specific time window. It quantifies the overall muscle activity during that time interval. By integrating the absolute values, IAV emphasizes the overall magnitude of the signal, providing information about muscle activation levels. The equation for IAV is given by:

$$\text{IAV} = \sum_{i=1}^N |x_i| \quad (2.4)$$

E. Wilson Amplitude (WAMP):

WAMP is a time-domain feature that quantifies the number of times the EMG signal crosses its mean value within a specified time window. It counts both positive and negative excursions across the mean value. WAMP is useful for assessing the variations and fluctuations in the EMG signal, offering insights into the dynamics of muscle activity. The equation for WAMP is given by:

$$\text{WAMP} = \sum_{i=0}^N f(|x_i - x_{i+1}|) \quad (2.5)$$

where $f(x)$ is defined as:

$$f(x) = \begin{cases} 1, & \text{if } x > \text{threshold}, \\ 0, & \text{otherwise.} \end{cases}$$

E. Variance (VAR):

VAR is a statistical time-domain feature that measures the variability or spread of the EMG signal within a time window. It calculates the average of the squared deviations from the mean value. Variance provides information about the dispersion of the signal's amplitude, helping to assess the consistency or variability of muscle contractions. The equation for VAR is given by:

$$\text{VAR} = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 \quad (2.6)$$

G. Slope Sign Changes (SSC):

SSC is a time-domain feature that characterizes the number of times the slope of the EMG signal changes its direction within a specified time window. It counts the instances where the signal changes from increasing to decreasing or vice versa. SSC can reveal the abruptness or smoothness of muscle contractions, which can be relevant in certain applications like gait analysis. The equation for SSG is given by:

$$\text{SSG} = \sum_{i=1}^N f(x) \quad (2.7)$$

where $f(x)$ is defined as:

$$f(x) = \begin{cases} -x, & \text{if } (x_i > x_{i-1} \text{ and } x_i > x_{i+1}) \text{ or } (x_i < x_{i-1} \text{ and } x_i < x_{i+1}) \\ & \text{and } |x_i - x_{i+1}| \geq \epsilon \text{ or } |x_i - x_{i-1}| \geq \epsilon, \\ 0, & \text{otherwise.} \end{cases}$$

II. Frequency-domain Features:

Frequency-domain features involve analyzing the distribution of frequency components in the EMG signal. It allows us to understand muscle activation frequencies, detect changes over time, and assess power in different frequency bands. Common features include power spectral density, median frequency, spectral moments, and dominant frequency. These features provide valuable insights into muscle function, fatigue, and control applications.

A. Auto-Regressive Coefficients (AR):

Autoregressive modeling fits the EMG signal to an autoregressive model. The AR model represents the current value of the signal as a linear combination of its past values. The coefficients obtained from the AR model can serve as features. AR modeling is useful for capturing the temporal dependencies and predicting future

values of the EMG signal. The equation for x_i is given by:

$$x_i = \sum_{j=1}^n a_j x_{i-j} \quad (2.8)$$

In equation (2.8), a_j represents the coefficient of an autoregressive (AR) model of a signal x_i .

B. Power Spectral Density (PSD):

PSD represents the distribution of power across different frequency bands in the EMG signal. We can use PSD to assess changes in muscle activation patterns across different frequency ranges. The Power Spectral Density (PSD) $S_x(f)$ is defined as:

$$S_x(f) = \lim_{T \rightarrow \infty} E \left\{ \frac{1}{2T} \left| \int_{-T}^T x_m(t) e^{-j2\pi f t} dt \right|^2 \right\} \quad (2.9)$$

From equation (2.9), $x_m(t)$ represents a signal and $S_x(f)$ represents the average of the squared magnitudes of its Fourier transform over a time interval T .

C. Frequency Median (FMD):

The median frequency is a frequency-domain feature that divides the power spectrum into two equal halves. It provides an estimate of the central frequency of the EMG signal. Median frequency is often used to assess muscle fatigue, as fatigue tends to shift the median frequency to lower values. The frequency median (FMD) is defined as:

$$FMD = \frac{1}{2} \sum_{i=1}^N \text{PSD}_i \quad (2.10)$$

From equation (2.10) N represents the length of the PSD.

III. Time-Frequency Domain:

The time-frequency domain allows us to observe the signal's energy in both the time and frequency dimensions. Unlike the time domain, this method requires a transformation, making it computationally more challenging.

A. Wavelet Transform:

Wavelet transform allows simultaneous analysis of time and frequency information. It decomposes the EMG signal into time-varying frequency components using wavelet functions. This method is advantageous when the frequency content of the EMG signal changes over time, as it can capture transient muscle activation patterns. The Wavelet Transform (CWT) is defined as:

$$W_x(a, b) = \frac{1}{\sqrt{a}} \int_{-\infty}^{+\infty} x(t) \Psi^* \left(\frac{t-b}{a} \right) dt \quad (2.11)$$

B. The Fourier Transform (FT):

FT is a mathematical technique that expresses a signal as a combination of cosine and sine functions with varying frequencies, phases, and amplitudes. It provides a frequency spectrum of the real signal, showing the contribution of different

frequencies to the overall signal. FT is useful for analyzing non-periodic functions and is reversible, meaning the original signal can be reconstructed from its frequency representation. The Fourier Transform (FT) of a signal $x(t)$ is defined as:

$$\mathcal{F}\{x(t)\} = x^\wedge(f) = F\{x(t)\} = \int_{-\infty}^{\infty} x(t) e^{-i2\pi f t} dt \quad (2.12)$$

The Inverse Fourier Transform of $x^\wedge(f)$ is defined as:

$$\mathcal{F}^{-1}\{x^\wedge(f)\} = x(t) = \int_{-\infty}^{\infty} x^\wedge(f) e^{i2\pi f t} df \quad (2.13)$$

C. Short Time Fourier Transform (STFT):

STFT is another time-frequency domain technique that reveals the time-varying frequency components of the EMG signal. It divides the signal into short overlapping segments and performs the Fourier transform on each segment. STFT provides a time-frequency representation that can be useful for analyzing muscle activities that evolve over time. The Short-Time Fourier Transform (STFT) of a signal $x(t)$ is defined as:

$$\text{STFT}(t, \omega) = \int_{-\infty}^{\infty} W^*(\tau - t) x(\tau) e^{-j\omega\tau} d\tau \quad (2.14)$$

From equation (2.14), the symbol $W(t)$ represents the window function, $*$ denotes the complex conjugate, t represents time, and ω denotes the angular frequency.

2.3. Machine Learning

So far, the muscular system and the nature of electromyography signals have been discussed. In this section, different features of machine learning for the classification of sEMG signals will be addressed. Machine learning is a subset of artificial intelligence that empowers computers to learn from data without explicit programming. It involves developing algorithms and statistical models that identify patterns and relationships in data to make predictions or decisions. These algorithms can be categorized as supervised, unsupervised, and reinforcement learning. Applications of machine learning are vast, including image recognition, natural language processing, recommendation systems, autonomous vehicles, medical diagnosis, and more, driving advancements across diverse industries.

2.3.1. Supervised Learning

Supervised learning is a machine learning paradigm where algorithms learn from labeled data to make predictions. It includes two main types: regression (for continuous output prediction) and classification (for categorical output prediction). Common algorithms are linear regression, logistic regression, SVM, KNN, decision trees, random forests, gradient boosting, and neural networks. It finds applications in various fields like image recognition, speech processing, sentiment analysis, and more.

A. Support Vector Machine (SVM):

Support Vector Machine (SVM) is a powerful and widely used supervised learning algorithm for both classification and regression tasks. SVM is particularly effective in scenarios where the data is not linearly separable, and the algorithm aims to find the optimal hyperplane that best separates the data points of different classes. As shown in Figure 2.7, the key idea is to maximize the margin, which is the distance between the hyperplane and the closest data points (support vectors) of each class.

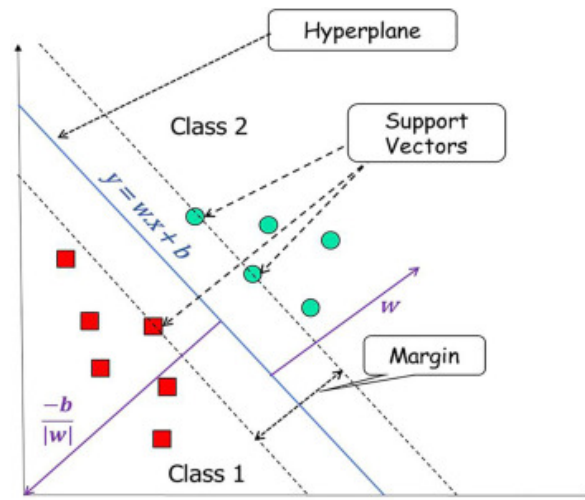


Figure 2.7. Components of Support Vector Machine (SVM) [19]

B. K-Nearest Neighbors (KNN):

K-Nearest Neighbors (KNN) is a simple yet powerful supervised learning algorithm used for both classification and regression tasks. As shown in Figure 2.8, it operates on the principle that similar data points tend to have similar outcomes. In KNN, the "K" represents the number of nearest neighbors (data points) used to make predictions for a new data point.

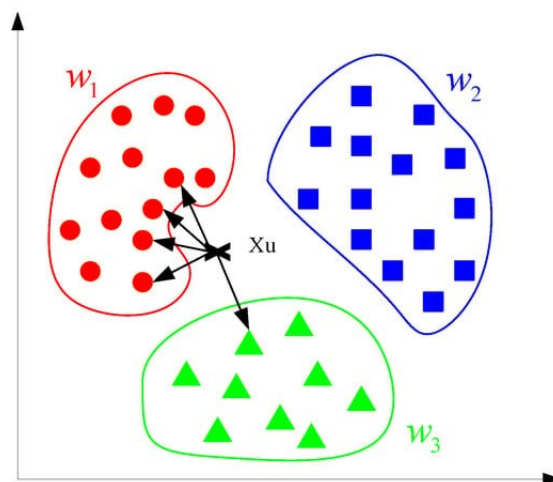


Figure 2.8. Example for KNN Algorithm (k=5) [20]

2.3.2. Unsupervised Learning

Unsupervised learning is a type of machine learning where the algorithm is trained on unlabeled data, meaning the data does not have corresponding target labels or output values. The goal of unsupervised learning is to discover patterns, relationships, or structures within the data without any explicit guidance on what the output should be.

A. Clustering:

As shown in Figure 2.9, in clustering, the algorithm groups similar data points together into clusters based on their features or attributes. The goal is to partition the data into meaningful groups or clusters, with data points within a cluster being more similar to each other than to those in other clusters. Common clustering algorithms include K-Means, Hierarchical Clustering, and Density-Based Spatial Clustering of Applications with Noise (DBSCAN).

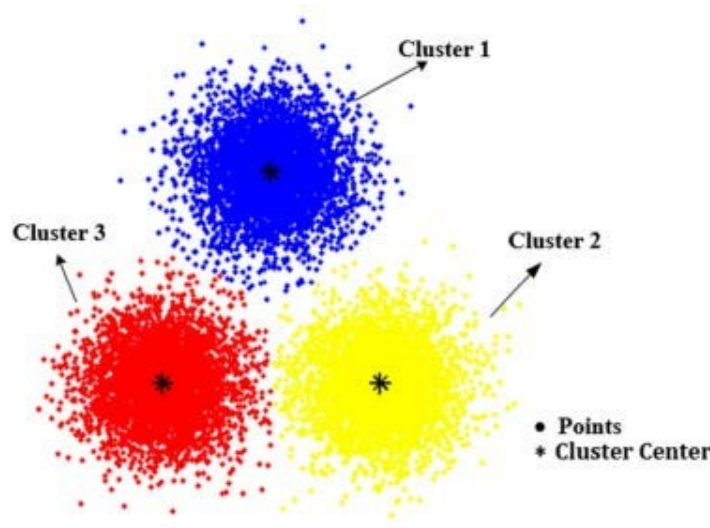


Figure 2.9. Centroid-based Clustering [21]

B. Dimensionality Reduction:

Dimensionality reduction techniques aim to reduce the number of features in the data while preserving its essential characteristics. This is useful for visualizing high-dimensional data, speeding up computations, and reducing the risk of overfitting in certain models. Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE) are popular dimensionality reduction algorithms.

2.3.3. Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning that focuses on training agents to make decisions in an environment to achieve specific goals. Unlike supervised learning, RL does not rely on labeled data, but rather, the agent learns by interacting with the environment and receiving feedback in the form of rewards or penalties. As shown in Figure 2.10, the RL agent takes actions in the environment, and the environment responds with new states and corresponding rewards. The goal of the agent is to learn an optimal

policy, which is a mapping from states to actions, to maximize the cumulative rewards over time.

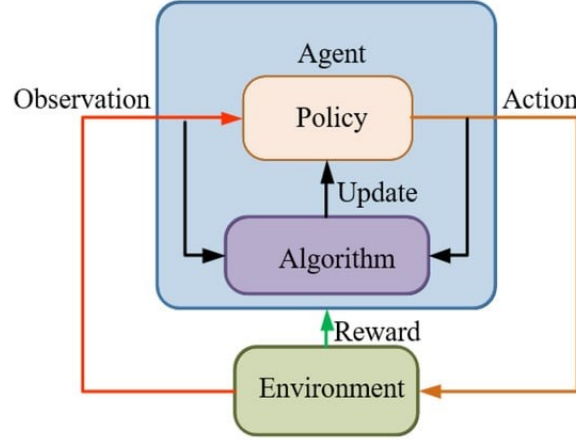


Figure 2.10. The Fundamental Framework of Reinforcement Learning (RL) [22]

2.4. Artificial Neural Network (ANN)

An Artificial Neuron, also known as a Perceptron or Artificial Neural Unit, is a fundamental building block of Artificial Neural Networks (ANNs). It is an engineering abstraction inspired by the biological neuron found in the human brain. ANNs are a class of machine learning algorithms designed to model complex relationships and patterns in data.

2.4.1. Single-Layer Artificial Neural Network

A single-layer ANN, also known as a Perceptron, consists of only one layer of artificial neurons. This layer is the output layer, and there are no hidden layers. As shown in Figure 2.11, in a single-layer ANN the inputs are directly connected to the output neurons. In a single-layer ANN, the neurons' outputs are calculated using a linear combination of the inputs and their corresponding weights, followed by the application of an activation function. The Perceptron is limited in its ability to learn complex patterns and can only classify linearly separable data. Equation (2.15) describes the computation of the output of an artificial neuron (y_k) in an artificial neural network:

$$y_k = \phi \left(\sum_{i=1}^N X_i \cdot w_{ki} + b_k \right) \quad (2.15)$$

where y_k , ϕ , X_i , w_{ki} , and b_k represent the output, activation function, input, weight, and bias of the artificial neural network, respectively. Basically, an artificial neural network is composed of several key components including:

A. Inputs and Weights:

An Artificial Neuron receives multiple inputs, each representing a feature or piece of information. These inputs are typically real-valued numbers. Associated with each input is a weight, which represents the importance or strength of that input's

contribution to the neuron's output. The weights are the learnable parameters of the neuron that get adjusted during the learning process.

B. **Bias:**

In addition to the inputs and weights, an artificial neuron often includes a bias term. The bias is an additional learnable parameter that helps the neuron model an offset or baseline value, providing flexibility to the neuron's activation behavior.

C. **Weights:**

Weights represent the strengths of connections between neurons. These weights are learned during the training process and determine the impact of one neuron's output on another neuron's input.

D. **Activation Function:**

The weighted sum of inputs, including the bias, is then passed through an activation function. The activation function introduces non-linearity to the neuron's output. This non-linearity is crucial for the artificial neuron to learn and model complex relationships in data. Common activation functions include Sigmoid, ReLU (Rectified Linear Unit), Tanh, and others.

E. **Output Layer:**

This layer provides the final predictions, classifications, or outcomes based on the processed information from the hidden layers.

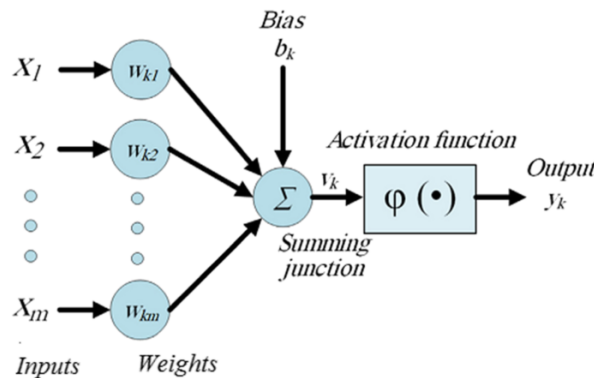


Figure 2.11. Simple Structure of ANN [23]

2.4.2. Activation Function

Activation functions are essential components in artificial neural networks and play a crucial role in introducing non-linearity to the model. Their purpose is to determine whether a neuron should be activated or not, which helps the neural network learn complex patterns and relationships in the data. Without activation functions, neural networks would behave like linear models, limiting their ability to solve more complex tasks.

A. **Binary Step Function:**

The Binary Step Function is a simple activation function used in artificial neural networks. It outputs 0 or 1 based on whether the input is less than or equal to a specified threshold, as shown in Figure 2.12. While it introduces non-linearity, it has limited use in multi-layer networks due to its lack of differentiability, making it

unsuitable for training. It is mainly employed in binary classification problems and lacks the flexibility of other activation functions commonly used in neural networks. The binary step function is defined as:

$$f(x) = \begin{cases} 0, & \text{if } x < 0, \\ 1, & \text{if } x \geq 0. \end{cases} \quad (2.16)$$

B. Linear Activation Function:

The Linear Activation Function, also known as the Identity Activation Function, is one of the simplest activation functions used in artificial neural networks. Unlike other activation functions that introduce non-linearity, the Linear Activation Function simply computes a weighted sum of the inputs without any transformation. It preserves the linearity of the data and is represented by a straight line with a slope, as shown in Figure 2.13.

The linear activation function is defined as:

$$f(x) = cx. \quad (2.17)$$

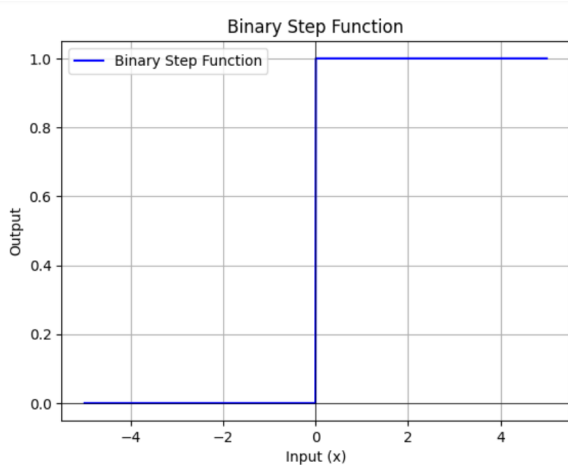


Figure 2.12. Binary Step Function

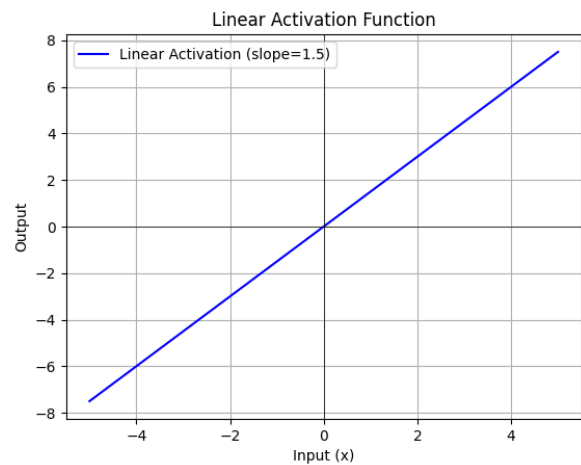


Figure 2.13. Linear Activation Function

C. Sigmoid Function (Logistic Activation):

The Sigmoid Function, also known as the Logistic Activation Function, is a widely used activation function in neural networks. It transforms the input into a range between 0 and 1, representing probabilities for binary classification tasks. As shown in Figure 2.14, the function's S-shaped curve introduces non-linearity, enabling the network to learn complex patterns and relationships. However, the Sigmoid Function suffers from vanishing gradient problems for extreme input values, which can slow down training. While other activation functions have become more popular in modern deep learning, the Sigmoid Function is still used in certain scenarios, particularly for the output layer in binary classification tasks.

The sigmoid function (logistic activation) function is defined as:

$$f(x) = \frac{1}{1 + e^{-x}}. \quad (2.18)$$

D. Hyperbolic Tangent Function:

The Hyperbolic Tangent Function, commonly known as tanh, is an activation function used in neural networks to introduce non-linearity and scale the output between -1 and 1. As shown in Figure 2.15, it is similar to the sigmoid function but produces zero-centered outputs, which can be advantageous in certain situations. The tanh function maps positive values to values close to 1 and negative values to values close to -1, making it more balanced than the sigmoid, which maps positive values to values close to 1 but negative values to values close to 0. The hyperbolic tangent function is defined as:

$$f(x) = \frac{2}{1 + e^{-2x}} - 1. \quad (2.19)$$

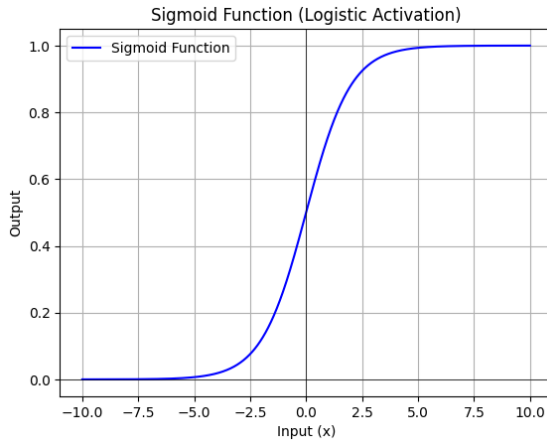


Figure 2.14. Sigmoid Activation Function

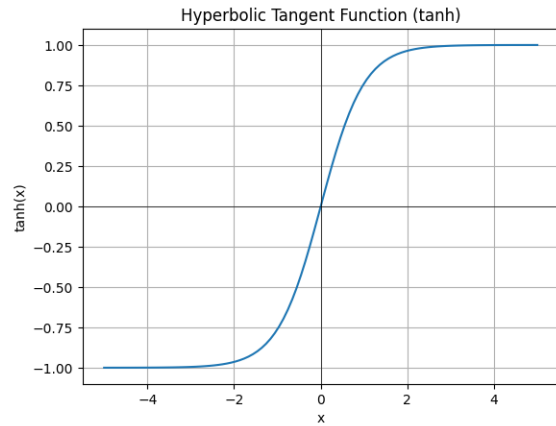


Figure 2.15. Hyperbolic Tangent Function

E. Rectified Linear Unit (ReLU):

The Rectified Linear Unit (ReLU) is a popular activation function used in neural networks due to its simplicity and ability to alleviate the vanishing gradient problem. The ReLU function returns the input value if it is positive, and zero otherwise, as shown in Figure 2.16. It can be represented as $f(x) = \max(0, x)$. ReLU introduces non-linearity to the network, allowing it to learn complex patterns effectively. However, it is susceptible to the "dying ReLU" problem, where neurons can get stuck and output zero for all inputs during training, preventing further learning. Various modifications, such as Leaky ReLU and Parametric ReLU, have been introduced to address this issue. The rectified linear unit (ReLU) is defined as:

$$f(x) = \max(0, x). \quad (2.20)$$

F. Leaky ReLU:

Leaky ReLU is a variation of the Rectified Linear Unit (ReLU) activation function designed to address the "dying ReLU" problem. In the Leaky ReLU function, instead of setting negative inputs to zero, a small positive slope (usually a small constant, like 0.01) is introduced for negative inputs, as shown in Figure 2.17. This small slope allows the neurons to still carry some gradient during backpropagation, preventing them from getting stuck and potentially "dying" during training.

The Leaky ReLU is defined as:

$$f(x) = \max(ax, x). \quad (2.21)$$

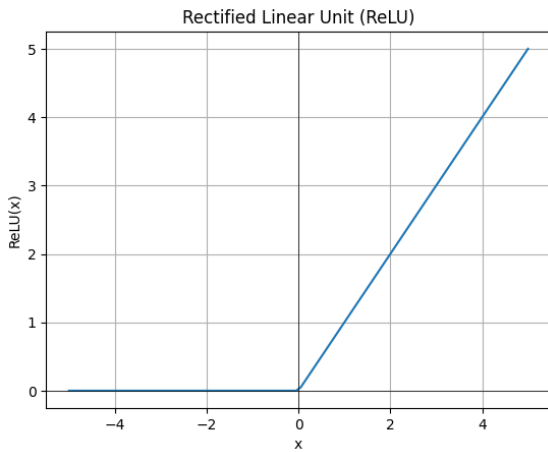


Figure 2.16. Rectified Linear Unit (ReLU)

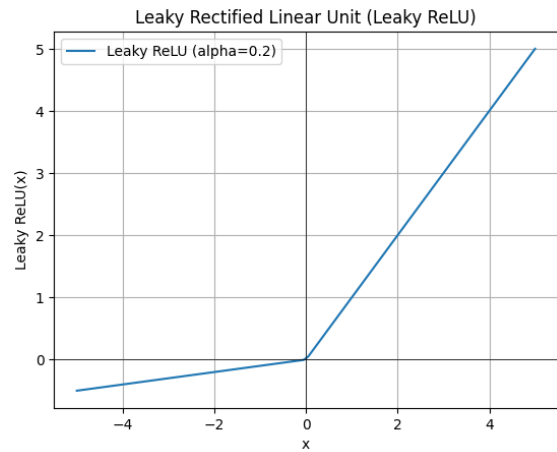


Figure 2.17. Leaky Rectified Linear Unit

G. Exponential Linear Unit (ELU):

The Exponential Linear Unit (ELU) is an activation function that addresses some of the limitations of ReLU and Leaky ReLU. It is similar to Leaky ReLU for negative inputs but differs for positive inputs. As Figure 2.18 shows, for positive values of x , ELU introduces a slight curvature, which helps mitigate the vanishing gradient problem. This slight curvature allows ELU to produce negative outputs for negative x values, helping avoid the "dying ReLU" problem. ELU is also zero-centered, which aids the optimization process in deeper networks. While ELU has shown promising results, it may be computationally more expensive than ReLU and its variants due to the exponential operation.

The exponential linear unit (ELU) is defined as:

$$f(x) = \begin{cases} a \cdot (e^x - 1), & \text{if } x \leq 0, \\ x, & \text{if } x > 0. \end{cases} \quad (2.22)$$

H. Softmax function:

The Softmax function is an activation function primarily used in the output layer of neural networks for multi-class classification tasks. It transforms the raw output scores, also known as logits, into probabilities, making it easier to interpret the model's

final predictions. The Softmax function takes an input vector and exponentiates each element to turn them into positive values, as shown in Figure 2.19. Then, it divides each exponentiated value by the sum of all exponentiated values, ensuring that the output probabilities sum up to 1. This allows the Softmax function to convert the logits into a probability distribution over multiple classes, where each value represents the probability of the input belonging to a specific class. The equation for softmax function is given by:

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad (2.23)$$

In equation (2.23), the variable x_i represents an input vector that can take any value from negative infinity to positive infinity. The term e^{x_i} represents the standard exponential function applied to the input vector x_i . Additionally, the term $\sum_j e^{x_j}$ represents a normalization term that ensures the output vector values sum up to 1 for the i -th class. This normalization step is essential for converting the raw output scores into probabilities.

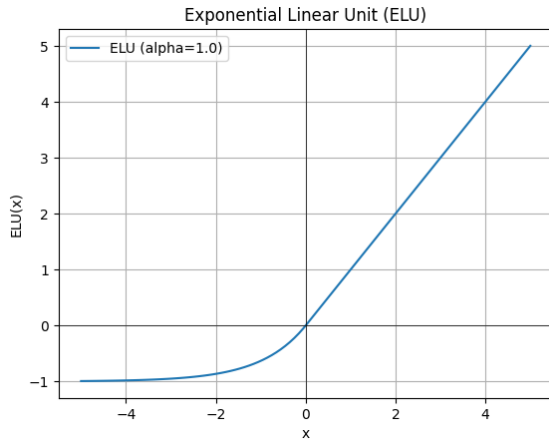


Figure 2.18. Exponential Linear Unit

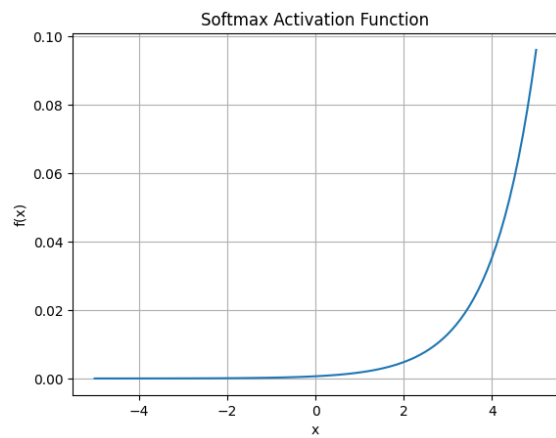


Figure 2.19. Softmax Activation Function

2.4.3. Multiple Layer Neural Network (MLPs)

A Multiple Layer Neural Network, also known as a Multi-Layer Perceptron (MLP) or Deep Neural Network (DNN), is a type of artificial neural network that consists of multiple layers of interconnected neurons. Unlike single-layer neural networks, which have only an input and output layer, MLPs contain one or more hidden layers between the input and output layers, as shown in Figure 2.20.

The key components of a Multiple Layer Neural Network are as follows:

A. Input Layer

The first layer of the network where the input data is fed. Each neuron in this layer corresponds to a feature of the input data.

B. Hidden Layers

Between the input and output layers, there can be one or more hidden layers. These layers are responsible for learning and representing complex patterns and relation-

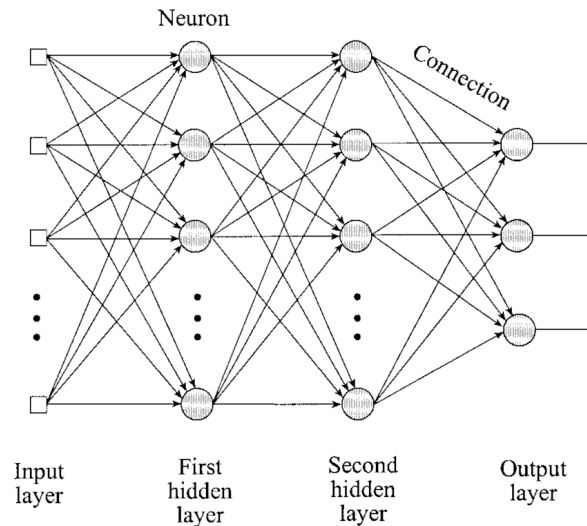


Figure 2.20. Components of Multiple Layer Neural Network [24]

ships in the data. Each neuron in a hidden layer receives inputs from the previous layer and produces outputs that are then passed to the next layer.

C. **Output Layer**

The last layer of the network that produces the final output or prediction. The number of neurons in the output layer depends on the task at hand. For binary classification, there would be one neuron, while for multi-class classification, there would be multiple neurons, each representing the probability of a specific class.

D. **Connections (Weights and Biases)**

Each neuron in the network is connected to every neuron in the adjacent layers through connections with associated weights and biases. These weights and biases are learned during the training process to enable the network to make accurate predictions.

E. **Activation Functions**

Neurons in the hidden layers and output layer are associated with activation functions, which introduce non-linearity to the network.

2.4.4. Deep Learning Algorithms

Deep Learning algorithms refer to a class of machine learning algorithms that are based on artificial neural networks with multiple hidden layers. These algorithms are designed to learn hierarchical representations of data, enabling them to model complex relationships and patterns. Some popular Deep Learning algorithms are:

A. **Convolutional Neural Networks (CNNs)**

Convolutional Neural Networks (CNNs) are a category of Deep Learning algorithms specifically designed for image and video recognition tasks. They are inspired by the visual processing mechanism of the human brain and excel at learning hierarchical representations from images. CNNs have become the backbone of many computer vision applications and have achieved state-of-the-art results in tasks such as image classification, object detection, and segmentation. As shown in Figure 2.21, CNNs are

composed of several key components that work together to extract and learn features from the input data.

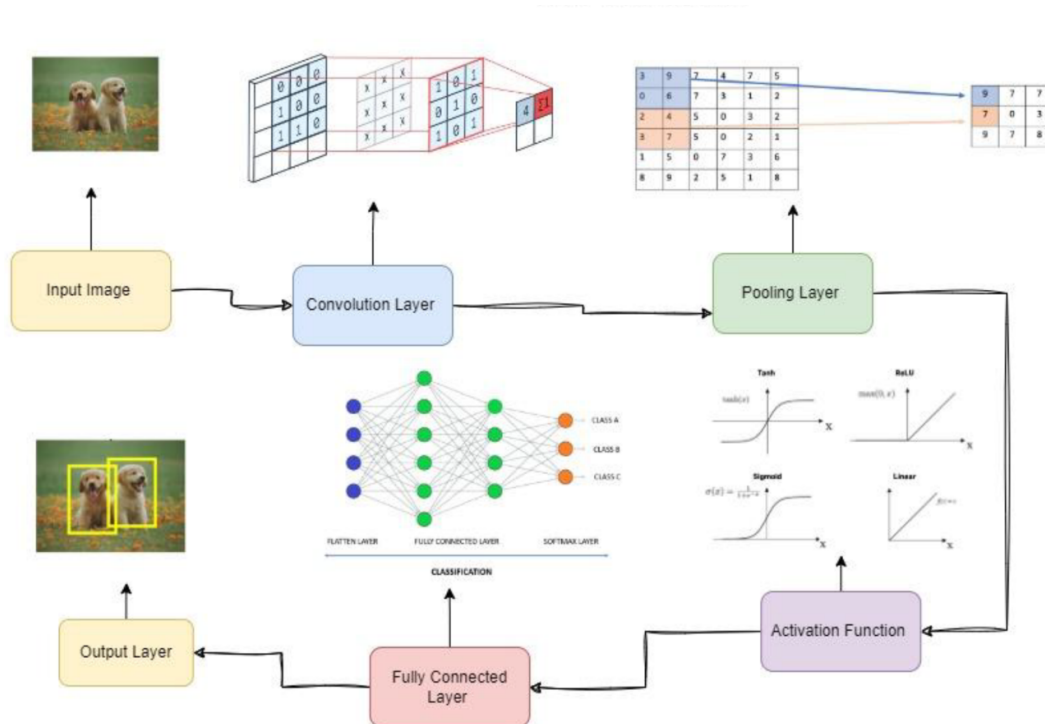


Figure 2.21. Components of Convolutional Neural Network [25]

a) Convolutional Layers

These layers consist of small filters (also known as kernels) that slide over the input image to perform convolutions. Each filter captures local patterns or features, such as edges or textures. The convolutions create feature maps that represent different aspects of the input image.

b) Pooling Layers

Pooling layers are used to downsample the spatial dimensions of the feature maps, reducing computational complexity while retaining important features. Max-pooling and average-pooling are common pooling operations.

c) Fully Connected Layers

After several convolutional and pooling layers, the feature maps are flattened and passed through fully connected layers. These layers connect every neuron from the previous layer to every neuron in the current layer, enabling high-level feature learning and classification.

B. Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are a class of Deep Learning algorithms specially designed for sequential data processing, such as time series, natural language, and speech. Unlike traditional feedforward neural networks, RNNs have recurrent connections that allow them to maintain memory of past inputs, making them suitable for tasks that involve sequential dependencies. As shown in Figure 2.22, key features of Recurrent Neural Networks include:

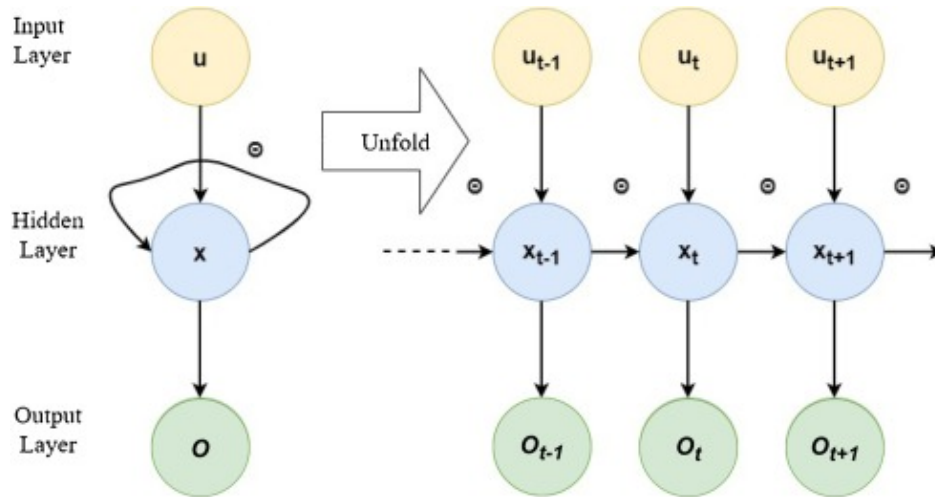


Figure 2.22. Components of Recurrent Neural Networks (RNNs) [26]

a) **Recurrent Connections**

RNNs have loops that allow information to be persisted and passed from one time step to another. This loop enables RNNs to capture temporal dependencies in sequential data, making them effective for tasks like language modeling and speech recognition.

b) **Hidden State**

At each time step, RNNs maintain a hidden state vector that serves as an internal memory. The hidden state is updated based on the current input and the previous hidden state.

c) **Vanishing and Exploding Gradient**

Training deep RNNs can suffer from the vanishing or exploding gradient problem, where gradients either become too small, causing slow convergence, or too large, leading to unstable training. To address this, variants like Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) were introduced, which help mitigate these issues.

C. Long Short-Term Memory Networks (LSTMs)

Long Short-Term Memory Networks (LSTMs) are a specialized type of Recurrent Neural Networks (RNNs) designed to address the vanishing gradient problem and effectively capture long-term dependencies in sequential data. LSTMs were introduced to overcome the limitations of traditional RNNs, which struggled to retain and propagate information over extended time sequences.

As shown in Figure 2.23, key characteristics of Long Short-Term Memory Networks include:

a) **Memory Cells**

LSTMs are built around memory cells, which allow them to store and update information over time. These memory cells are equipped with three essential gates:

- **Forget Gate**

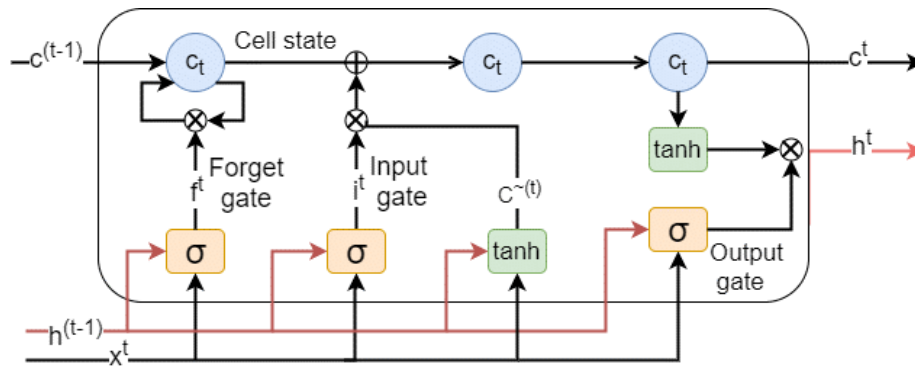


Figure 2.23. Fundamental Components of an LSTM Cell [27]

Controls what information from the previous time step should be forgotten or discarded.

- **Input Gate**

Determines which new information from the current time step should be stored in the memory cell.

- **Output Gate**

Controls which information from the memory cell should be used to produce the output.

b) **Gating Mechanisms**

The three gates in an LSTM are controlled by activation functions (usually sigmoid and tanh) that allow the network to adaptively learn how much information should be retained or forgotten at each time step.

c) **Constant Error Carousels**

LSTMs use constant error carousels, which enable them to retain gradients over long sequences, thus addressing the vanishing gradient problem.

3. State of the Art

In this section, the core principles, significant advancements, and persistent obstacles that shape the present landscape of sEMG-based hand grip recognition will be elaborated. This exploration aims to provide a comprehensive overview of the state of the art in this dynamic and rapidly evolving field.

3.1. Deep Learning

3.1.1. Convolutional Neural Network (CNN)

CNN architectures are widely used for sEMG-based gesture recognition, and studies can be categorized based on whether they focus on improving intra-session accuracy or on evaluating the model's performance across different sessions or subjects. Both types of evaluations are essential for developing reliable and practical sEMG-based gesture recognition systems [28], [29].

BAKIRCIOĞLU et al [30] aim to enhance the classification accuracy and performance of sEMG-based hand gesture recognition through the use of Convolutional Neural Networks (CNNs). The dataset used in this study contains sEMG signals collected from five subjects performing six fundamental hand movements. The signals are obtained from three surface EMG electrodes attached to the forearm, allowing the detection of different gripping movements [31].

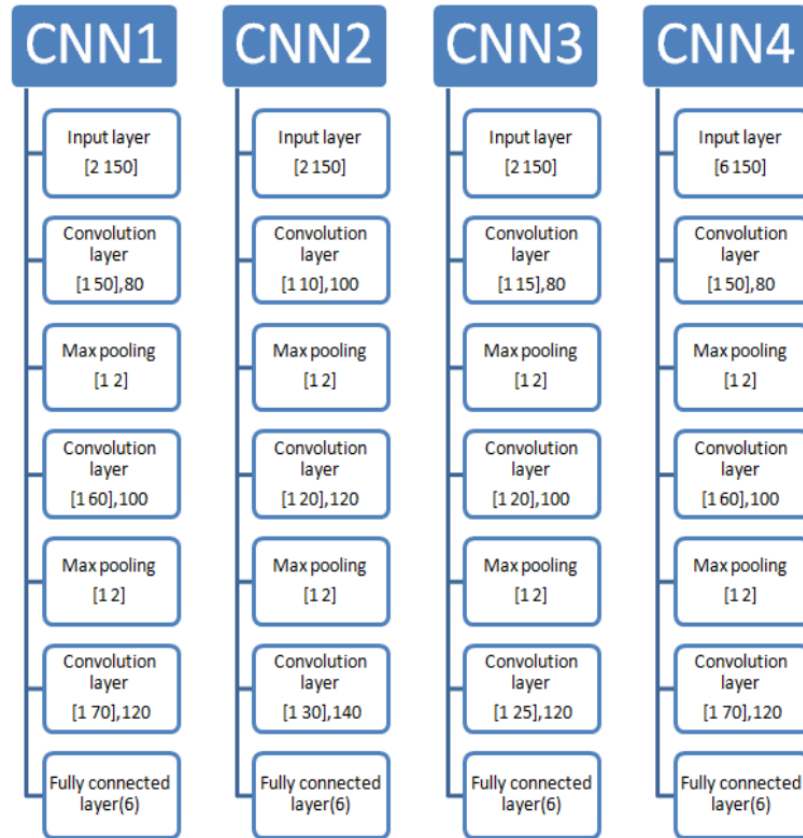


Figure 3.1. Four Different CNN Architectures [30]

The materials and methods section describes the data collection process and pre-processing steps. The sliding window technique is applied to focus on the contracted muscle segments, and various pre-processing methods, including Fourier Transform, Root Mean Square, and Empirical Mode Decomposition (EMD), are used to extract features from the signals. The data set is then divided into training, validation, and test sets, with 5-fold cross-validation applied for result analysis. As shown in Figure 3.1, four CNN architectures, namely CNN1 (using raw EMG signals with windowing), CNN2 (using signal FFT), CNN3 (using signal RMS), and CNN4 (using signal IMFs obtained through EMD), are implemented and evaluated [30].

As BAKIRCIOĞLU et al [30] discussed CNN4, with IMF features, achieves the highest classification accuracy of 0.959 using 5-fold cross-validation, outperforming the other CNN architectures. Additionally, the proposed CNN models are compared to previous studies on the same dataset. It is demonstrated that even with raw EMG signals and without extensive feature extraction, the CNN achieved an accuracy of 0.902.

Additionally, Atzori et al [28] present a literature review on the application of convolutional neural networks (CNNs) to classify hand movements using electromyography (sEMG) data. The study uses data from the Ninapro database, which includes sEMG data from 78 subjects (11 transradial amputees and 67 intact subjects) divided into three datasets. Dataset 1 consists of 27 intact subjects, dataset 2 consists of 40 intact subjects, and dataset 3 consists of 11 transradial amputees [32].

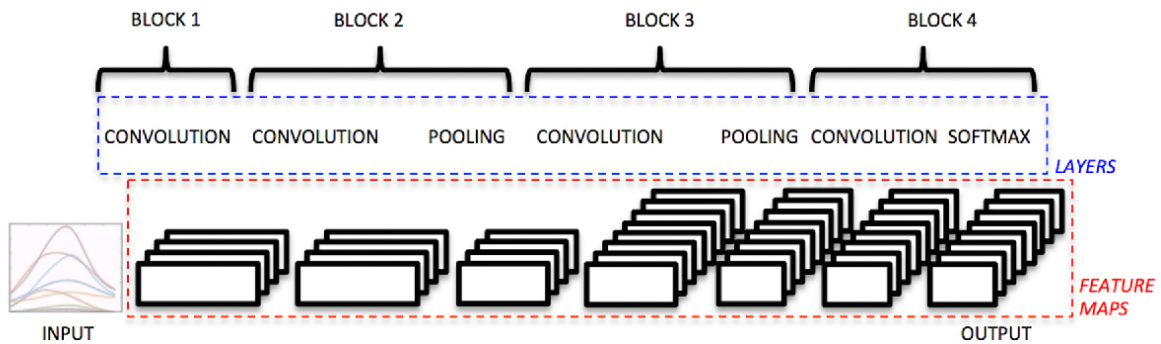


Figure 3.2. CNN Structure Utilized for the Classification of the sEMG Signal [28]

The data analysis aims to classify hand movements into more than 50 classes using CNNs and compare the results with classical machine learning techniques. The data are pre-processed, synchronized, and labeled using a generalized likelihood ratio algorithm. As shown in Figure 3.2 the CNN architecture used is a modified version of LeNet [28].

The results of Atzori et al [28] show that the classification accuracy obtained with CNNs using the proposed architecture is comparable to the average results obtained from classical classification techniques but lower than the best classical methods. The average accuracy on dataset 1 is 0.6659 ± 0.064 , on dataset 2 is 0.6027 ± 0.077 , and on dataset 3 (amputees) is 0.380 ± 0.1429 . The study discusses the factors influencing classification accuracy, including network architecture, pre-processing, and optimization parameters.

3.1.2. Parallel Block CNN

Ding et al [33] present a deep convolutional network (CNN) architecture for sEMG-based hand gesture recognition using the Ninapro project's DB2 database. The dataset includes 40 subjects performing 50 hand, wrist, and functional movements. The sEMG signals are recorded using Trigno wireless electrodes and processed through various steps such as filtering, synchronization, and relabeling [32], [33].

As shown in Figure 3.3, the proposed CNN architecture, named Convolution with two Parallel Blocks (C-B1PB2), consists of two parallel blocks for feature extraction and classification. The feature extractor includes convolution and pooling layers, while the classifier has fully connected layers. The CNN architecture is compared with other reference experiments, such as Classical Classification (CC) and other CNN-based methods [33].

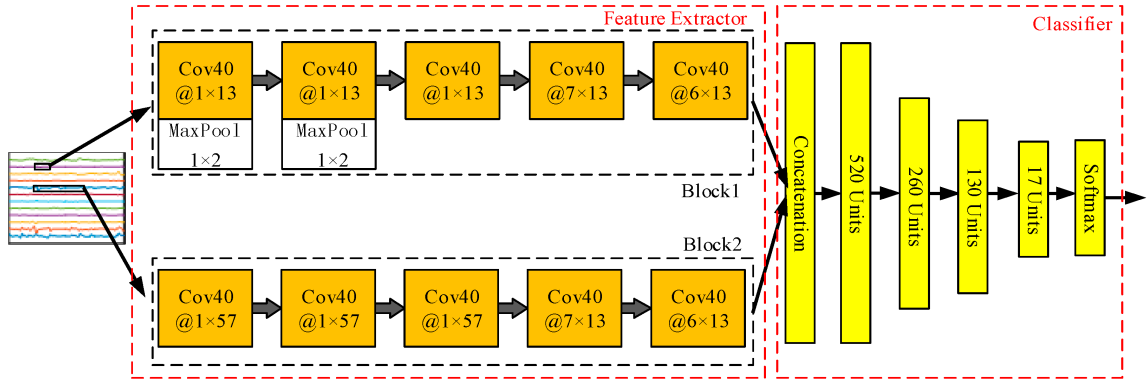


Figure 3.3. CNN Structure Utilized for the Classification of the sEMG Signal [33]

In their comparative evaluation, the CNN achieved an accuracy of 78.71%, while the Random Forest (RF) model reached 75.27%. Additionally, the CNN (Single Frame) and CNN (20 Frames) models achieved accuracies of 76.1% and 77.8%, respectively. The k-Nearest Neighbors (KNN) model yielded an accuracy of 70%, and the Support Vector Machine (SVM) reached 77.44%. The results show that the C-B1PB2 method achieves the highest classification accuracy compared to other methods. It outperforms Classical Classification and other CNN architectures, demonstrating the effectiveness of CNNs in sEMG-based gesture recognition. Additionally, the results show that considering the size of the receptive field (kernel filter) and muscle independence is crucial for improving classification accuracy [33].

However, the proposed model still has some limitations. First, the dataset used may not fully represent all possible hand gestures, which could affect the generalizability of the model to real-world scenarios. Second, the study does not consider preprocessing techniques such as low-pass filtering and fast Fourier transform, which are common in sEMG signal processing. These preprocessing steps can help reduce noise and improve classification performance in some cases. Finally, while the C-B1PB2 architecture performs well on the DB2 dataset, its performance on other datasets or in real-time applications remains to be validated.

3.1.3. Feedforward Neural Network (FFNN)

Feedforward Neural Networks (FFNNs) have a rich history, from the McCulloch-Pitts neuron in 1943 to the development of backpropagation in the 1980s, leading to the deep learning revolution in the 2010s [34].

As shown in Figure 3.4, [35] used a comprehensive approach, starting with collecting sEMG signals and elbow joint angle data during forearm exercises. To ensure data accuracy, sEMG signals were carefully filtered, and precise joint angle measurements were obtained. To address muscle activation timing, the sEMG data was segmented in the time domain using specific parameters. The extracted signal features were categorized based on their characteristics, including time and frequency domains, events, integrals, and domain knowledge. These features were then used as input for training a feedforward neural network (FFNN) to predict elbow joint angles.

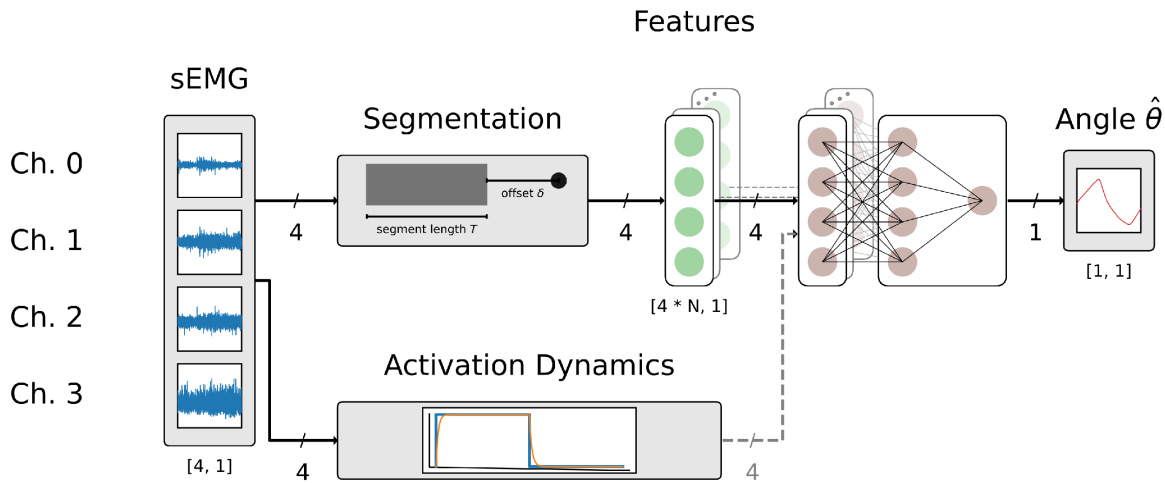


Figure 3.4. Block Diagram for Prediction Using a Feedforward Neural Network [35]

In the [35], the performance of elbow-joint movement prediction using a Feedforward Neural Network (FFNN) architecture was rigorously evaluated. Single sEMG features were examined to identify those yielding the highest predictive accuracy, with the IEMG feature standing out as the best performer across various segment lengths. Segmentation parameters, specifically segment length and offset, were explored, with a segment length of 0.6 seconds and an offset of 0.3 seconds yielding the best predictive results. Additionally, multi-feature sets were constructed, with the combination of IEMG, MNE, and ACT features resulting in superior predictive accuracy compared to commonly used feature sets. This study underscores the importance of feature selection and segmentation parameters in enhancing sEMG-based limb movement prediction, showcasing the potential for more accurate predictive models beyond conventional methods.

3.1.4. Extreme Learning Machine (ELM)

Peng et al. [36] proposed a gesture recognition method based on surface electromyography (sEMG) signals. As shown in Figure 3.5, the method consists of several key components: signal preprocessing, feature extraction and selection, and an ensemble Extreme Learning

Machine (ELM) classifier. In the signal preprocessing step, the sEMG signals were filtered using a Butterworth high-pass filter and segmented into windows to convert sequential data into frame data, making them suitable for further analysis. Feature extraction involves computing 16 time and frequency domain features from each signal channel, which are commonly used in sEMG analysis. To reduce redundancy and improve efficiency, the authors employed a feature selection technique called RF-RFE (Random Forest - Recursive Feature Elimination) to select the most informative features.

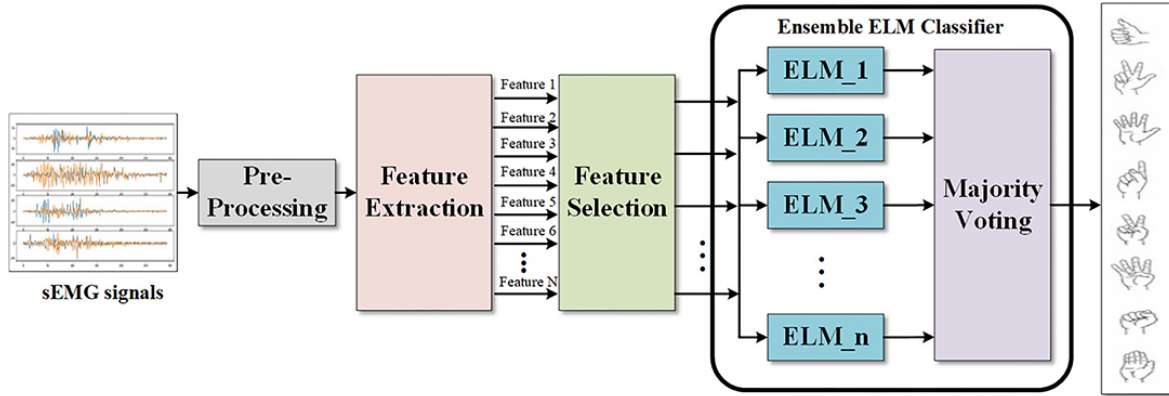


Figure 3.5. ELM Block Diagram to Classify sEMG Signal [36]

In this paper, multiple ELM base learners were constructed, and their results were combined using the majority voting method to enhance recognition accuracy and stability. The experimental results on the Ninapro DB5 dataset, which includes 52 different hand movements, demonstrated the effectiveness of the proposed approach. The findings revealed that the feature selection process were significantly improved gesture recognition performance across different classification methods (Decision Tree, ELM, Random Forest, and EELM). Moreover, the proposed EELM method outperforms the other methods and achieves the best accuracy (77.9%) for recognizing hand movements. The paper also discusses the real-time applicability of the method and its computational efficiency, indicating its suitability for practical applications [36].

3.1.5. RNN

Bagherian Azhiri presents a new architecture for real-time classification of EMG (Electromyography) signals using recurrent neural networks (RNNs) with discrete wavelet transform-based feature extraction. In this literature, EMG signals are divided into batches of consecutive samples (data windowing), with each window containing 400 samples with a 200-sample overlap with the previous window. The wavelet transform features are calculated for each window and used as inputs to the RNN structure. The authors consider a 2-level discrete wavelet transform decomposition using a mother wavelet, which is a hyperparameter selected empirically [37], [38].

Recurrent neural networks (RNNs) are utilized in the architecture due to their ability to process sequences of variable lengths. As shown in Figure 3.6, the authors introduce two basic units (BU1 and BU2) that form the building blocks of their RNN and bidirectional

RNN (BRNN) architectures. BU1 takes two inputs and returns two outputs, while BU2 takes three inputs and returns three outputs. The original RNN and BRNN are constructed by stacking these basic units. The number of basic units used in each architecture is a hyperparameter tuned empirically, and two different input approaches are considered: "Same Inputs" and "Sequential Inputs" [37].

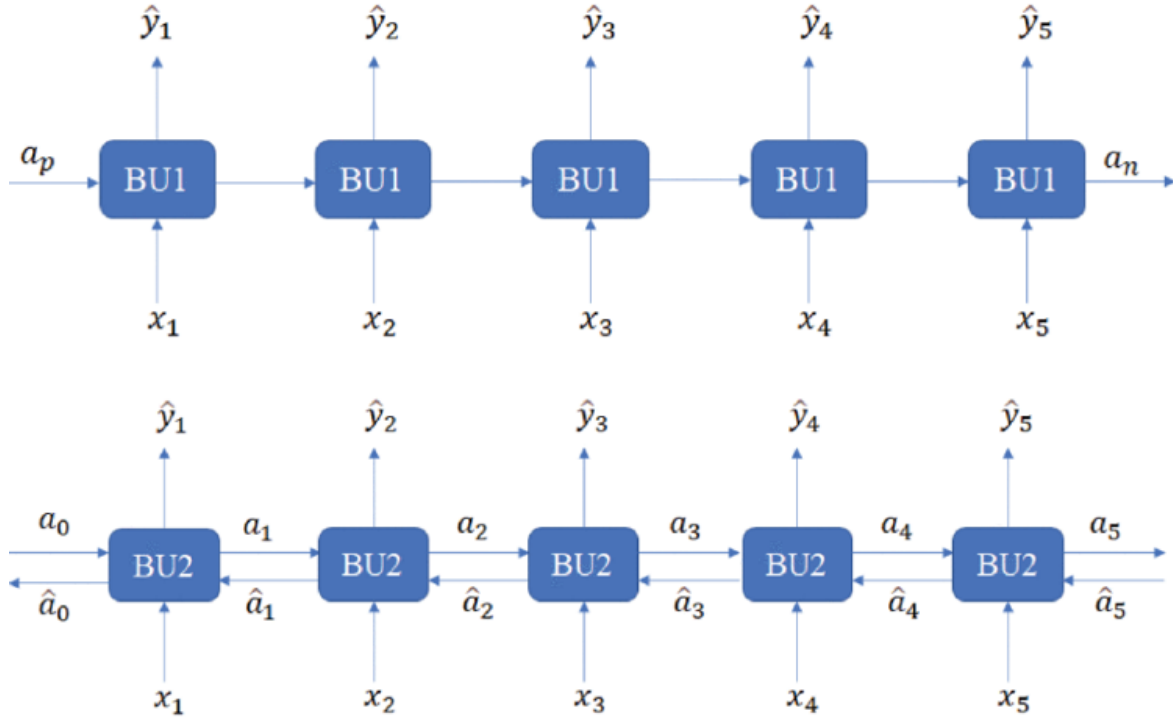


Figure 3.6. RNN and BRNN Architectures to Classify sEMG Signal [37]

The proposed architectures are evaluated on two datasets containing EMG data of finger movements. The results show that the BRNN with the same inputs achieves the highest accuracy in the shortest signal length. For instance, for the first dataset, BRNN with the same inputs achieves the highest accuracy in 600 msec, outperforming other proposed RNN architectures. This outcome is attributed to the two-sided connectivity of basic units in the BRNN architecture, which enables better capture of input relationships and feature sharing among units, reducing misclassification and improving overall accuracy. For the eight-channel (8C) dataset, both BRNN with the same inputs and RNN with sequential inputs exhibit better performance than other architectures, achieving an accuracy of 93.3% in 500 msec. The uniform accuracy observed for different finger movement classes underscores the stable performance of the BRNN architecture. Additionally, the proposed architecture's performance is compared with other state-of-the-art approaches. The results demonstrate competitive accuracy, especially in real-time applications, where the proposed architecture outperforms other methods in terms of accuracy and signal length. The delay time for the system is provided, accounting for feature extraction and classification times for the 2C dataset using BRNN at 600 msec signal length, totaling approximately 7.84 msec [37].

3.1.6. LSTM

Millar et al [39] discussed the classification of EMG signals in real-time using LSTM network architecture. Long Short-Term Memory (LSTM) is a recurrent neural network (RNN) that can handle sequences of variable lengths. It uses gated cells to control the flow of data, allowing it to learn bi-directional long-term dependencies between time steps [40].

As shown in Figure 3.7, the LSTM network used in [39] comprises six layers, including a sequence input layer, a bidirectional LSTM layer, a dropout layer for regularization, a fully connected layer, a softmax layer for activation, and a classification layer for mutual exclusive classes. The LSTM network was optimized by adjusting the number of hidden units, dropout rate, and additional layers. They began with base parameters and incrementally adjusted hidden units and added bi-LSTM layers. Their best results came from a two-layer LSTM network with 170 and 60 hidden units, achieving 91.76% accuracy. Thumb movements were challenging to classify, with 61.5% and 71.4% accuracy for thumb extension and flexion. Overall, their approach significantly outperformed previous research, achieving an average accuracy of 86.76% and individual networks reaching 90%. [39], [41].

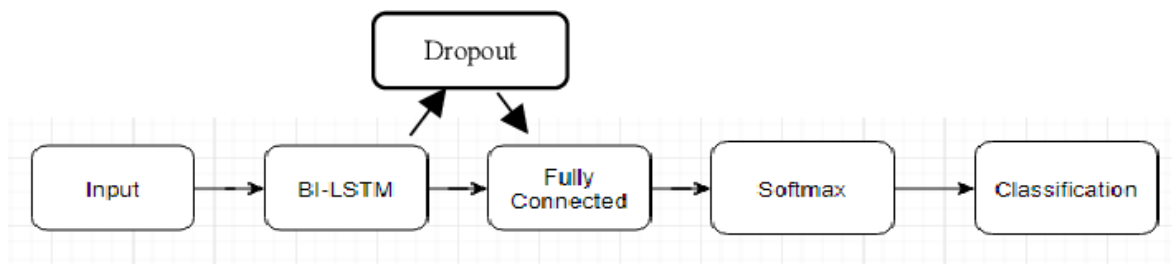


Figure 3.7. LSTM Architecture for Real-Time EMG Signal Classification [39]

As compared to LSTM, RNNs are able to process sequences of variable lengths, making them suitable for time-series data like EMG signals. LSTM addresses the vanishing gradient problem of traditional RNNs by using gated cells that enable the network to retain important information over longer sequences. This capability is crucial for capturing temporal dependencies in EMG signals, which can help improve classification accuracy [42].

3.2. Traditional Machine Learning

3.2.1. SVM

SVMs are commonly used for classifying things like body movements, pictures, and sounds. They create a special line or plane in a space with lots of features, kind of like different aspects we're looking at. This helps tell things apart, even when they're not easily separated. The idea came from Vapnik and Corina in 1995 [43], [44].

Chen et al [45] summarized a study focusing on hand gesture recognition via SVM and surface electromyography (sEMG) data from the Myo armband device. The wearable device incorporates 8 sEMG channels, capturing muscle activity from the skin's surface. The research aimed to classify five gestures using data from 9 healthy participants who executed each gesture five times. Preprocessing involved denoising sEMG signals and

applying Fourier and Butterworth filters. Feature extraction was optimized by segmenting datasets and utilizing sliding sub-windows, leading to an improved feature vector with enriched bag of functions.

As shown in Figure 3.8, SVM constructs hyperplanes in a high-dimensional space to achieve classification. [45] employed Gaussian radial basis function as the kernel function to deal with non-linear operations. The proposed classifier achieves an overall recognition accuracy of 0.89 for the 5 hand gestures. The gesture “fist” has the highest accuracy of 0.93. The proposed model outperforms other methods, including Myo Armband self-method, k-NN and DTW-based model, and linear SVM-based model with common features. In

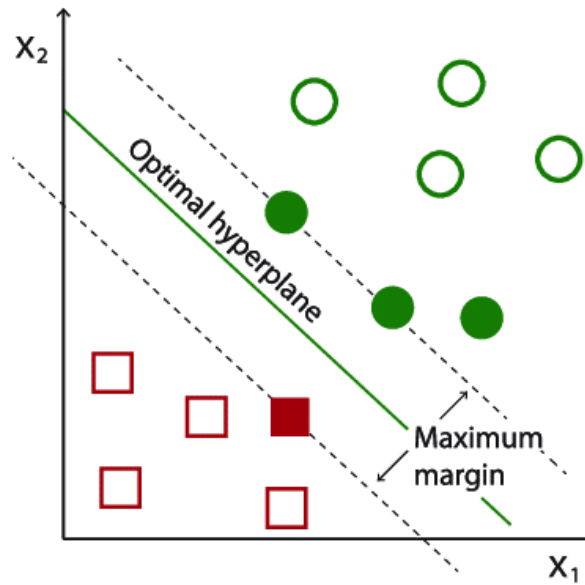


Figure 3.8. Geometry of Support Vector Machine (SVM) [43]

the context of Support Vector Machines (SVM), the training process is reformulated into a Quadratic Programming (QP) problem, yielding a unique solution. When handling training data pairs $(x_1, y_1), \dots, (x_m, y_m)$, where x_i denotes an input and y_i its class label (-1 or $+1$), the scenario of non-linearly separable data entails a nonlinear transformation $\phi: \mathbb{R}^N \rightarrow \mathbb{R}^M$. This mapping places the data within a novel space $\mathbb{R}^M$, achieving linear separability [43]. The hyperplane distinguishing various object types is expressed as:

$$\omega \cdot \phi(x) + b = 0, \quad (3.1)$$

where $\omega \in \mathbb{R}^M$ and $b \in \mathbb{R}$. The key QP objective is to construct an optimal hyperplane maximizing separation margin while considering error margins $\xi = (\xi_1, \dots, \xi_m)$. The optimization problem is:

$$\min_{\omega, b, \xi} \frac{1}{2} \|\omega\|_2^2 + C \sum_{i=1}^m \xi_i, \quad (3.2)$$

subject to

$$y_i(\omega \cdot \phi(x_i) + b) \geq 1 - \xi_i, \quad i = 1, \dots, m. \quad (3.3)$$

In cases of closely spaced data points posing separation challenges, a kernel function K is introduced for effective separation. This leads to a dual formulation of the optimization problem:

$$F(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{j,k=1}^m \alpha_j \alpha_k y_j y_k K(x_j, x_k), \quad (3.4)$$

subject to

$$\sum_{i=1}^m y_i \alpha_i = 0, \quad (3.5)$$

$$0 \leq \alpha_i \leq C, \quad i = 1, \dots, m. \quad (3.6)$$

The kernel function $K(x_j, x_k)$ takes diverse forms, like the Radial Basis Function (RBF), Gaussian, polynomial, etc. For instance, a polynomial kernel is:

$$K_P(x_j, x_k) = (1 + x_j \cdot x_k)^d. \quad (3.7)$$

The RBF kernel connects feature vectors via:

$$K_R(x_j, x_k) = \exp(-\gamma \|x_j - x_k\|_2^2), \quad (3.8)$$

where $\gamma = \frac{1}{2\sigma^2}$ and σ is the standard deviation.

For SVM's multi-class classification, two strategies are used: One Against One (OAO) and One Against All (OAA). OAO evaluates classes individually through pairwise comparisons, while OAA isolates each class for separate classification [43].

3.2.2. KNN

The first kNN algorithm was suggested by Cover and Hart in 1968. Later on, people looked at it closely and studied it deeply in theory. This is one of the most important ways to work with data that doesn't depend on specific parameters [46], [47].

Haries et al. [48] collected EMG data from four individuals, investigating nine finger movements using a 2-channel kit with three electrodes. Signals from muscles like the brachioradialis were recorded, with each gesture repeated six times for a duration of five seconds. Preprocessing involved noise reduction using a third-order Butterworth filter, followed by feature computation such as root mean square and zero crossings to enhance classification accuracy.

As shown in Figure 3.9, [48] utilized two classification algorithms which were k-Nearest Neighbors (k-NN) and Artificial Neural Networks (ANN). The k-NN approach utilized various distance metrics, including Euclidean distance (Ed), cosine distance (Cd), correlation distance (Cod), and city block distances (Cbd) for classification. On the other hand, the ANN implementation involved a backpropagation neural network with input, hidden, and output layers.

The distance metrics are defined as follows:

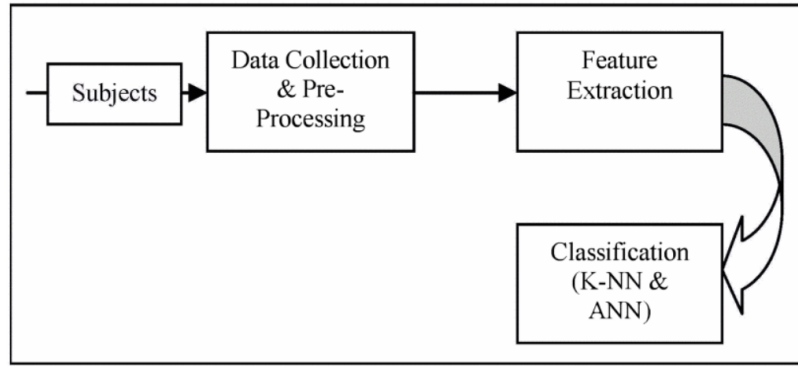


Figure 3.9. Block Diagram for KNN and ANN Classification [48]

$$Ed = \frac{1}{N} \sqrt{N \sum_{i=1}^N (C_k - C_j)^2} \quad (3.9)$$

$$Cd = 1 - \frac{(C_k - \bar{C}_k)(C_j - \bar{C}_j)}{\sqrt{(C_k - \bar{C}_k)(C_k - C_k)(C_j - \bar{C}_j)(C_j - C_j)'}} \quad (3.10)$$

$$Cod = 1 - \frac{C_k C_j'}{\sqrt{(C_k C_j)(C_l C_l)'}} \quad (3.11)$$

$$Cbd = \sum_{k=1}^N |C_k - C_j| \quad (3.12)$$

Where C_k and C_j correspond to data columns or channels within EMG signal data.

Testing revealed that k-NN achieved an accuracy of approximately 0.86, while ANN, particularly when using the Levenberg-Marquardt training algorithm, achieved an accuracy of about 0.93. The study concluded that EMG-based recognition of finger movements was feasible, with ANN outperforming k-NN, LVQ, and Adaboost classifiers. It also highlighted opportunities for improvement by enhancing signal quality and expanding the dataset [48].

3.2.3. Decision Tree and Random Forest

Decision Trees are a fundamental machine learning and data mining technique that have been developed and refined by various researchers and practitioners over several decades. For example, the Classification and Regression Tree (CART) and C4.5 are popular decision tree algorithms that are developed by Quinlan, [49], [50].

Ercan et al. [51] worked on the classification of Electromyography (EMG) signals for diagnosing neuromuscular disorders using decision tree algorithms. EMG data were collected from three groups: healthy individuals (control group), individuals with myopathy, and individuals with Amyotrophic Lateral Sclerosis (ALS). As referred in Figure 3.10, [51] employed Multi-scale Principal Component Analysis (MSPCA) for denoising the EMG signals and Discrete Wavelet Transform (DWT) for feature extraction. Then three different classification algorithms, namely CART, C4.5, and Random Forest, were applied to catego-

rize the EMG signals into their respective groups. The study used K-fold cross-validation to evaluate the performance of these classifiers. The results demonstrated that Random Forest achieved the highest classification accuracy, reaching 96.67%, followed by C4.5 with 91.11%, and CART with 90.88%.



Figure 3.10. Block Diagram for Decision Trees Classification [51]

Generally, [51] highlights the efficiency of Random Forest classification in diagnosing neuromuscular disorders using intramuscular EMG signals. It emphasizes the importance of preprocessing techniques such as MSPCA in enhancing classification accuracy, making the proposed method a valuable tool for clinicians for the accurate diagnosis of these disorders.

Additionally, [52] focused on identifying surface EMG signals using the Random Forest algorithm. It highlights the design of a crucial signal acquisition and conditioning circuit, which includes primary amplification, high-pass and low-pass filtering, secondary amplification, and industrial frequency interference elimination. The paper also discusses the importance of feature selection, specifically time-domain features like Root Mean Square, Mean Absolute Value, Waveform Length, Zero Crossing, and Signal Slope Change, which are chosen for characterizing hand gestures. [52] chose the Random Forest model for classification due to its effectiveness with complex datasets, outperforming other models like Linear Discriminant Analysis, Support Vector Machines, Nearest Neighbor algorithm, and Decision Trees. In summary, it presents a strong method for hand gesture identification using surface EMG signals, emphasizing signal acquisition, feature extraction, and Random Forest classification. The results affirm its suitability for applications like myoelectric prosthesis control.

3.2.4. Native Bayes Classifier

Naïve Bayes, a key classification algorithm in machine learning and statistics, was developed in the mid-20th century within the realm of statistical pattern recognition, with foundational contributions from researchers like T. J. Hooper and Robert Duda [53].

Pamungkas et al. [54] discussed hand gesture recognition using surface EMG (sEMG) signals to control devices such as prosthetic hands and robots. It evaluates the effectiveness of two algorithms, Naïve Bayes and K-NN, for gesture recognition based on Root Mean Square (RMS) in the time domain. The experiments involve five typical hand gestures, and

sEMG data is captured using a Myo armband. As referred in Figure 3.11, the recognized gestures are utilized to control a customized mobile robot.

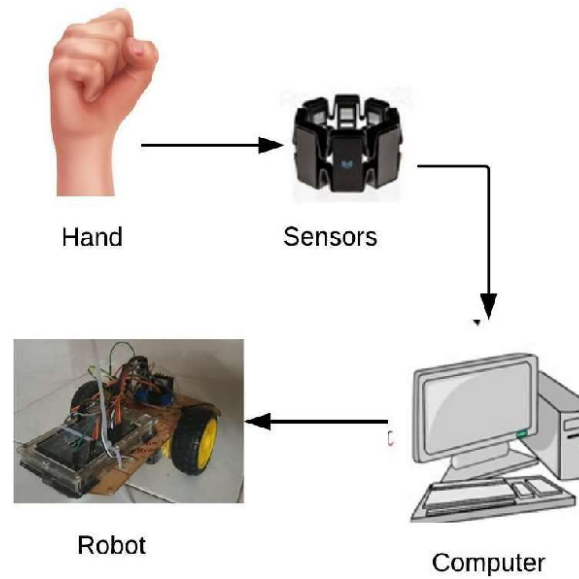


Figure 3.11. Block Diagram for Native Bayes Classification [54]

Results showed that both algorithms can effectively recognize hand gestures, with Naïve Bayes slightly outperforming K-NN by about 2%. Furthermore, Naïve Bayes is found to be more responsive and faster in controlling the mobile robot, completing tasks about 7% faster than K-NN.

3.2.5. Extreme Value Machine (EVM)

Extreme Value Machine (EVM) was explored by [55] as a promising classifier for ElectroMyoGraphy (EMG) datasets. EMG signal processing is essential for real-time gesture recognition based on the correlation between EMG signals and finger movements. As shown in Figure 3.12, the study focused on extracting features from EMG signals using reflection coefficients derived from an auto-regressive model. These coefficients helped to capture the frequency properties of EMG signals. The dataset employed in the experiments comprises EMG data collected from eight participants who performed ten distinct finger movements.

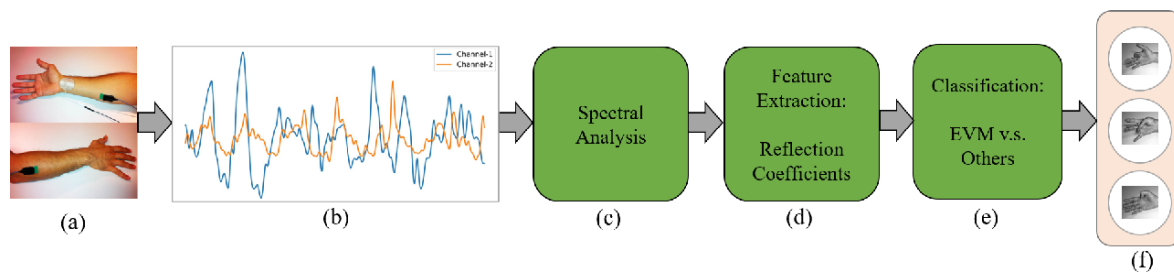


Figure 3.12. Block Diagram for EMV Classification [55]

In this study the EVM classifier was compared to seven conventional classifiers, including Support Vector Machines (SVM), k-Nearest Neighbors (KNN), and Decision Trees (DT),

among others. The results demonstrated that EVM outperforms other classifiers, achieving an average accuracy of 91%. This accuracy highlights EVM's potential in EMG signal classification. This study concentrated on all ten movements, and it's noteworthy that the achieved accuracy is obtained without the use of additional processing mechanisms such as feature selection or fusion.

3.3. Hybrid Systems

3.3.1. CNN-SVM

Chen et al. [56] involved eight healthy subjects and aimed to explore the effectiveness of a synthetic approach that combines Convolutional Neural Networks (CNNs) for feature extraction and Support Vector Machine (SVM) classifiers for classification based on sEMG data. As shown in Figure 3.13, the synthetic approach used a CNN for feature extraction, with a detailed description of the network's architecture. The CNN was followed by an SVM classifier for gesture recognition.

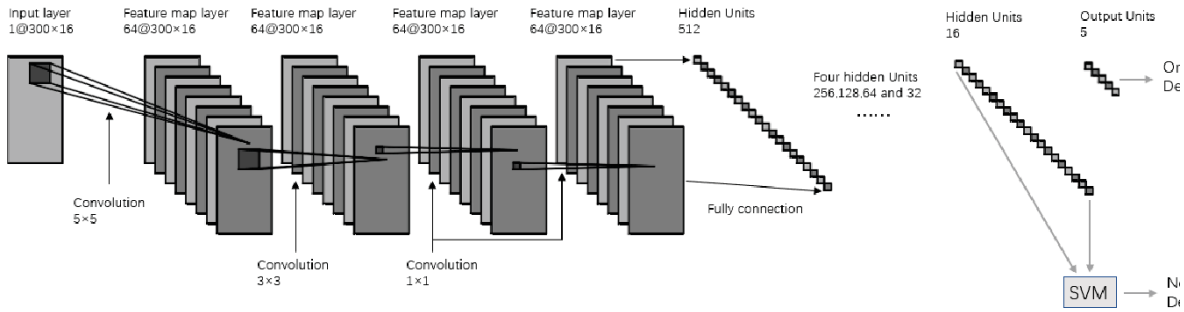


Figure 3.13. Hybrid CNN-SVM Architecture for sEMG Signal Classification [56]

Generally, [56] evaluated gesture recognition performance in an inter-subject scenario, where one subject's data was used for testing, and the rest were used for training. The results showed that the synthetic approach combining CNN and SVM outperformed traditional methods based on time-domain features (MAV, ZC, SSC). Compared to using CNN in isolation, the synthetic approach yielded an improvement in accuracy of approximately 2.5%. Moreover, when compared with traditional methods, characterized by the use of time-domain features and an SVM classifier, the synthetic approach showcased an impressive advantage, achieving a remarkable boost in accuracy of approximately 9.7%. In the inter-session evaluation, different electrode biases were introduced, and the position of the electrodes changed during data acquisition. The synthetic approach again showed superior performance compared to CNN and traditional methods, particularly for finger movement data.

3.3.2. CNN-RNN

Hu et al [57] proposed attention-based hybrid CNN-RNN architecture for sEMG-based gesture recognition. The attention mechanism is inspired by the human perception of paying attention to different regions and is applied to model the importance of muscles involved in hand movements [57], [58].

3. State of the Art

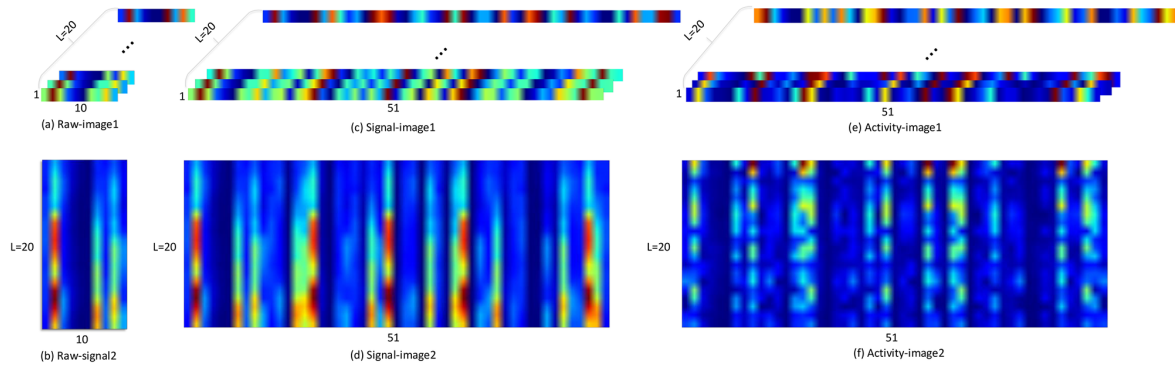


Figure 3.14. Six Different sEMG Raw Signal Used for Classification [57]

As shown in Figure 3.15, the proposed architecture consists of two stages: feature extraction and attention-based sequential modelling. The feature extraction stage converts sEMG signals into sEMG images using various methods, as shown in Figure 3.14. The CNN model is then applied to extract useful features from the sEMG images. The attention-based sequential modeling stage incorporates RNN units with attention to improve the performance of the hybrid CNN-RNN architecture. The proposed architecture combines the strengths of CNN and RNN models to effectively capture spatial-temporal information in sEMG signals [57].

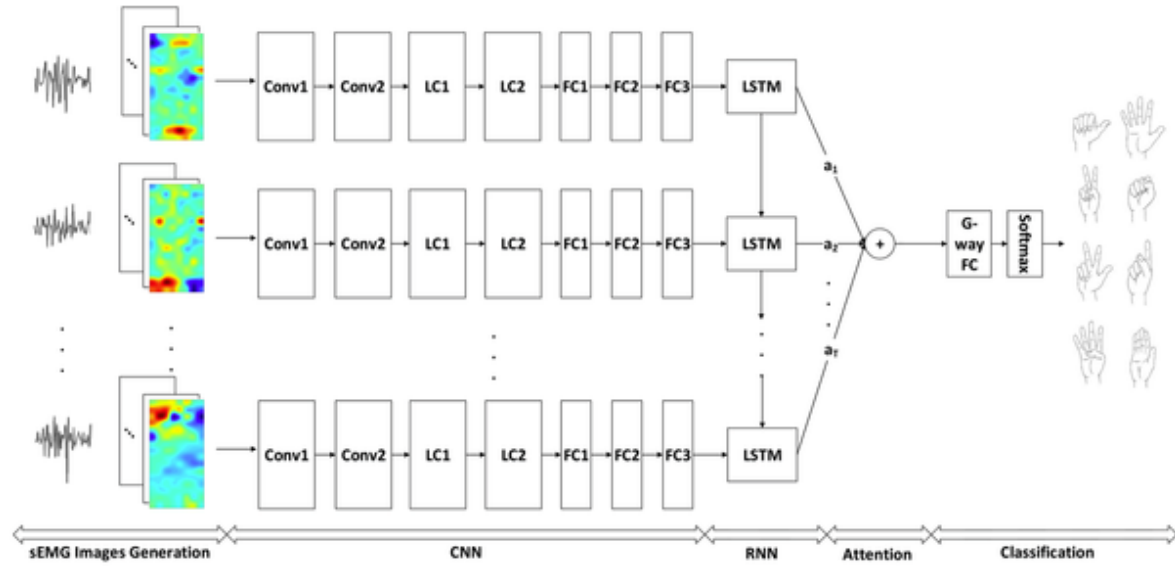


Figure 3.15. Hybrid CNN-RNN Architecture for sEMG Signal Classification [57]

The evaluation of the proposed architecture was performed on five sEMG benchmark databases, including NinaProDB1, NinaProDB2, BioPatRec26MOV, CapgMyo-DBa, and csl-hdemg. Remarkably, the proposed method achieved recognition accuracies of 87.0% for NinaProDB1, 82.2% for NinaProDB2, 94.1% for BioPatRec26MOV, 99.7% for CapgMyo-DBa, and 94.5% for csl-hdemg. Notably, these results represented improvements of 9.2% for NinaProDB1, 3.5% for NinaProDB2, 1.2% for BioPatRec26MOV, 0.2% for CapgMyo-DBa, and 5.2% for csl-hdemg. The model included an attention mechanism, allowing intelligent focus on specific segments of the electromyogram signal. The results showed that

the attention-based hybrid CNN-RNN architecture outperforms other deep learning approaches and traditional machine learning methods on these databases [29], [57], [59]–[61]. The study also explored different image representation methods for sparse multi-channel sEMG databases and found that the feature-signal-image1 method performs the best in improving recognition accuracy [57].

3.3.3. CNN-LSTM

Naveen et al. [62] discussed the implementation and evaluation of the proposed Hybrid EMGHandNet model, which is a combination of Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (Bi-LSTM) networks for the classification of hand activities based on surface electromyography (sEMG) signals, as shown in figure 3.16. As referred in Figure 3.17, the model aims to bypass the need for theoretical neuro-mechanical models and uses machine learning to learn the mapping of sEMG signals to limb kinematics.

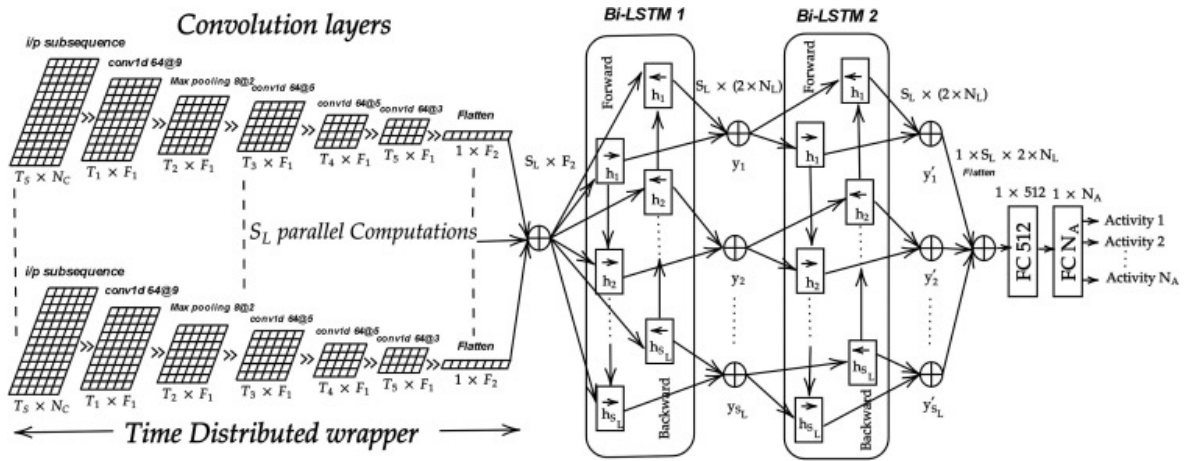


Figure 3.16. Hybrid CNN-LSTM Architecture for sEMG Signal Classification [62]

The model is evaluated on five publicly available benchmark datasets: NinaPro DB1, NinaPro DB2, NinaPro DB4, BioPatRec DB2, and UCI Gesture. These datasets contain hand activities categorized into gestures, wrist movements, grasping objects, and hand movements, with sEMG sensors placed on various muscle locations on the upper limbs [62], [63].

The EMGHandNet model was compared with several other models, including MsCNN, EvCNN, CNNLM, and KNN classifier using energy features, and evaluated using subject-wise analysis and aggregated data analysis. The subject-wise analysis involved training and testing the model with subject-specific data, while the aggregated data analysis used data from multiple subjects. The proposed Hybrid EMGHandNet model exhibited superior performance, boasting an average classification accuracy improvement ranging from 4% to a remarkable 18.65%. The model's performance is influenced by input shape, preprocessing methods, and the choice of Bi-LSTM layers [62].

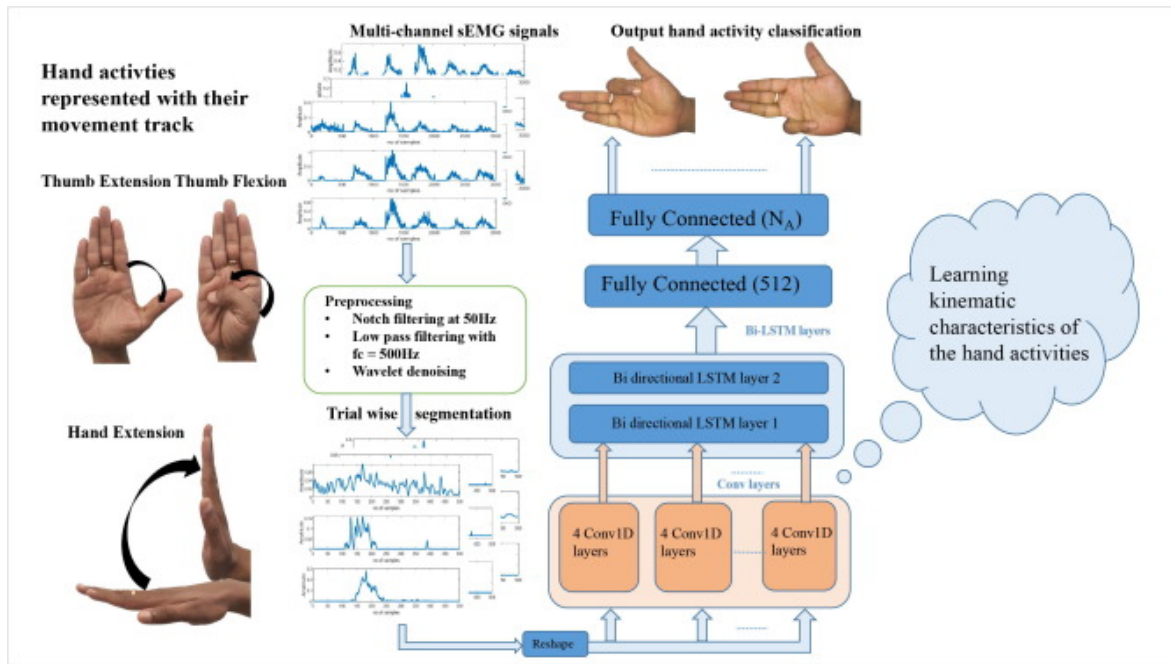


Figure 3.17. Block Diagram for CNN-LSTM Classification [62]

3.3.4. HMM-SVM

Rossi et al. [64] explored hand gesture classification using Electromyography (EMG) signals, employing Hidden Markov Models (HMM) and Support Vector Machines (SVM) for enhanced accuracy. As shown in Figure 3.18, the EMG data acquisition involved Ottobock 13E200 electrodes, connected to a differential amplifier and an ARM CORTEX M4 microcontroller, with Bluetooth transmission to a laptop. Six distinct hand gestures were studied through a systematic acquisition protocol, each recorded with 10 repetitions to minimize learning effects.

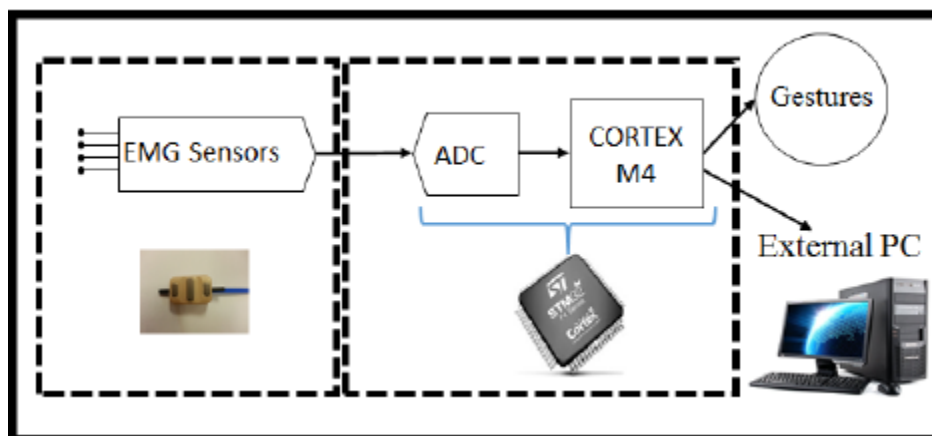


Figure 3.18. EMG Data Acquisition for HMM-SVM Classification [64]

As shown in Figure 3.19, an analysis was conducted by [64] to enhance classification accuracy by identifying characteristics of the signal prone to misclassification. Myoelectric signals were categorized into two states: steady state (constant muscle contraction/relaxation) and transient state (muscle transitioning between rest and voluntary contrac-

tion). Observations revealed that steady-state data achieved significantly higher accuracy compared to transient data, in line with prior research. To ensure robust and reliable classification, a two-step approach was implemented. Hidden Markov Models (HMM) distinguished between steady-state and transient data, while Support Vector Machines (SVM) were exclusively used for EMG signal classification during the steady-state phase [65]–[68].

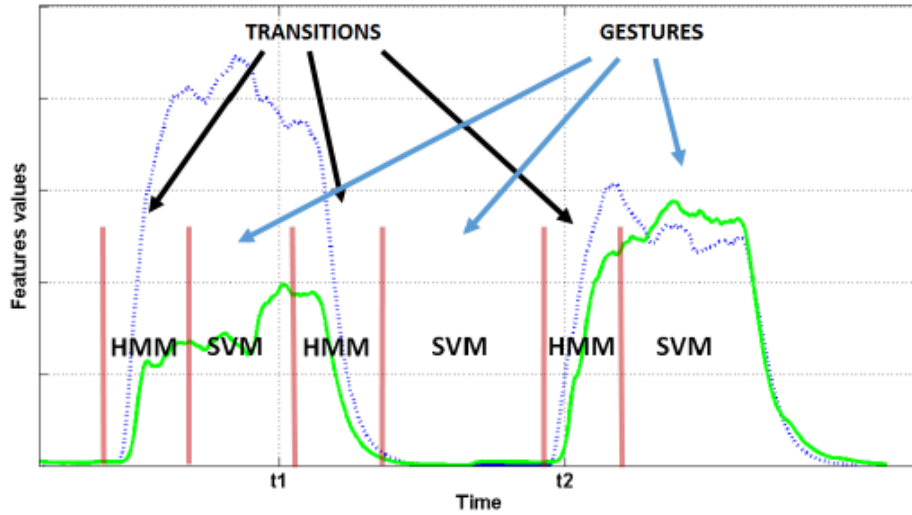


Figure 3.19. Classification of sEMG Based on Category [64]

Rossi et al. [64] executed two critical experiments. The first experiment assessed SVM performance and revealed root causes of SVM classification errors in EMG signals. Their findings emphasized that misclassifications frequently transpired during transitional phases, where EMG signals bore strong resemblance across diverse hand gestures. Consequently, a substantial 66% of classification errors occurred during these transients. To mitigate such errors and enhance robustness, [64] introduced a time-series classifier, leveraging Hidden Markov Models (HMM) to exclude transients from SVM classification, a computationally light strategy suitable for embedded implementations essential for prosthetic control systems. The second experiment showcased the substantial gains achieved with the HMM-SVM hybrid approach. With this innovation, they witnessed a consistent increase in classification accuracy. Notably, while the rest classification maintained an accuracy rate exceeding 99%, the average accuracy across other gesture classifications surged by more than 12.5%. However, it's worth acknowledging that excluding initial transients from SVM classification may introduce a delay in prosthesis control when monitored muscles initiate contraction. In order to maintain equilibrium between the precision of classification and the time it takes for control actions to be executed, they established a maximum limit of 300 milliseconds for the duration of the initial transient phase. This decision stemmed from their commitment to meeting strict real-time requirements, where gesture recognition plays a pivotal role in enabling precise artificial limb movements.

4. Materials and Methods

In this chapter, sEMG classification procedures for hand grip recognition will be explored while delving into the selection of our chosen machine learning model. The steps involved in classifying sEMG signals for accurate grip recognition will be examined. Additionally, insights will be provided into why a specific machine-learning model was opted for, offering a comprehensive understanding of our approach.

4.1. Data Acquisition

In this thesis, the data sets are taken from the UI Irvine machine learning repository [69]. Data acquisition is the initial step in the process of collecting, recording, and digitizing data from various physical phenomena or sources, such as sensors, instruments, or devices, for further analysis and processing. In the context of sEMG (Surface Electromyography) signal classification, data acquisition involves capturing electrical signals generated by muscle contractions using electrodes placed on the skin's surface. As depicted in Figure 4.1, this section provides a more detailed explanation of the data acquisition process in sEMG signal classification, as taken from the UC Irvine Machine Learning Repository.

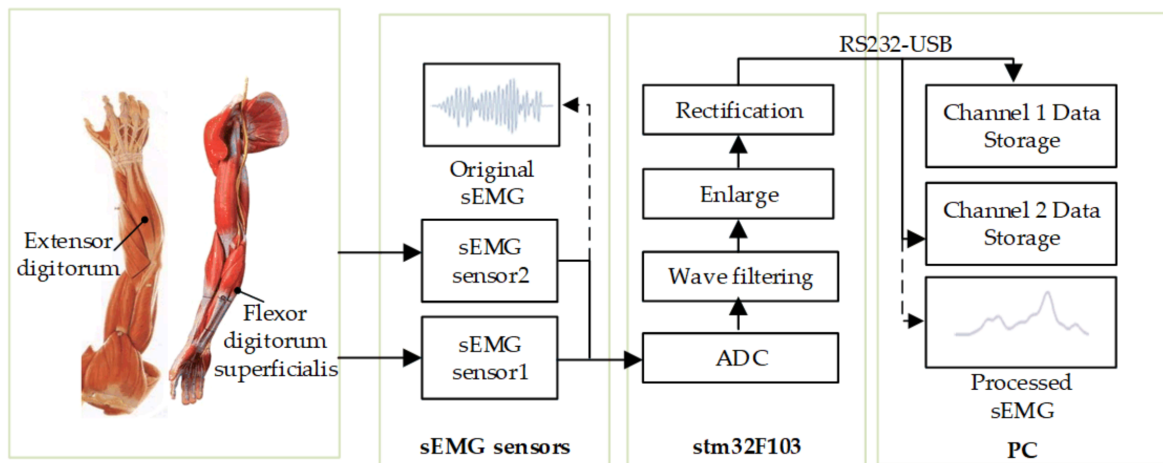


Figure 4.1. Data Acquisition Procedures [68]

1. Electrode Placement

The process of data acquisition typically begins with the placement of surface electrodes on the skin above specific muscles of interest. These electrodes are designed to detect and capture the electrical activity produced by muscle contractions. The precise placement of electrodes is crucial for accurately recording signals from the target muscles. In this data acquisition process, two differential EMG Sensors were used to capture the sEMG signals from the muscles of interest. These sensors are specialized devices designed to accurately measure muscle electrical activity. The signals from these sensors were then transmitted to a 2-channel EMG system by Delsys Bagnoli™ Handheld EMG Systems. This system likely provided the necessary amplification and conditioning of the signals before they were acquired by the computer.

2. **Signal Detection**

Once the electrodes are in place, they detect electrical signals known as sEMG signals. These signals are generated when motor neurons within the muscles send electrical impulses to control muscle contractions. The electrodes pick up these electrical signals from the skin's surface.

3. **Amplification**

sEMG signals are very weak, typically on the order of microvolts. To make these signals more suitable for measurement and analysis, they are usually amplified using specialized amplifiers. This amplification step increases the signal strength while maintaining its integrity.

4. **Filtering**

sEMG signals can be contaminated with various types of noise, including power line interference and movement artifacts. To ensure the quality of the collected sEMG signals, they were subjected to signal filtering. A Butterworth Band Pass filter was applied, which means that it allowed signals within a specific frequency range to pass through while attenuating signals outside that range. In this case, the low cutoff was set at 15 Hz, and the high cutoff was set at 500 Hz. This filtering focused on the frequency range of interest for sEMG signals. Additionally, a notch filter at 50 Hz was used to eliminate interference artifacts typically associated with power line noise.

5. **Sampling**

To facilitate digital processing and analysis, the analog sEMG signals are then sampled at a specific rate (sampling rate). The most common sampling rates for sEMG signals are in the range of 200 Hz to 2000 Hz. A higher sampling rate provides more data points and better temporal resolution but requires more storage and processing power. In UI Irvine machine learning repository [69] experiment the data collection process involved recording sEMG signals, which represent electrical activity in muscles, at a sampling rate of 500 Hz. This means that measurements were taken 500 times per second, providing a detailed representation of the muscle activity over time.

6. **Analog-to-Digital Conversion (ADC)**

The sampled analog signals are converted into digital format using an Analog-to-Digital Converter (ADC). This process quantizes the signal amplitudes into discrete digital values, typically represented as binary numbers. In UI Irvine machine learning repository experiment [69] the data collection hardware included an NI analog/digital conversion card, specifically the NI USB-009 model, which was installed in a personal computer (PC). This card allowed for the analog sEMG signals to be converted into digital data that could be processed and stored by the computer.

7. **Data Storage**

The digitized sEMG signals are stored in a computer or data storage device for further processing, analysis, and classification. The data may be saved in standard file formats like CSV, MAT, or HDF5. The software used for controlling and acquiring the data was National Instrument's (NI) LabVIEW. LabVIEW is a programming environment

commonly used for data acquisition and control applications. It allowed for the customization of the data collection process.

4.1.1. Feature Extraction

In this thesis CNN will be used as a feature extractor because selecting Convolutional Neural Networks (CNNs) for feature extraction over manual feature engineering offers several compelling advantages. As shown in Figure 4.2, CNNs possess a remarkable ability to automatically learn hierarchical features from raw data. In contrast, manual feature engineering often requires domain expertise and may rely on predefined rules or assumptions. sEMG signals are complex and can contain intricate patterns that might not be easily discerned by human experts. CNNs excel at capturing spatial patterns and hierarchies within these signals, eliminating the need for manually defining and extracting features. This automatic feature learning capability allows CNNs to adapt to the data's intricacies, making them particularly advantageous when dealing with diverse and dynamic sEMG datasets [70].

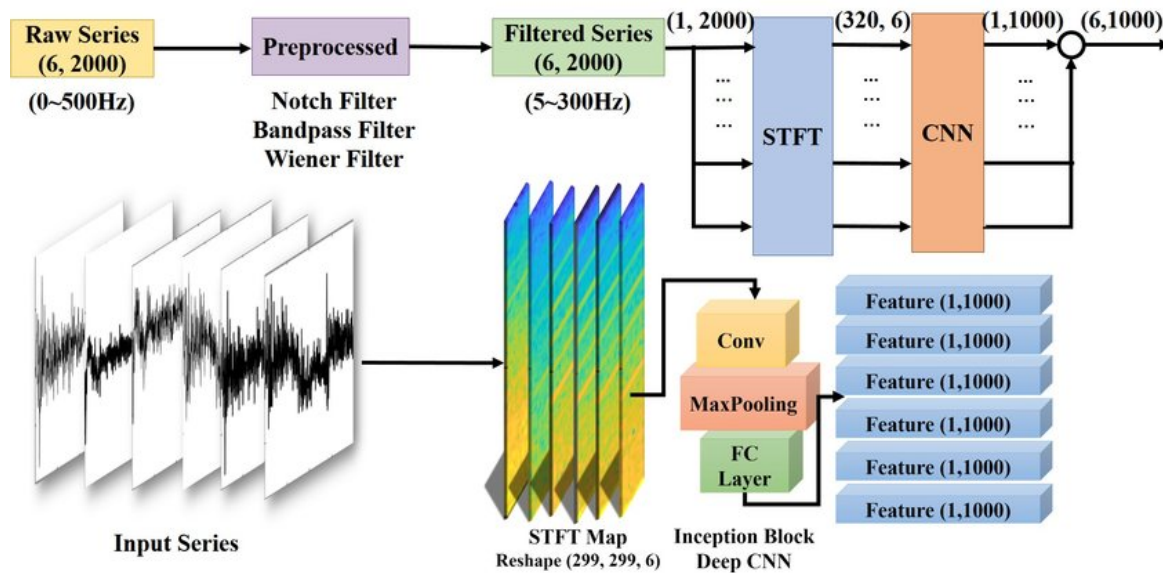


Figure 4.2. Feature Extraction by Using CNN [71]

CNN-based feature extraction promotes better adaptability to variations and changing data characteristics. Manual feature engineering typically involves designing a fixed set of features based on prior knowledge, which may not encompass all the relevant information present in the data. CNNs, on the other hand, can adapt to different signal conditions and variations encountered during real-world applications. They learn features that are most relevant to the specific classification task, enhancing generalization to unseen data. This adaptability can be especially beneficial when dealing with evolving sEMG signal patterns, which may change over time due to factors like muscle fatigue and electrode placement [71].

Manual feature engineering often involves subjective decisions about which features to extract, their representations, and their scaling. These choices can introduce bias

and may not fully capture the underlying patterns in the data. CNNs, being data-driven and objective, mitigate subjectivity by learning features directly from the signals. This reduces the risk of introducing human bias into the feature extraction process. Additionally, CNNs are less sensitive to subtle variations in feature engineering choices that can arise from different interpretations by domain experts, promoting consistency and reducing inter-observer variability. Manual feature engineering can be a time-consuming and labor-intensive process, particularly when dealing with high-dimensional and diverse sEMG datasets. CNN-based feature extraction streamlines this process by automating the extraction of relevant features. This not only saves time but also allows for scalability, making it feasible to handle large volumes of sEMG data efficiently. CNNs can process signals in parallel, making them well-suited for real-time and high-throughput applications where quick and accurate feature extraction is essential [72].

4.1.2. Machine Learning Model Selection

This section discusses the rationale behind the choice of a hybrid system for recognizing different hand grips using surface EMG signals. This thesis intends to utilize a hybrid system to capitalize on the strengths of diverse techniques and models, thereby enhancing overall performance. In the realm of machine learning, hybrid systems typically involve the integration of two or more distinct methods or algorithms to address complex tasks more effectively than relying solely on a single approach. A prominent example of such a hybrid system involves the amalgamation of Convolutional Neural Networks (CNNs) with traditional machine learning techniques, specifically Support Vector Machines (SVM) and k-Nearest Neighbors (KNN).

4.1.3. Standalone SVM

SVMs excel in handling high-dimensional feature spaces, a common characteristic of sEMG data when multiple sensors or electrodes are involved. This strength arises from SVM's ability to find an optimal hyperplane that maximizes the margin between different classes. This feature is vital for handling the complexity inherent in sEMG datasets, ensuring robust performance even when working with a large number of features [73].

SVMs exhibit inherent robustness to overfitting, a critical advantage when dealing with limited sEMG data. Overfitting occurs when a model fits the training data too closely, leading to poor generalization to new, unseen data. SVMs strike a balance between model complexity and generalization by maximizing the margin between classes while minimizing classification errors. This property helps ensure reliable and accurate classification, even with sparse sEMG datasets [74]–[76].

While SVMs are originally linear classifiers, they possess the capability to handle non-linear sEMG data patterns effectively. This is achieved by employing kernel functions like polynomial, radial basis function (RBF), or sigmoid kernels. By introducing non-linearity into the model, SVMs can capture complex relationships in the data without resorting to overly complex models, making them versatile for various sEMG classification tasks. SVMs are designed to maximize the margin between support vectors, which are data

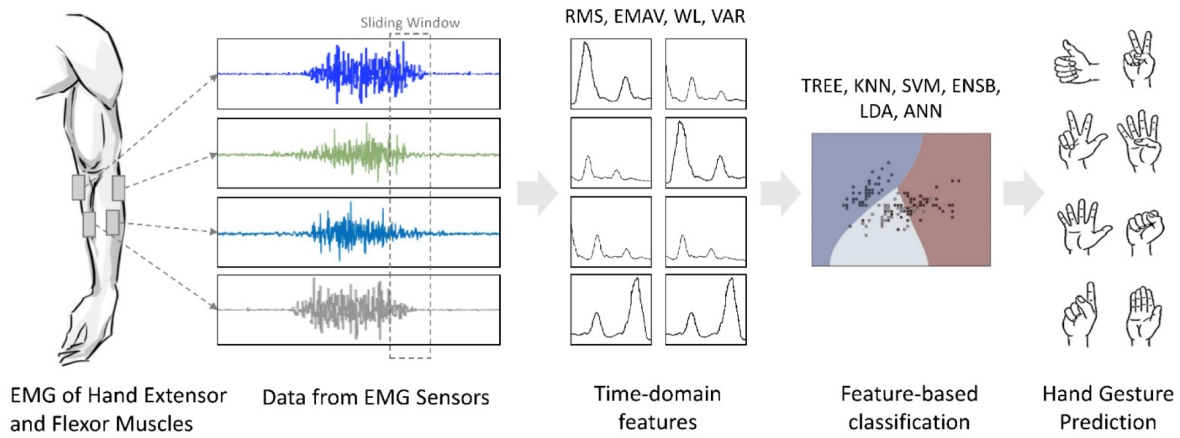


Figure 4.3. Traditional Machine Learning Classification with Manual Feature Extractors [77]

points closest to the decision boundary and belong to different classes. This focus on margin maximization results in models with better generalization and a reduced risk of misclassification. Moreover, SVMs solve a convex optimization problem, guaranteeing that the solution is the global optimum. This inherent global optimization property enhances the reliability and consistency of SVMs, making them a robust choice for sEMG-based gesture classification, especially when dealing with complex and nuanced patterns [45].

One significant limitation of SVMs in sEMG signal classification is their sensitivity to noise and outliers in the data. sEMG signals can be susceptible to various sources of noise, such as electrode placement variability, muscle fatigue, and external interference. SVMs aim to maximize the margin between classes, which can be heavily influenced by noisy or outlier data points. When the sEMG data contains a substantial amount of noise or outliers, SVMs may struggle to find an optimal decision boundary, leading to reduced classification accuracy. Preprocessing techniques, such as noise reduction and outlier detection, are often necessary to mitigate this limitation. SVMs can become computationally intensive and less scalable when dealing with large datasets or high-dimensional feature spaces. The time and memory requirements for training an SVM grow quadratically with the number of data points, which can make them impractical for massive datasets. Additionally, when working with a high number of features extracted from sEMG signals, SVMs may require significant computational resources, which could be a constraint in real-time applications or resource-constrained environments. Researchers often resort to dimensionality reduction techniques or alternative algorithms for scalability in such cases [78].

In sEMG signal classification, imbalanced datasets, where one class has significantly fewer samples than others, are common. SVMs tend to be biased towards the majority class, as their objective is to maximize the margin and reduce classification error. In imbalanced scenarios, this can lead to poor classification of minority classes. While techniques like class weighting and using different kernel functions can help alleviate this issue, handling imbalanced data effectively with SVMs may still require careful consideration and additional strategies. SVMs can achieve high classification accuracy by employing non-linear kernels to capture complex relationships in sEMG data. However, as SVMs move away from linearity, their interpretability diminishes. Understanding the decision boundaries and

feature contributions becomes challenging with non-linear kernels. This limitation can be crucial in medical applications where interpretability and explainability are essential for gaining insights into muscle activity patterns. In such cases, other interpretable machine learning models or feature selection techniques may be preferred [45].

4.1.4. Standalone KNN

One of the notable strengths of KNN in sEMG signal classification is its inherent ability to handle non-linear patterns in the data. sEMG signals can exhibit complex and non-linear relationships, especially in scenarios involving various muscle activities or gestures. KNN makes no assumptions about the underlying data distribution and relies on distance metrics to determine similarity between data points. This flexibility allows KNN to capture intricate patterns and relationships within sEMG signals without the need for complex model assumptions or feature engineering, making it suitable for a wide range of applications. KNN is known for its simplicity and transparency. The decision-making process of KNN is straightforward to understand, as it classifies data points based on the majority class among their k nearest neighbors. This interpretability is particularly valuable in medical and clinical contexts where understanding the reasoning behind classification decisions is crucial. Practitioners and domain experts can easily inspect which data points contribute to a particular classification outcome, making it easier to validate and gain insights into sEMG signal classification results [79], [80].

KNN is robust to the presence of irrelevant features or dimensions in the data. In sEMG signal classification, it's common to extract a large number of features from the signals, some of which may not be informative for the classification task. KNN considers only the relevant features when determining the nearest neighbors, effectively ignoring irrelevant dimensions. This feature selection property can simplify preprocessing and reduce the computational burden, as there's no need to perform feature selection or dimensionality reduction explicitly. KNN is known for its instance-based learning approach, which means it doesn't build a global model during training. Instead, it stores the training data and class labels and uses this information directly for classification. This adaptability to changing data distributions makes KNN suitable for scenarios where the sEMG data distribution may evolve over time, such as in rehabilitation settings where muscle activity patterns can change as patients progress in their therapy. KNN can continuously incorporate new data points into its decision-making process without requiring extensive retraining [81].

KNN is susceptible to imbalanced datasets, where one class has significantly more samples than others. Since KNN relies on majority voting among the k nearest neighbors, an imbalanced dataset can lead to biased classifications in favor of the majority class. The minority class may be consistently misclassified, resulting in a skewed performance evaluation. Techniques like resampling (e.g., oversampling or undersampling) or modifying the decision threshold can help address this issue, but it requires careful handling and consideration when dealing with imbalanced sEMG datasets. KNN is a non-parametric, instance-based learning algorithm, meaning it doesn't create a global model during training. Instead, it stores all training data, which can lead to a lack of model

generalization. KNN's performance heavily relies on the distribution of the training data. If the training data is sparse in certain regions of the feature space, KNN may struggle to make accurate predictions in those areas. This limitation can become apparent when dealing with complex and diverse sEMG signal patterns, as KNN may require an extensive and representative training dataset to generalize well to unseen samples. Additionally, KNN may be more prone to overfitting when k is small, as it can be sensitive to small fluctuations in the local data distribution [82].

4.1.5. Standalone CNN

One of the primary strengths of CNNs is their ability to automatically learn hierarchical features from raw data. In the context of sEMG signal classification, this means that CNNs can capture complex patterns and relationships in the signals without the need for explicit feature engineering. CNNs utilize convolutional layers to detect local patterns, such as edges and shapes, and then progressively build more abstract and high-level representations in deeper layers. This hierarchical feature learning makes CNNs particularly well-suited for tasks where the underlying patterns are intricate and may not be easily defined by hand-crafted features. CNNs are inherently translation-invariant, which means they can recognize patterns regardless of their spatial location within the input data. In the case of sEMG signals, this is advantageous because muscle activation patterns can vary in location and intensity. CNNs can identify relevant features regardless of where they appear in the signal, making them robust to slight variations in electrode placement or muscle activity. This property is especially valuable in applications where consistent and accurate pattern recognition is essential [83].

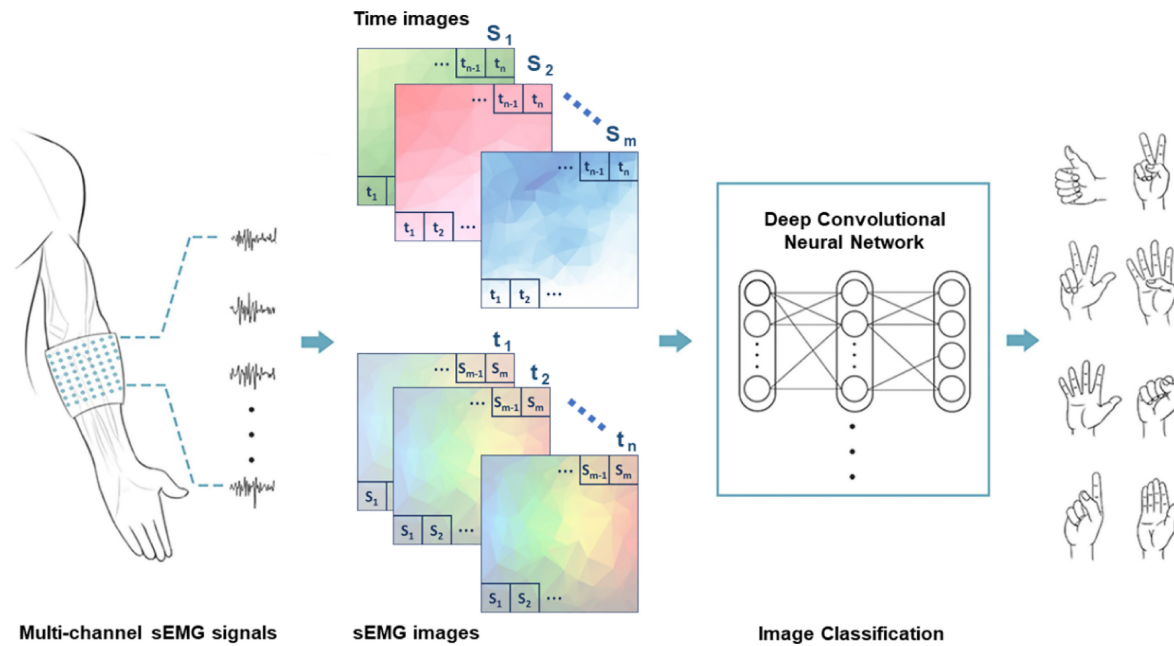


Figure 4.4. Stand Alone CNN Classifier [84]

CNNs utilize parameter sharing, where the same set of learnable weights is used across different regions of the input data. This sharing of parameters reduces the number of

learnable parameters compared to fully connected networks, making CNNs more efficient and less prone to overfitting, especially when dealing with limited sEMG data. Additionally, techniques like dropout and batch normalization can be easily integrated into CNN architectures to further improve regularization and prevent overfitting, resulting in models with better generalization performance. CNNs can be effectively scaled to handle various complexities of sEMG signal classification tasks. Whether you're working with a small dataset or a large-scale project, CNN architectures can be adapted to suit the task at hand. Moreover, CNNs are well-suited for transfer learning, where pre-trained models on large datasets (e.g., ImageNet) can be fine-tuned for sEMG tasks. This approach allows researchers and practitioners to leverage the knowledge and feature representations learned from extensive datasets, even when sEMG data is limited. It also reduces the need for extensive data annotation [85].

CNNs are data-hungry models, meaning they often require a substantial amount of labeled training data to perform well. In many sEMG applications, acquiring a large labeled dataset can be challenging and expensive. When training with limited data, CNNs are prone to overfitting, where they learn to memorize the training examples instead of generalizing from them. Overfitting can lead to poor performance on new, unseen sEMG signals. To mitigate this limitation, techniques like data augmentation and transfer learning can be used, but they may not completely alleviate the data scarcity issue. CNNs, especially deep architectures, can be computationally intensive and require significant computational resources for training and inference. This limitation can be a barrier in applications where real-time or low-power processing is crucial, as sEMG signal classification often occurs in healthcare, rehabilitation, or wearable devices. Training deep CNNs may also demand specialized hardware (e.g., GPUs or TPUs) and can be time-consuming. Hence, model selection and architecture design need to balance performance with computational efficiency [83].

One notable limitation of CNNs is their reduced interpretability compared to some traditional machine learning algorithms. CNNs function as complex black-box models, and it can be challenging to gain insights into how they arrive at classification decisions, particularly in multi-layer architectures. In medical and clinical settings, interpretability is crucial for understanding the reasoning behind model predictions and gaining insights into muscle activity patterns. This limitation can be addressed through techniques like visualization of feature maps or layer activations, but it remains a challenge. CNNs are primarily designed for processing spatial information, which makes them well-suited for image-based tasks. However, sEMG signals are inherently temporal, capturing changes in muscle activity over time. CNNs, in their traditional form, do not explicitly model temporal dependencies and may not capture dynamic aspects of sEMG data as effectively as models explicitly designed for time-series data, such as recurrent neural networks (RNNs) or hybrid models. This limitation can be particularly relevant in tasks where precise temporal information is critical, such as capturing the timing and coordination of muscle activations [85].

4.1.6. Hybrid System

Choosing a hybrid system, which combines Convolutional Neural Networks (CNNs) and traditional machine learning techniques, can be a compelling choice for sEMG (surface electromyography) signal classification because CNNs excel at capturing spatial patterns and hierarchies in data, making them well-suited for sEMG signal feature extraction and recognition. On the other hand, traditional machine learning techniques, such as Support Vector Machines (SVM) and k-Nearest Neighbors (KNN), can provide interpretability and handle certain aspects of data preprocessing and post-processing effectively. By combining these approaches, we can leverage the complementary strengths of both to improve overall performance. As in section 4.1.1 elaborated, CNNs are proficient at automatically extracting relevant features from raw sEMG signals, reducing the need for manual feature engineering. This can be particularly advantageous when dealing with complex and high-dimensional signal data. Traditional machine learning models can then operate on these extracted features, simplifying the classification process and potentially enhancing accuracy. sEMG data can be noisy and subject to variability due to factors like electrode placement, muscle fatigue, and individual differences. Hybrid systems can incorporate the noise robustness of CNNs, which can learn to recognize patterns despite variations in signal quality. Traditional models can then further refine the classification by leveraging their interpretability and domain-specific knowledge [56].

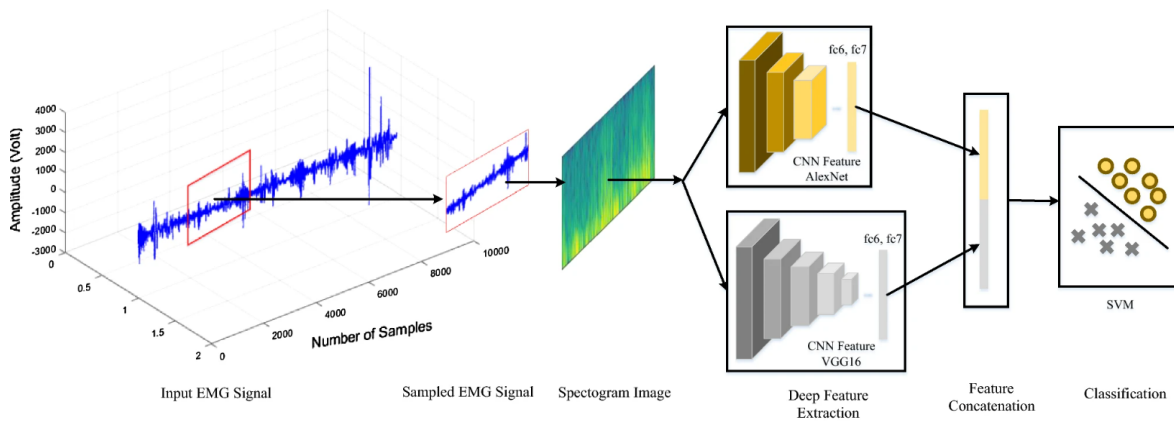


Figure 4.5. Combination of CNN with Traditional Machine Learning Classifier [86]

Combining CNNs with traditional machine-learning models can lead to better generalization. CNNs can capture intricate patterns in the sEMG signals, while traditional models can add regularization and interpretability to improve model generalization on unseen data. This is especially valuable when dealing with limited labeled sEMG data. Hybrid systems offer flexibility in terms of model design. We can tailor the hybrid architecture to our specific sEMG classification task by selecting appropriate CNN architectures, traditional ML models, and fusion strategies. This customization allows us to adapt the system to the unique requirements and characteristics of our dataset. In certain applications, such as medical diagnostics and rehabilitation, model interpretability is critical for gaining trust and insights into the classification results. Hybrid systems provide the means to explain the decisions made by combining both interpretable traditional models and the

learned features from CNNs. By integrating multiple models, each with its own strengths, hybrid systems effectively create an ensemble of classifiers. Ensemble methods often lead to improved classification accuracy and robustness by combining the diverse knowledge from different models [56].

5. Implementation of Experiment

In this chapter, delving into the details of the dataset employed for analysis, the chosen evaluation metrics to assess the performance of models, and a comprehensive exposition of the proposed hybrid model. Throughout the following sections, aiming to provide an exhaustive examination of each element, shedding light on the dataset's intricacies, the significance of the chosen evaluation metrics, and a comprehensive understanding of the hybrid model's architecture and functioning. Through this comprehensive analysis, seeking to offer a holistic view of the foundational components that underpin the study.

5.1. Dataset Description

In Section 4.1, the datasets sourced from the UCI Irvine Machine Learning Repository were elaborated upon [69]. In this section, a comprehensive exploration of the characteristics, components, and dimensions of the dataset will be provided.

5.1.1. Characteristics of the Dataset

The dataset exhibits the fundamental characteristic of being composed of time-series data, as shown in Table 5.1. Time-series data is a type of data where each data point is associated with a specific timestamp or time interval. In this context, the dataset consists of sequential records capturing surface electromyographic (sEMG) signals over time. These signals represent the electrical activity of muscles during hand movements. This temporal aspect is crucial for understanding how muscle activity evolves during the performance of various hand grips. Time-series data often requires specialized analysis techniques to uncover patterns and trends over time.

The dataset is situated within the domain of life sciences. Life science encompasses a wide range of scientific disciplines that explore living organisms and their interactions with their environment. In this case, the dataset is particularly relevant to fields like biomechanics, physiology, and rehabilitation. Researchers in these domains use sEMG data to study muscle function and motor control, making it an invaluable resource for gaining insights into human physiology and movement.

One of the primary purposes of this dataset is to facilitate classification tasks. Classification involves assigning data instances to predefined categories or classes based on their features. In the context of the sEMG dataset, the goal is to classify or recognize specific hand movements performed by subjects based on the recorded sEMG signals. This can be a challenging task, as it requires machine learning models to learn complex patterns in the data and make accurate predictions. Classification is a fundamental task in various applications, including grip recognition for human-computer interaction and rehabilitation.

The attribute type in this dataset is labeled as "Real," which signifies that the dataset contains real-valued attributes. In the context of sEMG signals, these real-valued attributes likely represent the amplitude or intensity of the electrical signals measured at specific time points. Real-valued attributes are common in quantitative datasets and are suitable

for various mathematical and statistical analyses. They provide a continuous range of values, allowing for precise characterization of the sEMG signals.

The dataset comprises a total of 3000 instances. Each instance corresponds to a recorded sEMG signal during one of the hand movements performed by the subjects. The number of instances in a dataset is a critical factor as it determines the amount of data available for model training, validation, and testing. A larger dataset often provides more opportunities for machine learning models to generalize effectively.

The dataset is characterized by 2500 attributes. These attributes are likely derived from the sEMG signals and serve as input features for classification models. The high dimensionality of the dataset, with a substantial number of attributes, presents both opportunities and challenges. On one hand, a large number of attributes can potentially capture intricate details in the sEMG signals, enhancing the model's ability to discriminate between hand movements. On the other hand, dimensionality reduction techniques and careful feature selection may be necessary to prevent overfitting and reduce computational complexity.

5.1.2. Instrumentation

The sEMG (surface electromyographic) dataset for basic hand movements was meticulously collected using advanced instrumentation and techniques. Data acquisition was carried out at a high sampling rate of 500 Hz, ensuring precise capturing of muscle activity during hand movements. National Instrument's (NI) LabVIEW served as the programming kernel for data acquisition, enabling the synchronization and control of the data collection process. To enhance data quality, the recorded sEMG signals underwent careful preprocessing steps. A Butterworth Band Pass filter was applied with low and high cutoff frequencies set at 15 Hz and 500 Hz, respectively. Additionally, a notch filter at 50 Hz was employed to eliminate undesirable line interference artifacts, ensuring that the acquired data remained clean and suitable for analysis.

5.1.3. Hardware and Sensor Setup

The hardware used for data acquisition was a National Instruments analog/digital conversion card, specifically the NI USB-009 model, integrated into a personal computer (PC). The sEMG signals were captured using two Differential EMG Sensors. These sensors were part of the Delsys Bagnoli™ Handheld EMG Systems, which are renowned for their precision and reliability in recording electromyographic data. The setup allowed for the simultaneous recording of muscle activity from two distinct channels, providing valuable insights into muscle activation patterns during hand movements.

5.1.4. Experimental Protocol

The dataset's collection protocol involved participants performing various hand movements under specific conditions. Participants were encouraged to grasp different items freely and repeatedly. Importantly, the protocol allowed for variations in movement speed and force, reflecting the natural variability in human hand gestures. To facilitate data collection, two forearm surface EMG electrodes were utilized, namely the Flexor Capri

Ulnaris and Extensor Capri Radialis (Longus and Brevis). These electrodes were secured in place using elastic bands. Additionally, a reference electrode was strategically placed in the middle to gather reference information about muscle activation.

5.1.5. Hand Movements

As shown in Figure 5.1, the dataset captures a comprehensive range of hand movements, encompassing six distinct gestures that mimic common daily grasps. These gestures include:

- Spherical: Designed for holding spherical tools.
- Tip: Intended for holding small tools.
- Palmar: Used for grasping with the palm facing the object.
- Lateral: Tailored for holding thin, flat objects.
- Cylindrical: Designed for holding cylindrical tools.
- Hook: Configured for supporting heavy loads.

These six hand movements represent a diverse set of actions that individuals perform routinely, making the dataset highly applicable to real-world scenarios and practical applications.

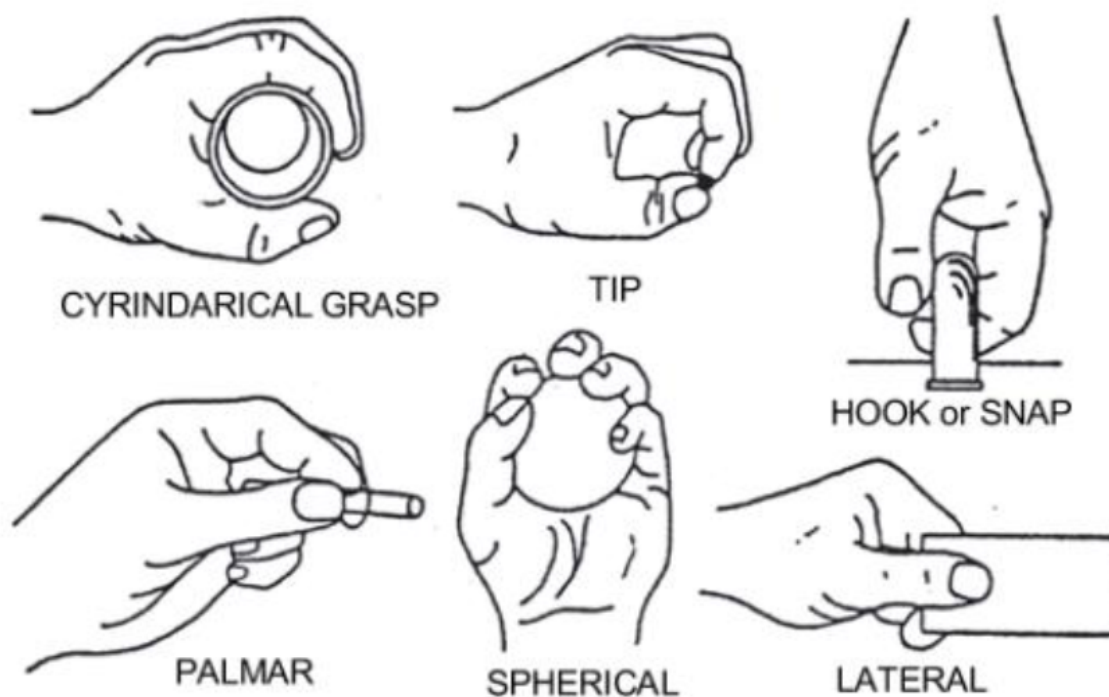


Figure 5.1. Six Different Hand Movements [69]

5.1.6. Databases

The dataset is divided into two distinct databases, each offering unique insights and applications. The first database comprises data from five healthy subjects (two males and three females) of similar age (approximately 20 to 22 years old). In this database, each

subject executed the six hand movements a total of 30 times. Each trial had a measured duration of 6 seconds, providing rich temporal information. A mat file is available for each subject, allowing for individual analysis. The second database features data from a single healthy subject (male, 22 years old) who performed the six hand movements 100 times each for three consecutive days. Each trial in this database had a measured duration of 5 seconds, and a mat file is provided for each day of data collection. This differentiation in data collection protocols offers researchers a comprehensive dataset suitable for various study designs and objectives.

5.1.7. Data Format

The format of each mat file within the dataset is highly organized. Specifically, the data for each grasp and each channel (representing the sEMG electrode) are stored in separate tables with intuitive naming conventions. For example, the "Spherical" grasp is stored as "spher_ch1" and "spher_ch2" for the two channels. Each row in these tables represents the entire sEMG signal for a single trial. The signal values are measured in Voltage, providing a consistent and interpretable unit of measurement.

5.1.8. Data Dimensions

Generally, each subject's mat file contains 12 matrices, with each matrix corresponding to one of the six hand movements. Depending on the database, each matrix either contains 30 rows and 3000 columns (for Database 1) or 100 rows and 2500 columns (for Database 2). These dimensions reflect the richness of the dataset, offering a substantial amount of data for analysis and model development.

Table 5.1. Dataset Characteristics

Characteristics	Details
Time-Series	Yes
Subject Area	Life Science
Associated Tasks	Classification
Attribute Type	Real
Number of Instances	3000
Number of Attributes	2500

5.2. Evaluation Metrics

In this section, various evaluation metrics utilized to gauge a model's strengths and weaknesses will be explained. Evaluation metrics encompass a range of quantifiable measures employed to assess the performance of machine learning models, algorithms, or systems. They play a pivotal role in comprehending the model's effectiveness across different tasks, offering valuable insights into both its strengths and weaknesses.

1. Precision

5. Implementation of Experiment

Precision is a crucial evaluation metric in machine learning that measures the accuracy of a model's positive predictions. In simpler terms, it tells us how many of the instances our model classified as positive were actually correct.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}} \quad (5.1)$$

High precision indicates that when the model says something is positive, it's usually right. In essence, precision helps us understand the quality of positive predictions and the extent to which we can trust the model's affirmative classifications.

2. Recall

Recall, also known as sensitivity or true positive rate, is an important evaluation metric in machine learning. It measures a model's ability to correctly identify all relevant instances of the positive class within a dataset. In simpler terms, recall answers the question: "Of all the actual positive instances, how many did the model correctly predict?".

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}} \quad (5.2)$$

A high recall score indicates that the model is good at capturing a large proportion of the actual positive instances, meaning it minimizes the number of false negatives.

3. Accuracy

Accuracy is a fundamental and widely used evaluation metric in machine learning. It measures the overall correctness of a model's predictions across all classes in a classification task or the closeness of predicted values to actual values in a regression task. In simpler terms, accuracy answers the question: "How many of the model's predictions were correct?".

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Number of Instances}} \quad (5.3)$$

Accuracy provides a clear and intuitive measure of a model's performance, and a high accuracy score indicates that the model is making a large proportion of correct predictions. However, accuracy may not be the most suitable metric in cases where class distribution is imbalanced. In such situations, a model that simply predicts the majority class can achieve a high accuracy, even if it performs poorly on minority classes.

4. F1 Score

The F1 Score is a commonly used evaluation metric in machine learning that combines both precision and recall into a single score. It provides a balanced measure of a model's performance, particularly in binary classification tasks, where there are two classes: positive and negative.

$$\text{F1 Score} = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}} \quad (5.4)$$

The F1 Score is particularly useful when we want to find a balance between making

accurate positive predictions (high precision) and capturing all positive instances (high recall). The F1 Score is the harmonic mean of precision and recall. This harmonic mean puts more weight on lower values, making the F1 Score useful for situations where we want to ensure a balance between precision and recall.

5. Macro Average

In multi-class classification scenarios, there are multiple classes to consider. The macro average is a method of evaluating the model's performance across all classes by computing the chosen metric (e.g., precision, recall, F1 Score) individually for each class and then taking the unweighted average of these class-specific scores.

$$\text{Macro Average} = \frac{1}{N} \sum_{i=1}^N \text{Metric}_i \quad (5.5)$$

This approach treats each class equally, irrespective of its size or prevalence in the dataset. It is beneficial when we want to assess the model's performance without favoring any specific class, making it particularly suitable for tasks with class imbalances.

6. Weighted Average

Similar to the macro average, the weighted average is used in multi-class classification but accounts for class prevalence. It calculates the chosen metric for each class and then computes the average, with each class's contribution weighted by its proportion in the dataset.

$$\text{Weighted Average} = \frac{1}{N} \sum_{i=1}^N \left(\frac{S_i}{T} \right) \cdot \text{Metric}_i \quad (5.6)$$

This approach addresses class imbalances by giving more importance to the performance of classes with larger representation. The weighted average is valuable when we need to consider class prevalence in your evaluation and want to ensure that the model's performance is appropriately weighted by class distribution.

5.3. Procedure Followed

As shown in Figure 5.2, the procedure section outlines the key steps involved in this thesis. It encompasses data preprocessing, feature extraction using Convolutional Neural Networks (CNN), which is subsequently used to train and evaluate Support Vector Machine (SVM) and k-Nearest Neighbors (KNN) classifiers. This systematic approach aims to achieve robust model performance and provides a comprehensive understanding of the intricacies of the classification task.

5.3.1. Define Schema of Dataset

In the initial phase of data preparation, a structured framework for organizing and categorizing the dataset was established. This framework involved the creation of a schema, typically implemented using dictionaries, lists, or other data structures. Within this schema, various essential components were defined, including databases, files, columns, and la-

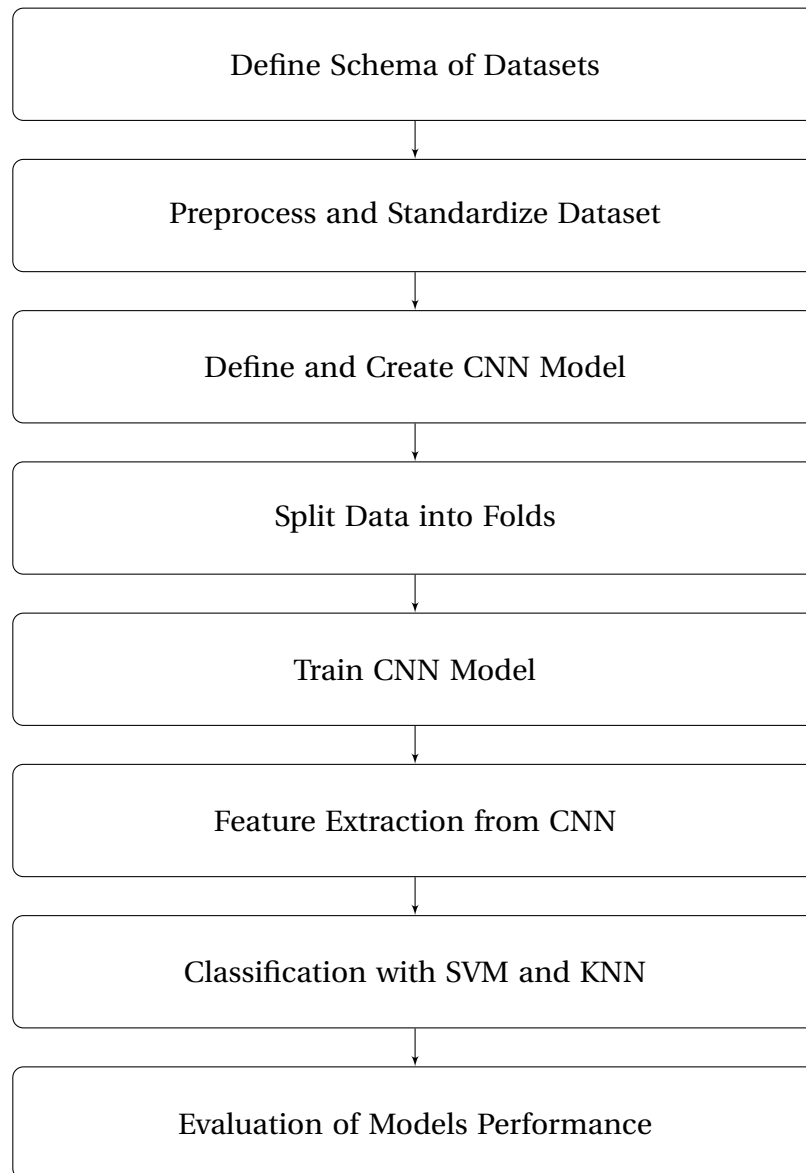


Figure 5.2. Machine Learning Pipeline Flowchart

bels. Each database was associated with a list of individual data samples, represented by filenames. Moreover, a list of columns was specified to identify the attributes or features of interest within each data sample, while a set of labels was defined to categorize these samples into distinct classes or categories. To streamline data manipulation and analysis, a mapping dictionary was often introduced to connect specific columns with their corresponding labels.

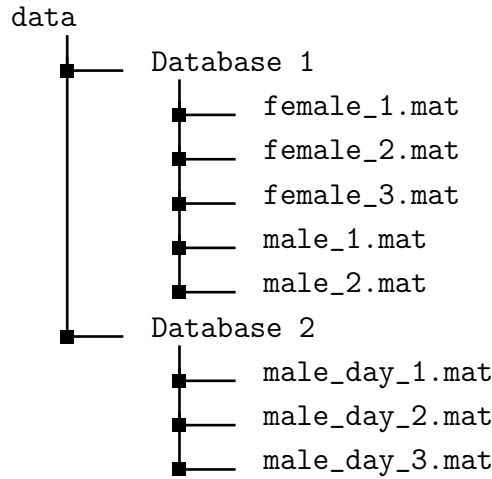


Figure 5.3. Folder Structure of the Dataset

5.3.2. Preprocess and Standardize Dataset

In this stage, the raw data was prepared and structured to ensure its suitability for analysis. This step typically involved several key tasks. Firstly, the data was loaded and filtered to extract relevant information while discarding unnecessary elements. Next, data samples were organized into a unified and coherent format, often combining them into a single dataset. During this process, data might have been reshaped, cleaned, and standardized to ensure consistency and comparability across different samples. Additionally, labels and identifiers were assigned to each data point, enabling easy tracking and categorization. Overall, this step was vital for transforming raw data into a clean, structured, and well-labeled dataset, setting the stage for subsequent modeling and analysis tasks.

5.3.3. Define and Create CNN Model

Here, a Convolutional Neural Network (CNN) architecture was meticulously crafted to address the specific requirements of the task at hand. This CNN model, a foundational component of deep learning, was designed with careful consideration of the dataset's characteristics and the desired level of complexity for feature extraction and classification.

As shown in Table 5.3, the architecture of this CNN model comprised a series of convolutional layers, strategically placed to identify low-level features within the input data. These convolutional layers were coupled with activation functions, notably Rectified Linear Units (ReLU), which introduced non-linearity and enabled the model to capture intricate relationships among the data points.

Pooling layers were judiciously incorporated to down-sample the feature maps, reducing dimensionality while preserving crucial information. This hierarchical feature extraction allowed the model to learn increasingly abstract and complex patterns as it traversed the layers.

To mitigate overfitting, dropout layers were thoughtfully added, randomly deactivating a fraction of neurons during training to promote generalization. This ensured that the model's performance extended beyond the training data to make accurate predictions on unseen data.

Table 5.2. Hyperparameters

Hyperparameter	Value
Batch Size	100
Learning Rate	0.001
Optimizer	Adam
Number of Epochs	250
Number of Folds (k)	5
SVM Kernel	Linear
SVM Regularization Parameter (C)	1
KNN Number of Neighbors (k)	5

As the model deepened, fully connected layers were introduced to serve as classifiers, leveraging the extracted features to make predictions. The final layer of the model employed a softmax activation function, facilitating multi-class classification by producing class probabilities.

It's worth noting that the architecture detailed in this step was meticulously fine-tuned, taking into account various hyperparameters such as learning rate and regularization strength, as shown in Table 5.2. This meticulous design process ensured that the CNN model was optimized to recognize patterns effectively and make precise predictions based on the input data.

5.3.4. Split Data into Folds

In this step, the dataset was systematically divided into distinct subsets or folds, a fundamental practice in machine learning for cross-validation and model evaluation. This step was executed with precision to ensure that the data was efficiently and fairly partitioned, facilitating robust model assessment.

A common practice, as employed in this step, was to employ k-fold cross-validation, where the dataset was divided into 'k' roughly equal parts or folds. For instance, in this particular case, a value of 'k' was set to 5, implying that the data was divided into five separate folds. Each fold represented a distinct subset of the dataset, and careful attention was paid to maintain the class distribution to prevent any bias in the evaluation process.

For every iteration, one of the folds was reserved as the testing set, while the remaining folds were combined to form the training set. This procedure was repeated 'k' times, with each fold taking on the role of the testing set exactly once. This practice ensured

Table 5.3. Proposed CNN Model Configuration

Layer	Configuration	Output Size
Input	Input Shape: (6000, 1)	(6000, 1)
Conv1D	Filters: 4, Kernel Size: 2, Activation: ReLU, Padding: Same	(6000, 4)
Pooling	Pool Size: 2, Strides: 2, Padding: Same	(3000, 4)
Conv1D	Filters: 8, Kernel Size: 2, Activation: ReLU, Padding: Same	(3000, 8)
Pooling	Pool Size: 2, Strides: 2, Padding: Same	(1500, 8)
Conv1D	Filters: 16, Kernel Size: 2, Activation: ReLU, Padding: Same	(1500, 16)
Pooling	Pool Size: 2, Strides: 2, Padding: Same	(750, 16)
Conv1D	Filters: 32, Kernel Size: 2, Activation: ReLU, Padding: Same	(750, 32)
Pooling	Pool Size: 2, Strides: 2, Padding: Same	(375, 32)
Conv1D	Filters: 64, Kernel Size: 2, Activation: ReLU, Padding: Same	(375, 64)
Pooling	Pool Size: 2, Strides: 2, Padding: Same	(188, 64)
Conv1D	Filters: 128, Kernel Size: 2, Activation: ReLU, Padding: Same	(188, 128)
Pooling	Pool Size: 2, Strides: 2, Padding: Same	(94, 128)
Flatten		$(94 * 128) = 12032$
Dense	Units: 128, Activation: ReLU, Regularization: L2 (0.01)	128
Dropout	Rate: 0.5	128
Dense	Units: 6 (Number of Classes), Activation: Softmax	6

that every data point was utilized for both training and testing, resulting in a robust and comprehensive assessment of the model's performance.

The benefits of this systematic data splitting approach included the ability to gauge the model's consistency across different subsets of data, detect potential overfitting or underfitting issues, and derive more reliable performance metrics. Additionally, it allowed for the estimation of the model's generalization capabilities to unseen data, a crucial aspect of model evaluation.

5.3.5. Train CNN Model

In the "Train CNN Model" step, the meticulously designed Convolutional Neural Network (CNN) architecture was put into action to learn and extract valuable patterns from the dataset. This crucial step involved feeding the prepared training data into the CNN model, enabling it to adjust its internal parameters through a process known as backpropagation.

The training data, which had been thoughtfully split into training and testing subsets as in 5.3.4 elaborated, played a pivotal role. The training subset was utilized to teach the CNN model to recognize intricate features and patterns within the data. During training, the model learned to minimize the difference between its predictions and the actual ground truth labels, effectively optimizing its ability to make accurate predictions.

The training process encompassed numerous epochs, with each epoch representing a complete pass through the entire training dataset. One notable aspect of this step was the experimentation with learning rates, ranging from 0.000001 to 0.1. The learning rate played a crucial role in determining the step size of the optimization process, affecting the model's convergence and generalization capabilities.

Optimization techniques such as the Adam optimizer were employed to fine-tune the

model's internal parameters. Careful tuning of hyperparameters, including the learning rate, was conducted to strike a balance between efficient convergence and avoiding over-shooting the optimal solution. This experimentation with learning rates allowed for a thorough exploration of the model's behavior under different optimization settings.

As the CNN model advanced through epochs, its performance on the training data improved, with the objective of generalizing well to unseen data. This iterative process was meticulously monitored, with metrics such as accuracy and loss being continuously evaluated to assess the model's progress.

5.3.6. Feature Extraction from CNN

In the feature extraction step, the Convolutional Neural Network (CNN) model, which had undergone rigorous training as in 5.3.5 explained, was repurposed to extract essential features from the data. This step was instrumental in transforming the data representations learned by the CNN into a format suitable for subsequent classification tasks.

To perform feature extraction, a new Sequential model was created, composed of the layers from the pre-trained CNN model up to, but excluding, the final softmax layer. This approach effectively removed the classification layer, leaving only the layers responsible for feature extraction. These layers retained the knowledge and hierarchical representations learned during training.

The dataset, carefully preprocessed and standardized, was then fed through this feature extractor. As the data passed through the layers of the CNN model, it underwent a series of convolutional and pooling operations, gradually transforming it into a set of abstract, high-level features. These features encapsulated the most salient information from the original data, making them highly informative for subsequent classification tasks, as shown in Figure 5.4.

One notable advantage of feature extraction from a pre-trained CNN model was the ability to leverage the transfer learning paradigm. By reusing the learned features, the model benefitted from the knowledge acquired during training on a potentially different, but related, dataset. This often led to improved performance, especially when dealing with limited data availability.

The feature extraction process resulted in feature vectors that captured the essence of the input data while eliminating irrelevant details and noise. These feature vectors could then be used as input for various machine learning classifiers, such as Support Vector Machines (SVM) or K-Nearest Neighbors (KNN), as demonstrated in Figure 5.4.

5.3.7. Classification with SVM and KNN

In the "Classification with SVM and KNN" step, the feature vectors extracted from the pre-trained Convolutional Neural Network (CNN) model were put to practical use for classifying the data into its respective categories. This pivotal step involved the application of two well-established classification algorithms: Support Vector Machines (SVM) and K-Nearest Neighbors (KNN), as referred in Figure 5.4.

Support Vector Machines, a robust and versatile classifier, were first employed. SVMs

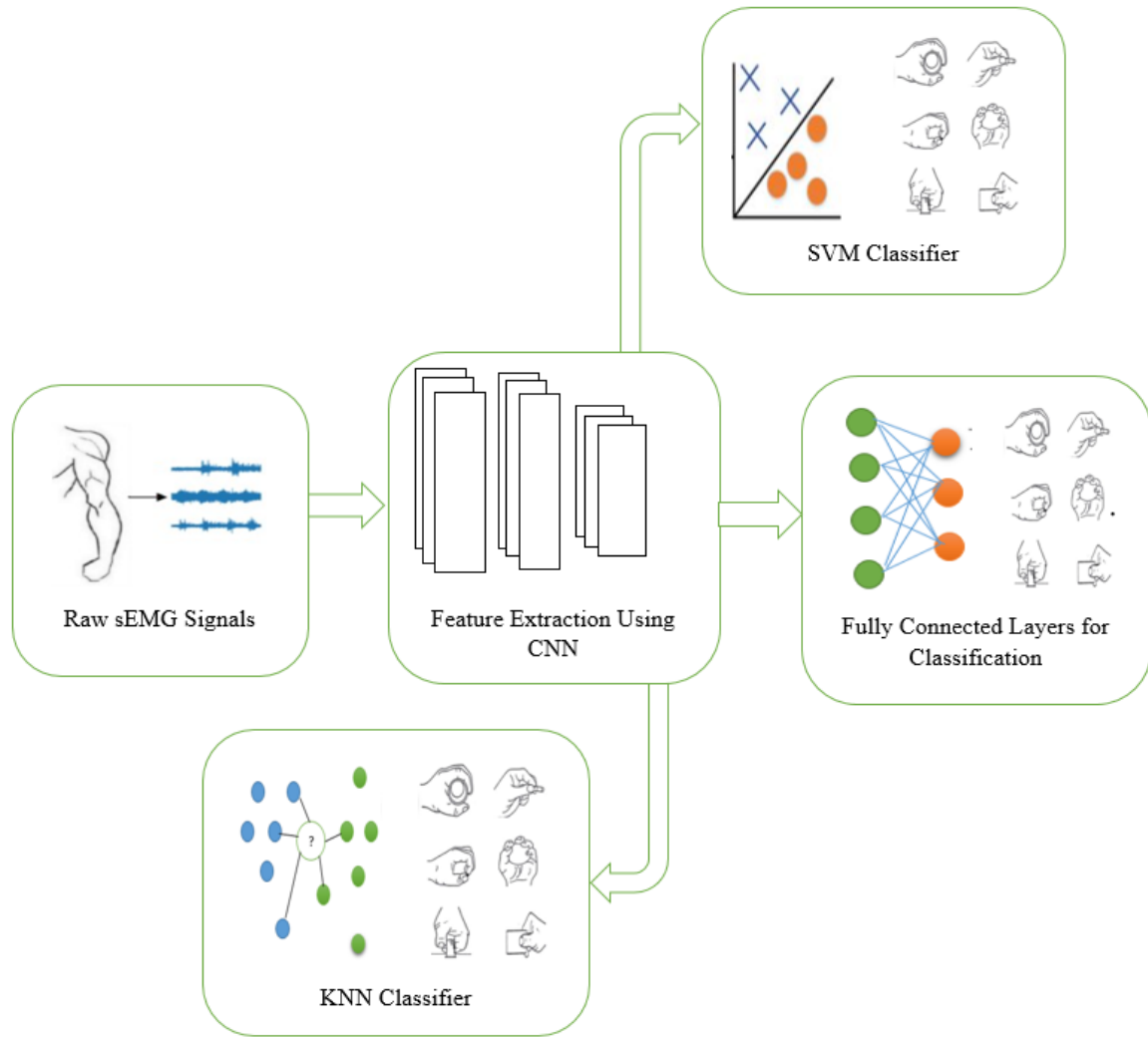


Figure 5.4. Proposed Classification Models

excel at finding the optimal hyperplane that maximizes the margin between different classes in the feature space. In this step, a linear kernel was chosen for the SVM classifier, as it was well-suited to the high-dimensional feature vectors obtained from the CNN. The linear kernel essentially sought to find a linear boundary that best separated the feature vectors into their respective classes.

Additionally, a preprocessing step involving feature scaling was implemented to ensure that all features contributed equally to the classification process. This involved standardizing the feature vectors to have zero mean and unit variance, a crucial step in SVM training.

The SVM classifier, integrated into a pipeline along with feature scaling, was then trained on the feature vectors extracted from the CNN model. Once trained, the SVM model was applied to the test data, making predictions based on the learned patterns in the feature space.

K-Nearest Neighbors (KNN), another powerful classification algorithm, was also employed in this step. KNN operates on the principle of proximity, classifying data points based on the majority class among their 'k' nearest neighbors in the feature space. The

choice of 'k' (number of neighbors) equal to 5 was carefully considered and tuned to optimize performance.

Similar to SVM, a feature scaling preprocessing step was applied to ensure equitable contributions from all features. The KNN classifier also integrated into a pipeline, was trained on the feature vectors and subsequently utilized for making predictions on the test data.

5.3.8. Evaluation of Models Performance

In the final step, the performance of the models, specifically the Convolutional Neural Network (CNN), Hybrid of CNN-SVM, and CNN-KNN classifiers were systematically assessed. This step played a pivotal role in gauging how well these models performed in their respective classification tasks and provided valuable insights into their strengths and weaknesses.

The evaluation process began by systematically computing a range of key performance metrics for each of the models. These metrics included accuracy, precision, recall, and F1-score, which are fundamental measures for assessing classification performance. By analyzing these metrics, it was possible to gain a comprehensive understanding of how well the models were able to correctly classify data points into their respective categories.

The performance metrics were averaged across all the folds, providing an overall assessment of how well each model performed across different subsets of the data. This approach ensured a robust evaluation that considered the models' consistency and generalization capabilities.

Additionally, confusion matrices were generated to visually represent the performance of each model. These matrices displayed the number of true positive, true negative, false positive, and false negative predictions, offering a more granular view of the models' classification performance. The results obtained from this step were instrumental in making informed decisions regarding model selection. They also provided valuable insights into areas where further improvements might be necessary.

6. Results and Discussion

In this chapter, the outcomes of this experimentation with hand grips classification using the proposed models: Convolutional Neural Network (CNN), CNN with Support Vector Machine (SVM) classification, and CNN with K-Nearest Neighbors (KNN) classification will be elaborated. As in section 5.1.6 elaborated, the dataset employed in this thesis comprises two distinct databases, each characterized by unique features and challenges. Here the results obtained for each of these databases will be presented and discussed.

6.1. Database 1

The hand grips classification task for Database 1 was assessed using three different machine learning models: CNN, CNN-SVM, and CNN-KNN. Each model's performance was evaluated across five-folds, and the results are summarized below.

6.1.1. CNN

As stated in Appendix 1, 2, 3, 4, and 5, during the training process of the CNN model, both the training and validation loss were monitored to assess the model's performance. Throughout the training process, the training loss gradually decreased, indicating that the model was learning from the training data. This reduction in training loss is expected as the model adjusts its parameters to minimize the error. Simultaneously, the validation loss was also monitored. In general, the training process was successful, which means the validation loss follows a similar decreasing trend as the training loss, indicating that the model is not overfitting to the training data.

In addition to loss, training and validation accuracy were also tracked during the CNN training process. Training accuracy represents the proportion of correctly classified examples in the training dataset. It provides an indication of how well the model fits the training data. Validation accuracy, on the other hand, measures the model's performance on the validation dataset, which consists of data that the model has not seen during training.

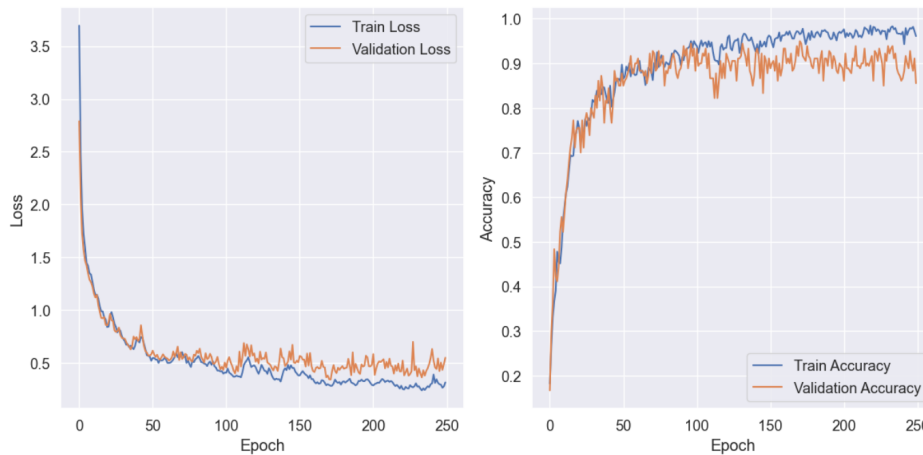


Figure 6.1. Validation and Training Metrics for Database 1

As shown in Figure 6.1, both training and validation accuracy increase as the model learns from the data. An increasing training accuracy signifies that the model is becoming more proficient at classifying the training examples. Simultaneously, validation accuracy is rising indicating that the model is generalizing well to new data. Monitoring accuracy is essential for assessing the model's performance in terms of correct classifications. A large gap between training and validation accuracy may suggest overfitting, as the model may be memorizing the training data rather than learning to generalize. These metrics provide insights into how well the model is learning and generalizing from the data, helping to identify potential issues such as overfitting and ensuring the model's reliability in making predictions on new, unseen data.

The CNN exhibited promising performance in classifying the hand grips dataset from Database 1. The average test accuracy across five folds was approximately 89%, indicating that the CNN correctly classified an average of 89% of the data instances.

As shown in Table 6.1, the classification report is evident that the CNN demonstrated competitive precision, recall, and F1-scores across various hand grip categories. Notably, the CNN achieved strong performance in classifying "Tip" and "Palmar" grips, with F1-scores above 90% for both categories. However, it showed room for improvement in recognizing "Spherical" grips, where recall was comparatively lower.

Table 6.1. Classification Report of CNN for Database 1

Gesture	Precision	Recall	F1-Score
Spherical	0.85	0.85	0.85
Tip	0.97	0.92	0.94
Palmar	0.93	0.86	0.90
Lateral	0.92	0.84	0.88
Cylindrical	0.87	0.92	0.89
Hook	0.82	0.94	0.88
Accuracy	0.89		
Macro Avg	0.89		
Weighted Avg	0.89		

The macro-average F1-score for the CNN was 0.89, highlighting a well-balanced performance across all categories. The weighted average F1-score, also at 0.89, underscored the CNN's consistency in classification. The results suggest that the CNN is a viable choice for hand grips classification in Database 1.

6.1.2. CNN-SVM

The CNN-SVM demonstrated commendable performance in classifying hand grips from Database 1. The average test accuracy across five folds was approximately 91%, indicating that the CNN-SVM accurately classified an average of 91% of the data instances.

As shown in Table 6.2, the classification report is evident that the CNN-SVM exhibited strong precision, recall, and F1-scores across various hand grip categories. Notably, the CNN-SVM excelled in classifying "Tip" and "Cylindrical" grips, achieving F1-scores

above 0.93 for both categories. The CNN-SVM consistently demonstrated competitive performance in recognizing other grips, with F1-scores consistently above 0.87.

The macro-average F1-score for the CNN-SVM was an impressive 0.91, indicating a robust balance between precision and recall across all categories. The weighted average F1-score, also at 0.91, highlighted the CNN-SVM's remarkable consistency in classification. The results suggest that the CNN-SVM is a strong performer for hand grip classification in this database.

Table 6.2. Classification Report of CNN-SVM for Database 1

Gesture	Precision	Recall	F1-Score
Spherical	0.88	0.93	0.91
Tip	0.98	0.95	0.96
Palmar	0.90	0.89	0.90
Lateral	0.89	0.89	0.89
Cylindrical	0.96	0.90	0.93
Hook	0.85	0.89	0.87
Accuracy	0.91		
Macro Avg	0.91		
Weighted Avg	0.91		

6.1.3. CNN-KNN

The CNN-KNN classifier displayed remarkable performance in classifying hand grips from Database 1. The average test accuracy across five folds was approximately 92%, emphasizing the CNN-KNN's proficiency in accurately classifying an average of 92% of the data instances.

The classification report is evident that the CNN-KNN exhibited strong precision, recall, and F1-scores across various hand grip categories, as shown in Table 6.3. Notably, the CNN-KNN excelled in classifying "Tip," "Palmar," and "Cylindrical" grips, achieving F1-scores above 0.94, 0.90, and 0.94, respectively. The CNN-KNN consistently demonstrated competitive performance in recognizing other grips, with F1-scores consistently above 0.87.

The macro-average F1-score for the CNN-KNN was an impressive 0.92, indicating a robust balance between precision and recall across all categories. The weighted average F1-score, also at 0.92, highlighted the CNN-KNN's remarkable consistency in classification. The results strongly suggest that the CNN-KNN is a top-performing model for hand grip classification in Database 1.

Generally, all three models demonstrated strong performance in classifying hand grips from Database 1. The CNN-SVM and CNN-KNN models, in particular, achieved outstanding results, with the CNN-KNN exhibiting the highest accuracy and overall F1-score. These results underscore the suitability and reliability of these models for hand grip classification in this database.

Table 6.3. Classification Report of CNN-KNN for Database 1

Gesture	Precision	Recall	F1-Score
Spherical	0.88	0.97	0.92
Tip	0.98	0.96	0.97
Palmar	0.91	0.90	0.90
Lateral	0.90	0.89	0.90
Cylindrical	0.99	0.91	0.94
Hook	0.87	0.90	0.89
Accuracy	0.92		
Macro Avg	0.92		
Weighted Avg	0.92		

6.1.4. Confusion Matrices

As shown in Figure 6.2, the confusion matrices provide valuable insights into the performance of our machine learning models in classifying hand grips across various categories. These matrices reveal the number of true positive, true negative, false positive, and false negative predictions for each grip category. Across all models, we observed strong diagonal patterns in the confusion matrices, indicating that the majority of hand grips were correctly classified. This alignment with the diagonal signifies high true positive and true negative rates. However, some off-diagonal elements in the confusion matrices suggest occasional misclassifications between certain grips categories. These errors are essential for fine-tuning the models and improving overall accuracy. Despite these minor misclassifications, the overall performance of our models, as reflected in the accuracy, precision, recall, and F1-scores, remains robust and suitable for practical applications in hand grip recognition.

6.2. Database 2

In the hand grip classification task for Database 2, the three machine learning models were evaluated across five folds to assess their performance. The results of this evaluation are summarized below.

6.2.1. CNN

In the training and validation process on Database 2, the deep convolutional neural network (CNN) model exhibited significant learning capabilities, as shown in Figure 6.3. As referred in Appendix 6, 7, 8, 9, and 10, over the course of training, both the training and validation accuracies steadily increased, reaching an impressive accuracy of approximately 91% by the end of the training epochs. Concurrently, the training and validation loss consistently decreased, indicating that the model was effectively minimizing its error and improving its generalization to unseen data. This behavior suggests that the CNN model learned intricate patterns and features within the dataset, which allowed it to make accurate predictions. The convergence of both accuracy and loss curves underscores the

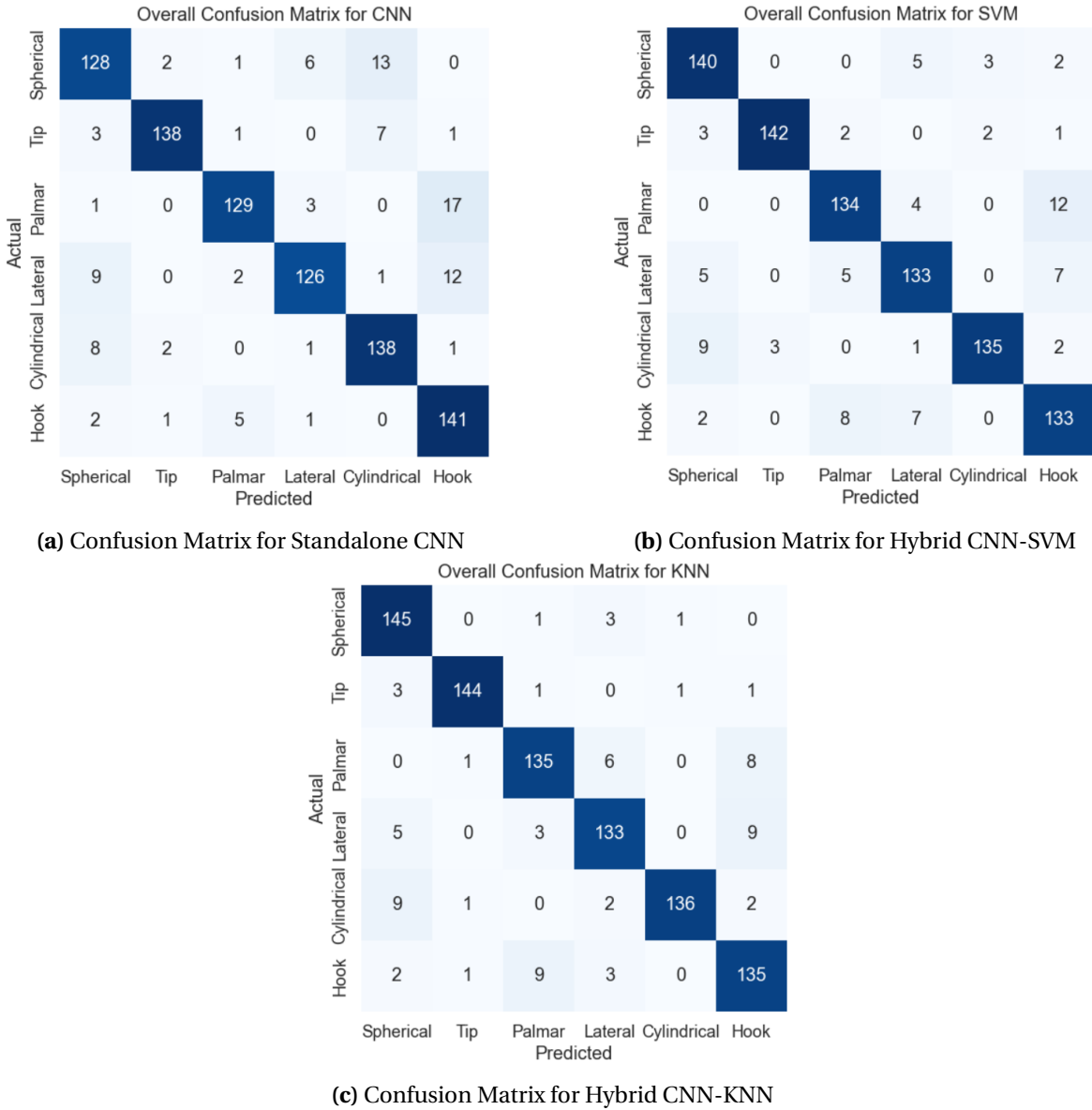


Figure 6.2. :Confusion Matrices of Database 1 for a) Standalone CNN b) Hybrid of CNN and SVM and c) Hybrid of CNN and KNN

robustness and effectiveness of the model in handling the classification tasks associated with Database 2.

The CNN exhibited strong performance in classifying hand grips from Database 2, with an average test accuracy of approximately 91%. This result suggests that the CNN accurately classified an average of 91% of the data instances.

As shown in Table 6.4, the classification report reveals that the CNN demonstrated competitive precision, recall, and F1-scores for various hand grip categories. Notably, the CNN excelled in classifying "Spherical" and "Cylindrical" grips, achieving F1-scores of 0.97 and 0.99, respectively. The model also demonstrated strong performance in recognizing "Tip" grips, with an F1-score of 0.94. However, it showed room for improvement in classifying "Lateral" grips, where recall was comparatively lower.

The macro-average F1-score for the CNN was an impressive 0.91, indicating a well-balanced

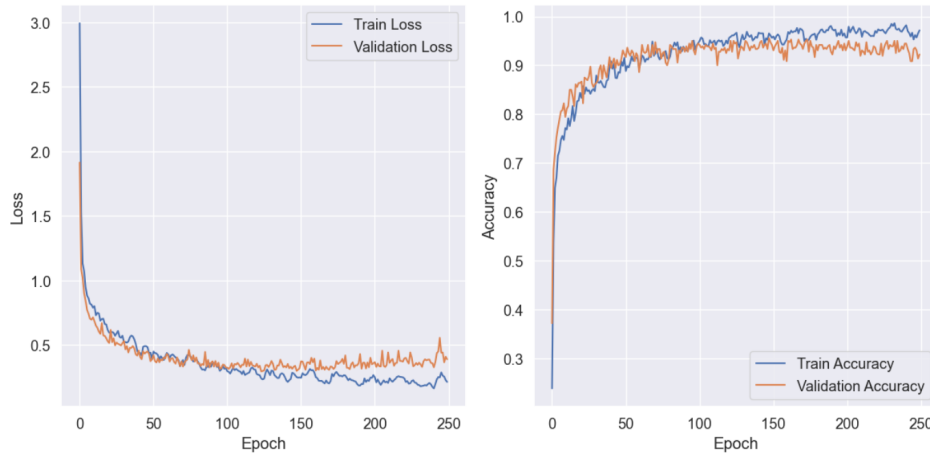


Figure 6.3. Validation and Training Metrics for Database 2

performance across all categories. The weighted average F1-score, also at 0.91, underscored the CNN's consistency in classification. Overall, these results suggest that the CNN is a robust choice for hand grip classification in Database 2.

Table 6.4. Classification Report for CNN for Database 2

Gesture	Precision	Recall	F1-Score
Spherical	0.95	0.98	0.97
Tip	0.94	0.93	0.94
Palmar	0.89	0.86	0.87
Lateral	0.80	0.84	0.82
Cylindrical	1.00	0.98	0.99
Hook	0.90	0.86	0.88
Accuracy	0.91		
Macro Avg	0.91		
Weighted Avg	0.91		

6.2.2. CNN-SVM

The CNN-SVM model showcased remarkable performance in classifying hand grips from Database 2, with an average test accuracy of approximately 93%. This indicates that the CNN-SVM accurately classified an average of 93% of the data instances.

Analyzing the classification report, it is evident that the CNN-SVM demonstrated strong precision, recall, and F1-scores across various hand grip categories, as shown in Table 6.5. The CNN-SVM excelled in classifying "Spherical," "Tip," and "Cylindrical" grips, achieving F1-scores of 0.98, 0.95, and 1.00, respectively. The model consistently exhibited competitive performance in recognizing other grips, with F1-scores consistently above 0.87.

The macro-average F1-score for the CNN-SVM was an impressive 0.93, highlighting a robust balance between precision and recall across all categories. The weighted average F1-score, also at 0.93, emphasized the CNN-SVM's remarkable consistency in classification. These results strongly support the CNN-SVM as an excellent choice for hand grip classification in this database.

Table 6.5. Classification Report for CNN-SVM for Database 2

Gesture	Precision	Recall	F1-Score
Spherical	0.98	0.99	0.98
Tip	0.96	0.95	0.95
Palmar	0.88	0.90	0.89
Lateral	0.83	0.83	0.83
Cylindrical	1.00	1.00	1.00
Hook	0.91	0.88	0.90
Accuracy	0.93		
Macro Avg	0.92		
Weighted Avg	0.92		

6.2.3. CNN-KNN

The CNN-KNN classifier displayed exceptional performance in classifying hand grips from Database 2, with an average test accuracy of approximately 93%. This underscores the CNN-KNN's proficiency in accurately classifying an average of 93% of the data instances.

Reviewing the classification report, it is evident that the CNN-KNN exhibited strong precision, recall, and F1-scores across various hand grip categories, as shown in Table 6.6. The model excelled in classifying "Spherical" and "Cylindrical" grips, achieving F1-scores of 0.99 and 1.00, respectively. It also demonstrated strong performance in recognizing "Tip" grips, with an F1-score of 0.95. However, it exhibited room for improvement in classifying "Lateral" grips, where recall was comparatively lower.

The macro-average F1-score for the CNN-KNN was an impressive 0.93, indicating a robust balance between precision and recall across all categories. The weighted average F1-score, also at 0.93, highlighted the CNN-KNN's remarkable consistency in classification. These results firmly establish the CNN-KNN as a top-performing model for hand grip classification in Database 2.

Table 6.6. Classification Report for CNN-KNN for Database 2

Gesture	Precision	Recall	F1-Score
Spherical	0.98	0.99	0.99
Tip	0.95	0.95	0.95
Palmar	0.89	0.90	0.89
Lateral	0.83	0.88	0.85
Cylindrical	0.99	1.00	1.00
Hook	0.93	0.85	0.89
Accuracy	0.93		
Macro Avg	0.93		
Weighted Avg	0.93		

Generally, all three models—CNN, CNN-SVM, and CNN-KNN, demonstrated strong performance in classifying hand grips from Database 2. The CNN-SVM and CNN-KNN models, in particular, achieved outstanding results, with both models exhibiting high

accuracy and overall F1-scores. These results emphasize the suitability and reliability of these models for hand grip classification in this database.

6.2.4. Confusion Matrices

As shown in Figure 6.4, the confusion matrix for Database 2 reveals the models' performance in classifying hand grips. Across the three models—CNN, CNN-SVM, and CNN-KNN strong diagonal elements indicate accurate predictions, while off-diagonal elements represent misclassifications. Notably, both CNN-SVM and CNN-KNN exhibited minimal misclassifications, as evident from their high diagonal dominance. This reflects their robustness in distinguishing between different hand grip categories. The confusion matrix underscores the CNN-SVM and CNN-KNN models' reliability and effectiveness in achieving high classification accuracy in Database 2.

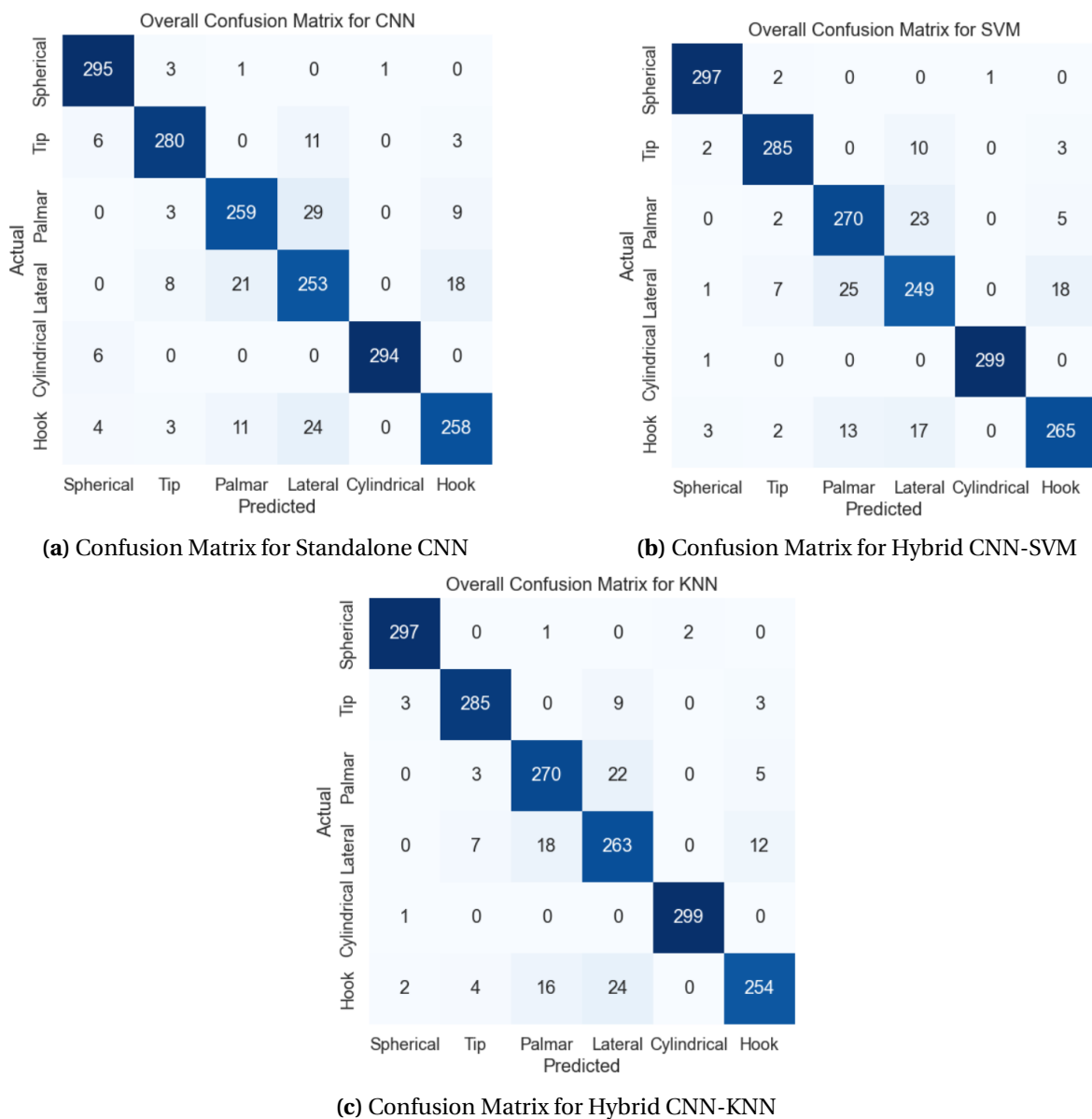


Figure 6.4. Confusion Matrices of Database 2 for a) Standalone CNN b) Hybrid of CNN and SVM and c) Hybrid of CNN and KNN

6.3. Comprehensive Model Performance Analysis by Grip Category

The bar graphs in Database 1 offer a comprehensive view of how three distinct models—CNN, CNN-SVM, and CNN-KNN—perform across various grip categories, as shown in Figure 6.5. Starting with "Spherical" grips, all three models exhibit high precision, indicating their ability to accurately identify this grip with minimal false positives. Remarkably, CNN-KNN stands out with the highest F1 score, suggesting its remarkable equilibrium between precision and recall. This suggests that CNN-KNN excels not only in correctly identifying "Spherical" grips but also in minimizing false negatives, making it a strong choice in this category. Moving to "Tip" grips, CNN and CNN-SVM demonstrate impressive precision,

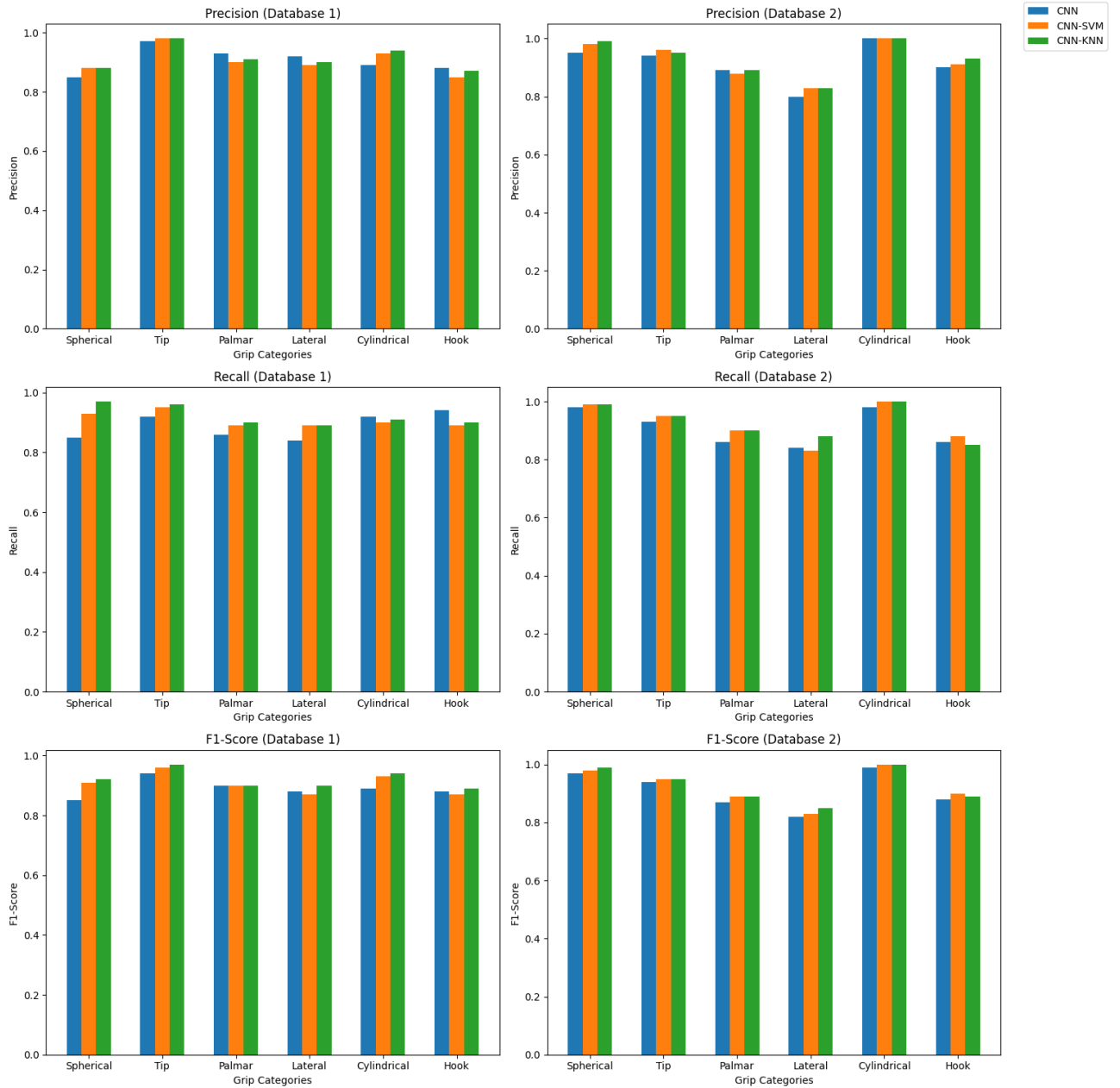


Figure 6.5. Model Performance Comparison by Grip Category for Database 1 and Database 2

with CNN-SVM achieving the highest F1 score. This suggests that CNN-SVM optimally

balances the capture of true positives and minimization of false negatives, positioning it as the top performer for "Tip" grips. For "Palmar" grips, CNN-SVM and CNN-KNN present comparable precision and recall. However, CNN-KNN slightly outperforms CNN-SVM in terms of F1 score, implying its overall superior performance for "Palmar" grips. In the "Lateral" grip category, CNN-KNN showcases the highest precision, while CNN-SVM attains the highest F1 score. This suggests that CNN-SVM effectively balances precision and recall for "Lateral" grips, making it a suitable choice. For "Cylindrical" and "Hook" grips, CNN-KNN shines with the highest precision and F1 scores, underlining its strong performance across these categories.

As shown in Figure 6.5, Database 2's bar graphs continue the detailed model comparisons across different grip categories. "Spherical" grips, yet again, see all models exhibit high precision and recall, resulting in impressive F1 scores. CNN-KNN stands out with the highest F1 score, indicating its exceptional ability to capture true positives and reduce false negatives in this category. In "Tip" grips, CNN-SVM shines with the highest precision, while CNN-KNN achieves the highest F1 score. This suggests that CNN-KNN maintains a well-balanced performance for "Tip" grips, establishing itself as the top performer. For "Palmar" grips, all models maintain relatively consistent precision and recall. CNN-KNN slightly outperforms the others in terms of F1 score, indicating its superior balance between precision and recall for "Palmar" grips. In the "Lateral" grip category, CNN-KNN takes the lead with the highest precision, while CNN-SVM secures the highest F1 score. Once again, this highlights the balanced performance of CNN-SVM for "Lateral" grips. "Cylindrical" and "Hook" grips see CNN-KNN as the top performer with the highest precision and F1 scores, reaffirming its strong and consistent performance.

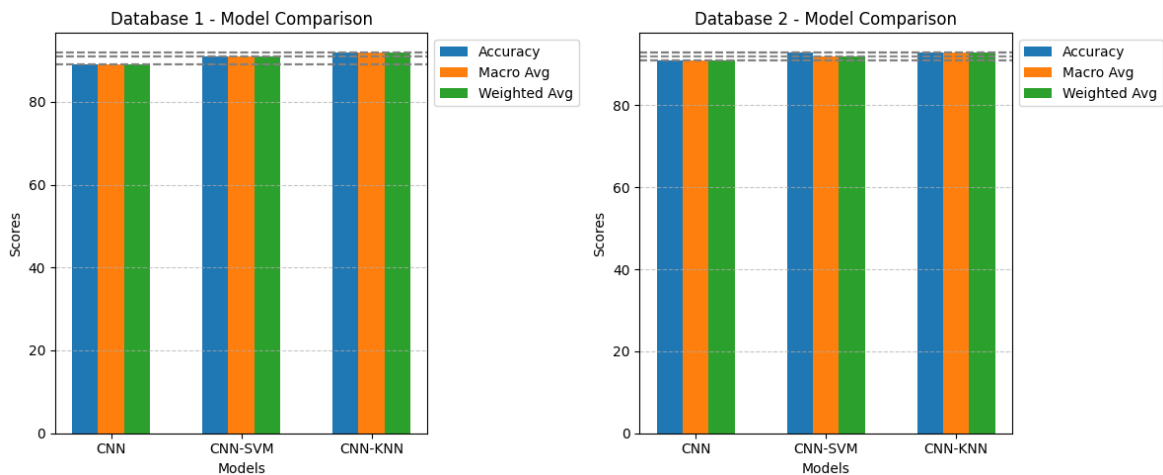


Figure 6.6. Overall Model Performance Comparison

In summary, the bar graphs reveal that CNN-KNN consistently demonstrates strong performance across various grip categories in both databases, achieving high precision, recall, and F1 scores. Meanwhile, CNN-SVM often strikes a delicate balance between precision and recall, making it a robust choice for several grip categories. CNN, while competitive, occasionally lags slightly behind the other models in specific categories.

These insights offer valuable guidance for selecting the most suitable model based on the specific grip category of interest, providing valuable information for practical applications in hand grip recognition.

6.4. Overall Model Performance Comparison

As shown in Figure 6.6, in both databases, the "CNN-KNN" model consistently demonstrates superior performance across multiple metrics, including accuracy, macro average, and weighted average scores. This suggests that the combined approach of Convolutional Neural Networks (CNN) and K-Nearest Neighbors (KNN) in the "CNN-KNN" model enables it to effectively capture complex patterns in the data and make accurate predictions. On the other hand, the "CNN-SVM" model generally performs slightly below the "CNN-KNN" model but outperforms the basic "CNN" model, indicating that the integration of Support Vector Machines (SVM) enhances predictive capabilities. Overall, the results emphasize the significance of hybrid models, incorporating diverse machine learning techniques, in achieving robust and high-performing solutions across different datasets and contexts.

7. Summary

In conclusion, this thesis embarked on a comprehensive exploration of machine learning models for the classification of hand grips using sEMG signals, which find applications in diverse fields ranging from sign language recognition to human-computer interaction. The study employed three models: Convolutional Neural Network (CNN), CNN with Support Vector Machine (SVM), and CNN with K-Nearest Neighbors (KNN).

Across two distinct databases, varying model performance was observed. In Database 1, the CNN achieved an average accuracy of approximately 89%, while the CNN-SVM hybrid excelled with enhanced precision and recall, boasting an average accuracy of about 91%. The CNN-KNN hybrid consistently outperformed, showcasing the benefits of combining deep learning and traditional machine learning, achieving an average accuracy of approximately 92%. In Database 2, the CNN performed strongly with an average accuracy of about 91%, and the CNN-SVM hybrid maintained its excellence, emphasizing its suitability for high-accuracy applications, achieving an average accuracy of approximately 93%. Notably, the CNN-KNN hybrid remained the top performer with an average accuracy of around 93%. It's important to note that the accuracy of all models in Database 1 was lower than in Database 2, which may be due to the differing nature of the databases. The first database exhibits variability as it comprises data from five healthy subjects (two males and three females) of similar ages (approximately 20 to 22 years old), whereas the second database features data from a single healthy subject (male, 22 years old) for three consecutive days.

This research highlights the versatility of machine learning models in the domain of hand grip classification. The choice of the most suitable model should be guided by specific application requirements and constraints. The CNN-SVM and CNN-KNN hybrid models present compelling options for achieving high accuracy and balanced classification performance because hybrid systems derive their remarkable ability to harness the strengths of both deep learning and traditional machine learning. This strategic fusion effectively mitigated individual model limitations, resulting in a holistic and robust approach to hand grip classification.

Future research endeavors could focus on further fine-tuning and optimizing these models, as well as exploring additional feature engineering techniques. This holistic perspective aims to harness the full potential of machine learning models, pushing the boundaries of their capabilities and enhancing their performance across a wide range of applications.

References

- [1] J. R. Cram, "The history of surface electromyography", *Applied Psychophysiology and Biofeedback*, vol. 28, no. 2, pp. 81–91, 2003. DOI: 10.1023/A:1023802407132.
- [2] H. Sharif, S. B. Seo, and T. Kesavadas, "Hand gesture recognition using surface electromyography", in *Annu Int Conf IEEE Eng Med Biol Soc*, 2020, pp. 682–685. DOI: 10.1109/EMBC44109.2020.9175770.
- [3] H. Chen, R. Tong, M. Chen, Y. Fang, and H. Liu, "A hybrid cnn-svm classifier for hand gesture recognition with surface emg signals", in *2018 International Conference on Machine Learning and Cybernetics (ICMLC)*, Chengdu, China, 2018, pp. 619–624. DOI: 10.1109/ICMLC.2018.8526976.
- [4] K. Carter and M. Rutherford. "Muscular system", eCampus Ontario. (Aug. 4, 2020), [Online]. Available: <https://ecampusontario.pressbooks.pub/medicalterminology/chapter/muscular-system/>.
- [5] Biology Dictionary Editors. "Muscle - definition, function, types and structure", *Biology Dictionary*. (Dec. 8, 2017), [Online]. Available: <https://biologydictionary.net/muscle/>.
- [6] Cleveland Clinic Professional. "Muscle", Cleveland Clinic. (), [Online]. Available: <https://my.clevelandclinic.org/health/body/21887-muscle>.
- [7] E. Lord. "Learn about muscle structure", *WeightTraining.Guide*. (Apr. 13, 2020), [Online]. Available: <https://weighttraining.guide/weight-training/learn-about-muscle-structure/>.
- [8] L. M. Biga, S. Bronson, S. Dawson, *et al.* "12.5 the action potential", OpenStax/Oregon State University. (Sep. 26, 2019), [Online]. Available: <https://open.oregonstate.education/aandp/chapter/12-5-the-action-potential/>.
- [9] L. M. Biga, S. Bronson, S. Dawson, *et al.* "10.3 muscle fiber excitation, contraction, and relaxation", OpenStax/Oregon State University. (Sep. 26, 2019), [Online]. Available: <https://open.oregonstate.education/aandp/chapter/10-3-muscle-fiber-excitation-contraction-and-relaxation/>.
- [10] A. Bordoloi and S. Paitya, "Emg controlled automated system", *IOSR Journal of Electronics and Communication Engineering*, vol. 13, no. 3, pp. 16–18, 2018. [Online]. Available: <https://www.iosrjournals.org>.
- [11] B. Taşar and A. Gülten, "Emg-controlled prosthetic hand with fuzzy logic classification algorithm", in *Modern Fuzzy Control Systems and Its Applications*. InTech, 2017. DOI: 10.5772/intechopen.68242.
- [12] Péter, Andersson, Hegyi, *et al.*, "Comparing surface and fine-wire electromyography activity of lower leg muscles at different walking speeds", *Frontiers in Physiology*, vol. 10, 2019. DOI: 10.3389/fphys.2019.01283.
- [13] A. Waris, M. Z. u. Rehman, I. K. Niazi, *et al.*, "A multiday evaluation of real-time intramuscular EMG usability with ANN", *Sensors*, vol. 20, no. 12, 2020. DOI: 10.3390/s20123385.

- [14] M. B. Raez, M. S. Hussain, and F. Mohd-Yasin, "Techniques of EMG signal analysis: Detection, processing, classification and applications", *Biological Procedures Online*, vol. 8, pp. 11–35, 2006, Erratum in: *Biological Procedures Online*. 2006;8:163. DOI: 10.1251/bpo115.
- [15] N. Amrutha and V. H. Arul, "A review on noises in EMG signal and its removal", *International Journal of Scientific Research Publications*, vol. 7, no. 5, 2017, ISSN: 2250-3153. [Online]. Available: <http://www.ijsrp.org/research-paper-0517.php?rp=P656368>.
- [16] N. Nazmi, M. A. Abdul Rahman, S.-I. Yamamoto, S. A. Ahmad, H. Zamzuri, and S. A. Mazlan, "A review of classification techniques of EMG signals during isotonic and isometric contractions", *Sensors*, vol. 16, no. 8, p. 1304, 2016. DOI: 10.3390/s16081304.
- [17] S. Day, "Important factors in surface EMG measurement", *Bortec Biomedical Ltd Publishers*, pp. 1–17, 2002.
- [18] R. H. Chowdhury, M. B. I. Reaz, M. A. B. M. Ali, A. A. A. Bakar, K. Chellappan, and T. G. Chang, "Surface electromyography signal processing and classification techniques", *Sensors*, vol. 13, no. 9, pp. 12 431–12 466, 2013. DOI: 10.3390/s130912431.
- [19] J. Kumar, A. Rani, N. Kumar, and N. K. Sinha. "Machine learning for soil moisture assessment". Unknown. (Jan. 11, 2022), [Online]. Available: https://www.researchgate.net/publication/357744548_Machine_learning_for_soil_moisture_assessment.
- [20] G.-F. Fan, Y.-H. Guo, J.-M. Zheng, and W.-C. Hong, "Application of the weighted k-nearest neighbor algorithm for short-term load forecasting", *Energies*, vol. 12, no. 5, 2019. DOI: 10.3390/en12050916.
- [21] P. Carrión. "Basics of clustering - pedro carrión". Medium. (Aug. 8, 2022), [Online]. Available: <https://medium.com/@pedroecarrionz/basics-of-clustering-450160f3b43b>.
- [22] X. Yuan, Y. Wang, R. Zhang, *et al.*, "Reinforcement learning control of hydraulic servo system based on TD3 algorithm", *Machines*, vol. 10, no. 12, 2022. DOI: 10.3390/machines10121244.
- [23] Z. E. Mohamed, "Using the artificial neural networks for prediction and validating solar radiation", *Journal of the Egyptian Mathematical Society*, vol. 27, no. 1, pp. 1–13, 2019. DOI: 10.1186/s42787-019-0043-8. [Online]. Available: <https://doi.org/10.1186/s42787-019-0043-8>.
- [24] J. R. Jensen. "Figure 2. the topology of a typical multiple layer neural network..." ResearchGate. (n.d.), [Online]. Available: https://www.researchgate.net/figure/The-topology-of-a-typical-multiple-layer-neural-network-consisting-of-one-input-layer_fig2_228589481 (visited on 07/27/2023).
- [25] N. Jindal, *Weight sharing in convolution neural networks?*, ResearchGate, 2022. [Online]. Available: https://www.researchgate.net/post/Weight_Sharing_in_Convolution_Neural_Networks.

-
- [26] M. S. Alhajeri, J. Luo, Z. Wu, F. Albalawi, and P. D. Christofides, “Process structure-based recurrent neural network modeling for predictive control: A comparative study”, *Chemical Engineering Research and Design*, vol. 179, pp. 77–89, 2022, ISSN: 0263-8762. DOI: 10.1016/j.cherd.2021.12.046.
 - [27] I. Jenkins, L. Gee, A. Knauss, H. Yin, and J. Schroeder, “Accident scenario generation with recurrent neural networks”, Nov. 2018, pp. 3340–3345. DOI: 10.1109/ITSC.2018.8569661.
 - [28] M. Atzori, M. Cognolato, and H. Müller, “Deep learning with convolutional neural networks applied to electromyography data: A resource for the classification of movements for prosthetic hands”, *Frontiers in Neurorobotics*, vol. 10, p. 9, 2016. DOI: 10.3389/fnbot.2016.00009.
 - [29] W. Geng, Y. Du, W. Jin, W. Wei, Y. Hu, and J. Li, “Gesture recognition by instantaneous surface emg images”, *Scientific Reports*, vol. 6, p. 36571, 2016. DOI: 10.1038/srep36571. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5100974/>.
 - [30] K. Bakircioglu and N. Özkurt, “Classification of emg signals using convolution neural network”, *International Journal of Applied Mathematics Electronics and Computers*, vol. 8, no. 4, pp. 115–119, 2020. DOI: 10.18100/ijamec.795227.
 - [31] C. Sapsanis, G. Georgoulas, A. Tzes, and D. Lymberopoulos, “Improving emg based classification of basic hand movements using emd”, 2013. DOI: 10.1109/EMBC.2013.6610858.
 - [32] M. Atzori, A. Gijssberts, C. Castellini, *et al.*, “Electromyography data for non-invasive naturally-controlled robotic hand prostheses”, *Scientific Data*, vol. 1, p. 140053, 2014. DOI: 10.1038/sdata.2014.53. [Online]. Available: <https://www.nature.com/articles/sdata201453>.
 - [33] Z. Ding, C. Yang, Z. Tian, C. Yi, Y. Fu, and F. Jiang, “Semg-based gesture recognition with convolution neural networks”, *Sustainability*, vol. 10, no. 6, 2018. DOI: 10.3390/su10061865. [Online]. Available: <https://www.mdpi.com/2071-1050/10/6/1865>.
 - [34] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
 - [35] D. Leserri, N. Grimmelsmann, M. Mechtenberg, H. G. Meyer, and A. Schneider, “Evaluation of semg signal features and segmentation parameters for limb movement prediction using a feedforward neural network”, *Mathematics*, vol. 10, 2022. [Online]. Available: <https://www.mdpi.com/2227-7390/10/6/932>.
 - [36] J. N. Peng and J. Authors, “Gesture recognition by ensemble extreme learning machine based on surface electromyography signals”, *Frontiers in Human Neuroscience*, vol. 16, Jun. 2022. DOI: 10.3389/fnhum.2022.911204.
 - [37] R. Bagherian Azhiri, “Real-time emg signal classification via recurrent neural networks”, in *Proceedings of the 2021 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, IEEE, Dec. 2021.

- [38] A. S. U. Doulah, S. A. Fattah, W. Zhu, and M. Ahmad, "Wavelet domain feature extraction scheme based on dominant motor unit action potential of emg signal for neuromuscular disease classification", *IEEE Transactions on Biomedical Circuits and Systems*, vol. 8, no. 2, pp. 155–164, 2014. DOI: 10.1109/TBCAS.2013.2295697.
- [39] C. Millar, N. Siddique, and E. Kerr, "Lstm classification of semg signals for individual finger movements using low cost wearable sensor", in *2020 International Symposium on Community-centric Systems (CcS)*, 2020, pp. 1–8. DOI: 10.1109/CcS49175.2020.9231515.
- [40] S. Hochreiter and J. U. Schmidhuber, "Long short-term memory", *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [41] Y. Wu, B. Zheng, and Y. Zhao, "Dynamic gesture recognition based on lstm-cnn", in *Proc. 2018 Chinese Autom. Congr. CAC 2018*, 2019, pp. 2446–2450.
- [42] N. Arbel. "How lstm networks solve the problem of vanishing gradients". (May 2020), [Online]. Available: <https://medium.datadriveninvestor.com/how-do-lstm-networks-solve-the-problem-of-vanishing-gradients-a6784971a577>.
- [43] D. Toledo Pérez, J. Rodriguez, R. Gómez Loenzo, and J. Jauregui, "Support vector machine-based emg signal classification techniques: A review", *Applied Sciences*, vol. 9, p. 4402, Oct. 2019. DOI: 10.3390/app9204402.
- [44] V. Vapnik and C. Corinna, "Support-vector networks", *Machine Learning*, vol. 20, pp. 273–297, 1995.
- [45] W. Chen and Z. Zhang, "Hand gesture recognition using semg signals based on support vector machine", in *2019 IEEE 8th Joint International Information Technology and Artificial Intelligence Conference (ITAIC)*, 2019, pp. 230–234. DOI: 10.1109/ITAIC.2019.8785542.
- [46] T. M. Cover, "Rates of convergence for nearest neighbor procedures", in *Hawaii International Conference on System Sciences*, 1968.
- [47] B. Wan, R. Wu, K. Zhang, and L. Liu, "A new subtle hand gestures recognition algorithm based on emg and fsr", in *2017 IEEE 21st International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 2017, pp. 127–132. DOI: 10.1109/CSCWD.2017.8066682.
- [48] M. Haris, P. Chakraborty, and B. V. Rao, "Emg signal based finger movement recognition for prosthetic hand control", in *2015 Communication, Control and Intelligent Systems (CCIS)*, 2015, pp. 194–198. DOI: 10.1109/CCInteIS.2015.7437907.
- [49] L. Breiman, J. Friedman, R. Ohlsen, and C. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth International Group, 1984, pp. 43–49.
- [50] R. Quinlan, *C4.5: Programs for Machine Learning*. USA: Morgan Kaufmann, 1993.
- [51] E. Gokgoz and A. Subasi, "Comparison of decision tree algorithms for emg signal classification using dwt", *Biomedical Signal Processing and Control*, vol. 18, pp. 138–144, 2015, ISSN: 1746-8094.
- [52] R. Jia, L. Yang, Y. Li, and Z. Xin, "Gestures recognition of semg signal based on random forest", in *2021 IEEE 16th Conference on Industrial Electronics and Applications (ICIEA)*, 2021, pp. 1673–1678. DOI: 10.1109/ICIEA51954.2021.9516350.

-
- [53] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd. Prentice Hall, 2009.
 - [54] D. S. Pamungkas, I. Simatupang, and S. K. Risandriya, “Comparison gestures recognition using k-nn and naïve bayes”, in *2020 International Conference on Applied Science and Technology (iCAST)*, 2020, pp. 677–681. DOI: 10.1109/iCAST51016.2020.9557730.
 - [55] CatalyzeX. “Emg signal classification using reflection coefficients and extreme value machine: Paper and code”. Accessed on September 3, 2023. (2023), [Online]. Available: <https://www.catalyzex.com/paper/arxiv:2106.10561>.
 - [56] H. Chen, R. Tong, M. Chen, Y. Fang, and H. Liu, “A hybrid cnn-svm classifier for hand gesture recognition with surface emg signals”, in *2018 International Conference on Machine Learning and Cybernetics (ICMLC)*, vol. 2, 2018, pp. 619–624. DOI: 10.1109/ICMLC.2018.8526976.
 - [57] Y. Hu, Y. Wong, W. Wei, Y. Du, M. Kankanhalli, and W. Geng, “A novel attention-based hybrid cnn-rnn architecture for semg-based gesture recognition”, *PloS One*, vol. 13, no. 10, e0206049, 2018. DOI: 10.1371/journal.pone.0206049.
 - [58] S. Goferman, L. Zelnik-Manor, and A. Tal, “Context-aware saliency detection”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 10, pp. 1915–1926, 2012. DOI: 10.1109/TPAMI.2011.272.
 - [59] M. Ortiz-Catalan, R. Brånemark, and B. Håkansson, “Biopatrec: A modular research platform for the control of artificial limbs based on pattern recognition algorithms”, *Source code for biology and medicine*, vol. 8, no. 1, p. 11, 2013. DOI: 10.1186/1751-0473-8-11.
 - [60] C. Amma, T. Krings, J. Böer, and T. Schultz, “Advancing muscle-computer interfaces with high-density electromyography”, in *ACM Conference on Human Factors in Computing Systems*, 2015, pp. 929–938.
 - [61] X. Zhai, B. Jelfs, R. Chan, and C. Tin, “Self-recalibrating surface emg pattern recognition for neuroprosthesis control based on convolutional neural network”, *Frontiers in neuroscience*, vol. 11, p. 379, 2017. DOI: 10.3389/fnins.2017.00379.
 - [62] N. K. Karnam, S. R. Dubey, A. C. Turlapaty, and B. Gokaraju, “Emghandnet: A hybrid cnn and bi-lstm architecture for hand activity classification using surface emg signals”, *Biocybernetics and Biomedical Engineering*, vol. 42, no. 1, pp. 325–340, 2022. DOI: 10.1016/j.bbe.2022.02.005.
 - [63] E. Criswell, *Cram’s introduction to surface electromyography*. Jones & Bartlett Publishers, 2010.
 - [64] M. Rossi, S. Benatti, E. Farella, and L. Benini, “Hybrid emg classifier based on hmm and svm for hand gesture recognition in prosthetics”, in *2015 IEEE International Conference on Industrial Technology (ICIT)*, 2015, pp. 1700–1705. DOI: 10.1109/ICIT.2015.7125342.
 - [65] A. D. Chan and K. B. Englehart, “Continuous myoelectric control for powered prostheses using hidden markov models”, *Biomedical Engineering IEEE Transactions on*, vol. 52, no. 1, pp. 121–124, 2005.

- [66] B. E. Boser, I. M. Guyon, and V. N. Vapnik, "A training algorithm for optimal margin classifiers", in *Proceedings of the fifth annual workshop on Computational learning theory*, 1992, pp. 144–152.
- [67] T. W. Calvert and A. E. Chapman, "The relationship between the surface emg and force transients in muscle: Simulation and experimental studies", *Proceedings of the IEEE*, vol. 65, no. 5, pp. 682–689, 1977.
- [68] K. Englehart, B. Hudgin, and A. P. Parker, "A wavelet-based continuous classification scheme for multifunction myoelectric control", *Biomedical Engineering IEEE Transactions on*, vol. 48, no. 3, pp. 302–311, 2001.
- [69] C. Sapsanis, A. Tzes, and G. Georgoulas, *Semg for basic hand movements*, UCI Machine Learning Repository, 2014. [Online]. Available: <https://doi.org/10.24432/C5TK53>.
- [70] M. M. Taye, "Theoretical understanding of convolutional neural network: Concepts, architectures, applications, future directions", *Computation*, vol. 11, no. 3, 2023, ISSN: 2079-3197. DOI: 10.3390/computation11030052. [Online]. Available: <https://www.mdpi.com/2079-3197/11/3/52>.
- [71] H. Ye, *Fig. 3. the overview of preprocessing and the automatic feature extraction procedure*, ResearchGate, Accessed on Sep. 05, 2023, 2023. [Online]. Available: https://www.researchgate.net/figure/The-overview-of-preprocessing-and-the-automatic-feature-extraction-procedure-After_fig3_346496981.
- [72] S. Dey. "Cnn application on structured data - automated feature extraction". Accessed on Sep. 05, 2023. (Aug. 2023), [Online]. Available: <https://towardsdatascience.com/cnn-application-on-structured-data-automated-feature-extraction-8f2cd28d9a7e>.
- [73] D. K. "Top 4 advantages and disadvantages of support vector machine or svm". Accessed on Sep. 05, 2023. (Feb. 2023), [Online]. Available: <https://dhirajkumarblog.medium.com/top-4-advantages-and-disadvantages-of-support-vector-machine-or-svm-a3c06a2b107>.
- [74] J. Ma, A. Krishnamurthy, and S. Ahalt, "Svm training with duplicated samples and its application in svm-based ensemble methods", *Neurocomputing*, vol. 61, pp. 455–459, 2004, Hybrid Neurocomputing: Selected Papers from the 2nd International Conference on Hybrid Intelligent Systems, ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2004.04.004>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231204002395>.
- [75] e. a. Cai, "Svm-based classification of semg signals for upper-limb self-rehabilitation training", *Frontiers in Neurorobotics*, vol. 13, Jun. 2019. DOI: 10.3389/fnbot.2019.00031.
- [76] E. Raczko and B. Zagajewski, "Comparison of support vector machine, random forest and neural network classifiers for tree species classification on airborne hyperspectral apex images", *European Journal of Remote Sensing*, vol. 50, no. 1, pp. 144–154, 2017. DOI: 10.1080/22797254.2017.1299557. eprint: <https://doi.org/10.1080/22797254.2017.1299557>.

- 1080/22797254.2017.1299557. [Online]. Available: <https://doi.org/10.1080/22797254.2017.1299557>.
- [77] P. Gopal, A. Gesta, and A. Mohebbi, “A systematic study on electromyography-based hand gesture recognition for assistive robots using deep learning and machine learning models”, *Sensors*, vol. 22, no. 10, 2022, ISSN: 1424-8220. DOI: 10.3390/s22103650. [Online]. Available: <https://www.mdpi.com/1424-8220/22/10/3650>.
- [78] A. Raj. “Everything about support vector classification — above and beyond”. Accessed on Sep. 05, 2023. (Aug. 2022), [Online]. Available: <https://towardsdatascience.com/everything-about-svm-classification-above-and-beyond-cc665bfd993e>.
- [79] P. P. Pradhan and M. K. Soren, “A comparative analysis on emg signal classification approaches”, *International Research Journal of Engineering and Technology (IRJET)*, vol. 07, p. 1885, 08 2020, IRJET Impact Factor: 7.529, ISO 9001:2008 Certified Journal, ISSN: e-ISSN: 2395-0056, p-ISSN: 2395-0072. [Online]. Available: www.irjet.net.
- [80] Fricke, Alizadeh, Zakhary, Woost, Bogdan, and Classen, “Evaluation of three machine learning algorithms for the automatic classification of emg patterns in gait disorders”, *Frontiers in Neurology*, vol. 12, May 2021. DOI: 10.3389/fneur.2021.666458.
- [81] Y. Huo, F. Li, Q. Li, E. He, and J. Chen, “A novel method for hand movement recognition based on wavelet packet transform and principal component analysis with surface electromyogram”, *Computational Intelligence and Neuroscience*, vol. 2022, Nov. 2022. DOI: 10.1155/2022/8125186.
- [82] Bukhari. “Study of k-nearest neighbour classification performance on fatigue and non-fatigue emg signal features”. Accessed on Sep. 24, 2023. (2020), [Online]. Available: <https://openresearch.lsbu.ac.uk/item/8v81q>.
- [83] B. A. Alabsi, M. Anbar, and S. D. A. Rihan, “Cnn-cnn: Dual convolutional neural network approach for feature selection and attack detection on internet of things networks”, *Sensors*, vol. 23, no. 14, 2023, ISSN: 1424-8220. DOI: 10.3390/s23146507. [Online]. Available: <https://www.mdpi.com/1424-8220/23/14/6507>.
- [84] A. Bahador, M. Yousefi, M. Marashi, and O. Bahador, “High accurate lightweight deep learning method for gesture recognition based on surface electromyography”, *Computer Methods and Programs in Biomedicine*, vol. 195, p. 105643, Oct. 2020. DOI: 10.1016/j.cmpb.2020.105643.
- [85] R. Yamashita, M. Nishio, and R. e. a. Do, “Convolutional neural networks: An overview and application in radiology”, *Insights in Imaging*, vol. 9, pp. 611–629, 2018. DOI: 10.1007/s13244-018-0639-9.
- [86] F. Demir, V. Bajaj, M. C. Ince, S. Taran, and A. Şengür, “Surface emg signals and deep transfer learning-based physical action classification”, *Neural Computing and Applications*, vol. 31, no. 12, pp. 8455–8462, Oct. 2019. DOI: 10.1007/s00521-019-04553-7.

List of Symbols and Abbreviations

ACT – Average Conduction Time
ANN – Artificial Neural Network
AR – Auto-Regressive Coefficients
Bi-LSTM – Bidirectional Long Short-Term Memory
CC – Classical Classification
CNN – Convolutional Neural Networks
CNN-KNN – Hybrid of Convolutional Neural Networks with Support Vector Machines
CNN-RNN – Hybrid of Convolutional Neural Networks with Recurrent Neural Networks
CNN-SVM – Hybrid of Convolutional Neural Networks with K-Nearest Neighbors
DBSCAN – Density-Based Spatial Clustering of Applications with Noise
DNN – Deep Neural Network
EELM – Eagle Eye Learning Model
ELM – Extreme Learning Machine
ELU – Exponential Linear Unit
EMD – Empirical Mode Decomposition
EMG – Electromyography
EVM – Extreme Value Machine
FFNN – Feedforward Neural Network
FMD – Frequency Median
FN – False Negative
FP – False Positive
FT – The Fourier Transform
GRU – Gated Recurrent Units
HMM-SVM – Combination of Hidden Markov Models with Support Vector Machines
IAV – Integrated Absolute Value
IEMG – Integrated EMG
iEMG – Intramuscular Electromyography
KNN – K-Nearest Neighbors
Leaky ReLU – Leaky Rectified Linear Unit
LSTM – Long Short-Term Memory
MAV – Mean Absolute Value
MLPs – Multiple Layer Neural Network
MNF – Median Frequency
PCA – Principal Component Analysis
PSD – Power Spectral Density
ReLU – Rectified Linear Unit
RF-RFE – Random Forest - Recursive Feature Elimination
RL – Reinforcement Learning
RMS – Root Mean Square
RNNs – Recurrent Neural Networks

sEMG – Surface EMG
SSC – Slope Sign Changes
STFT – Short Time Fourier Transform
SVM – Support Vector Machines
SVM – Support Vector Machines
TN – True Negative
TP – True Positive
WAMP – Wilson Amplitude
ZC – Zero Crossing

List of Figures

2.1	Connective Tissue Layers [5]	12
2.2	Types of Muscle Tissue [7]	13
2.3	Action Potential [8]	13
2.4	Sample Raw EMG Signal	14
2.5	Measurement of Electromyography[10]	15
2.6	Intramuscular Electrode Insertion Sites on the Forearm [13]	15
2.7	Components of Support Vector Machine (SVM) [19]	21
2.8	Example for KNN Algorithm (k=5) [20]	21
2.9	Centroid-based Clustering [21]	22
2.10	The Fundamental Framework of Reinforcement Learning (RL) [22]	23
2.11	Simple Structure of ANN [23]	24
2.12	Binary Step Function	25
2.13	Linear Activation Function	25
2.14	Sigmoid Activation Function	26
2.15	Hyperbolic Tangent Function	26
2.16	Rectified Linear Unit (ReLU)	27
2.17	Leaky Rectified Linear Unit	27
2.18	Exponential Linear Unit	28
2.19	Softmax Activation Function	28
2.20	Components of Multiple Layer Neural Network [24]	29
2.21	Components of Convolutional Neural Network [25]	30
2.22	Components of Recurrent Neural Networks (RNNs) [26]	31
2.23	Fundamental Components of an LSTM Cell [27]	32
3.1	Four Different CNN Architectures [30]	33
3.2	CNN Structure Utilized for the Classification of the sEMG Signal [28]	34
3.3	CNN Structure Utilized for the Classification of the sEMG Signal [33]	35
3.4	Block Diagram for Prediction Using a Feedforward Neural Network [35]	36
3.5	ELM Block Diagram to Classify sEMG Signal [36]	37
3.6	RNN and BRNN Architectures to Classify sEMG Signal [37]	38

3.7	LSTM Architecture for Real-Time EMG Signal Classification [39]	39
3.8	Geometry of Support Vector Machine (SVM) [43]	40
3.9	Block Diagram for KNN and ANN Classification [48]	42
3.10	Block Diagram for Decision Trees Classification [51]	43
3.11	Block Diagram for Native Bayes Classification [54]	44
3.12	Block Diagram for EMV Classification [55]	44
3.13	Hybrid CNN-SVM Architecture for sEMG Signal Classification [56]	45
3.14	Six Different sEMG Raw Signal Used for Classification [57]	46
3.15	Hybrid CNN-RNN Architecture for sEMG Signal Classification [57]	46
3.16	Hybrid CNN-LSTM Architecture for sEMG Signal Classification [62]	47
3.17	Block Diagram for CNN-LSTM Classification [62]	48
3.18	EMG Data Acquisition for HMM-SVM Classification [64]	48
3.19	Classification of sEMG Based on Category [64]	49
4.1	Data Acquisition Procedures [68]	50
4.2	Feature Extraction by Using CNN [71]	52
4.3	Traditional Machine Learning Classification with Manual Feature Extractors [77]	54
4.4	Stand Alone CNN Classifier [84]	56
4.5	Combination of CNN with Traditional Machine Learning Classifier [86]	58
5.1	Six Different Hand Movements [69]	62
5.2	Machine Learning Pipeline Flowchart	66
5.3	Folder Structure of the Dataset	67
5.4	Proposed Classification Models	71
6.1	Validation and Training Metrics for Database 1	73
6.2	:Confusion Matrices of Database 1 for a) Standalone CNN b) Hybrid of CNN and SVM and c) Hybrid of CNN and KNN	77
6.3	Validation and Training Metrics for Database 2	78
6.4	Confusion Matrices of Database 2 for a) Standalone CNN b) Hybrid of CNN and SVM and c) Hybrid of CNN and KNN	80
6.5	Model Performance Comparison by Grip Category for Database 1 and Database 2	81
6.6	Overall Model Performance Comparison	82

List of Tables

5.1	Dataset Characteristics	63
5.2	Hyperparameters	68
5.3	Proposed CNN Model Configuration	69
6.1	Classification Report of CNN for Database 1	74
6.2	Classification Report of CNN-SVM for Database 1	75
6.3	Classification Report of CNN-KNN for Database 1	76
6.4	Classification Report for CNN for Database 2	78

6.5	Classification Report for CNN-SVM for Database 2	79
6.6	Classification Report for CNN-KNN for Database 2	79

List of Appendices

1.	Training and Validation Metrics in Fold 1 for Database 1	96
2.	Training and Validation Metrics in Fold 2 for Database 1	97
3.	Training and Validation Metrics in Fold 3 for Database 1	98
4.	Training and Validation Metrics in Fold 4 for Database 1	99
5.	Training and Validation Metrics in Fold 5 for Database 1	100
6.	Training and Validation Metrics in Fold 1 for Database 2	101
7.	Training and Validation Metrics in Fold 2 for Database 2	102
8.	Training and Validation Metrics in Fold 3 for Database 2	103
9.	Training and Validation Metrics in Fold 4 for Database 2	104
10.	Training and Validation Metrics in Fold 5 for Database 2	105
11.	Model Evaluation Metrics for Database 1	106
12.	Model Evaluation Metrics for Database 2	107

Appendix 1. Training and Validation Metrics in Fold 1 for Database 1

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	3.5683	0.1736	2.5709	0.2389
2	2.2623	0.2750	1.8521	0.3222
3	1.7553	0.3694	1.5658	0.4389
4	1.5326	0.4236	1.5651	0.4500
5	1.5564	0.4292	1.4296	0.4611
6	1.4583	0.4194	1.3784	0.4889
7	1.3994	0.4708	1.2814	0.5056
8	1.3699	0.4569	1.2819	0.5111
9	1.3565	0.4875	1.2249	0.5444
10	1.2907	0.5083	1.1919	0.5611
...				
100	0.3577	0.9542	0.4734	0.9333
101	0.3684	0.9403	0.5038	0.8944
102	0.3599	0.9542	0.4349	0.9278
103	0.3704	0.9528	0.7956	0.8278
104	0.4948	0.9250	0.5970	0.8444
105	0.4977	0.9097	0.5642	0.8833
106	0.5131	0.9139	0.5556	0.9056
107	0.4334	0.9514	0.5824	0.8833
108	0.4123	0.9597	0.6228	0.8889
109	0.4400	0.9306	0.5910	0.8667
110	0.4022	0.9514	0.5125	0.9000
...				
240	0.4303	0.9194	0.5082	0.8778
241	0.3917	0.9472	0.4701	0.9222
242	0.3581	0.9708	0.4406	0.9278
243	0.3426	0.9667	0.4605	0.9333
244	0.3391	0.9667	0.4432	0.9333
245	0.3186	0.9694	0.4778	0.9111
246	0.3171	0.9639	0.3904	0.9167
247	0.3247	0.9681	0.4006	0.9278
248	0.2811	0.9778	0.3676	0.9389
249	0.2586	0.9861	0.4066	0.9167

Appendix 2. Training and Validation Metrics in Fold 2 for Database 1

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	3.5469	0.1764	2.5835	0.3500
2	2.2584	0.3042	1.8160	0.4056
3	1.7248	0.3875	1.5433	0.4389
4	1.5545	0.4139	1.4345	0.4111
5	1.4879	0.4292	1.3740	0.4722
6	1.4427	0.4569	1.3252	0.4833
7	1.3742	0.4986	1.2767	0.4889
8	1.3734	0.4750	1.2864	0.5056
9	1.3728	0.4528	1.2837	0.5000
10	1.2940	0.5292	1.3335	0.5444
...				
100	0.4585	0.9458	0.6966	0.8333
101	0.4462	0.9486	0.6025	0.8667
102	0.4259	0.9542	0.6760	0.8500
103	0.4054	0.9653	0.6308	0.8556
104	0.3739	0.9736	0.6390	0.8444
105	0.3883	0.9611	0.6631	0.8444
106	0.4019	0.9514	0.5839	0.8333
107	0.3830	0.9542	0.6640	0.8389
108	0.4128	0.9458	0.5835	0.8611
109	0.4016	0.9444	0.7230	0.8111
110	0.4185	0.9556	0.6882	0.8111
...				
240	0.3580	0.9542	0.6548	0.8722
241	0.3948	0.9514	0.5896	0.9000
242	0.3622	0.9750	0.6864	0.8556
243	0.3318	0.9875	0.6272	0.8944
244	0.3171	0.9875	0.5683	0.9000
245	0.3265	0.9736	0.5878	0.8889
246	0.2903	0.9792	0.5557	0.9056
247	0.2947	0.9833	0.6185	0.8667
248	0.3072	0.9736	0.5874	0.8944
249	0.2879	0.9792	0.5936	0.8778

Appendix 3. Training and Validation Metrics in Fold 3 for Database 1

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	3.5474	0.1819	2.6021	0.2611
2	2.2440	0.2931	1.8376	0.3167
3	1.7246	0.3653	1.5753	0.4111
4	1.5463	0.3944	1.5069	0.3667
5	1.4952	0.4222	1.4587	0.4333
6	1.4484	0.4181	1.3815	0.4222
7	1.3787	0.4694	1.3478	0.4278
8	1.3696	0.4431	1.3302	0.5111
9	1.3172	0.5097	1.3370	0.4556
10	1.2954	0.4819	1.2594	0.4778
...				
100	0.4088	0.9500	0.6124	0.8667
101	0.4761	0.9278	0.7839	0.8000
102	0.4995	0.9250	0.6450	0.8556
103	0.4507	0.9528	0.6053	0.9056
104	0.4345	0.9514	0.5502	0.9000
105	0.4123	0.9500	0.5573	0.9000
106	0.3796	0.9597	0.5692	0.8889
107	0.3716	0.9653	0.5301	0.8889
108	0.3521	0.9722	0.5029	0.8944
109	0.3539	0.9681	0.5360	0.8722
110	0.3441	0.9722	0.5294	0.8722
...				
240	0.2870	0.9875	0.6056	0.8500
241	0.2898	0.9792	0.4946	0.9056
242	0.2583	0.9903	0.4504	0.9333
243	0.2632	0.9833	0.4854	0.8833
244	0.3821	0.9403	1.1209	0.7444
245	0.4106	0.9389	0.6535	0.8333
246	0.3896	0.9653	0.5766	0.8722
247	0.3762	0.9736	0.7087	0.8667
248	0.4260	0.9542	0.8387	0.8278
249	0.4646	0.9444	0.8049	0.7889

Appendix 4. Training and Validation Metrics in Fold 4 for Database 1

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	3.5343	0.1681	2.5571	0.1778
2	2.2030	0.2597	1.8333	0.3056
3	1.7232	0.3653	1.5639	0.4278
4	1.5571	0.4014	1.4757	0.4389
5	1.5055	0.4111	1.4611	0.4611
6	1.4424	0.4625	1.3909	0.4444
7	1.3852	0.4639	1.3033	0.5389
8	1.3231	0.5181	1.2733	0.5667
9	1.2956	0.5236	1.2976	0.5500
10	1.2692	0.5486	1.2429	0.5278
...				
100	0.3889	0.9556	0.5827	0.8944
101	0.3959	0.9569	0.6555	0.8833
102	0.3977	0.9472	0.5490	0.9111
103	0.3885	0.9542	0.5523	0.8889
104	0.4081	0.9347	0.6138	0.8889
105	0.4421	0.9361	0.6567	0.8500
106	0.4702	0.9236	0.6869	0.8500
107	0.4817	0.9278	0.6350	0.8944
108	0.4880	0.9319	0.5984	0.9000
109	0.4183	0.9569	0.6382	0.8889
110	0.4498	0.9361	0.6651	0.8778
...				
240	0.2812	0.9611	0.4676	0.9222
241	0.3636	0.9403	0.5573	0.8667
242	0.3716	0.9528	0.5833	0.8889
243	0.3765	0.9597	0.5784	0.8833
244	0.3580	0.9639	0.6080	0.8944
245	0.3307	0.9847	0.6098	0.8944
246	0.3427	0.9667	0.5898	0.8889
247	0.3259	0.9736	0.5901	0.9056
248	0.3145	0.9667	0.6185	0.8778
249	0.2937	0.9722	0.5586	0.9000

Appendix 5. Training and Validation Metrics in Fold 5 for Database 1

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	3.6941	0.1819	2.7891	0.1667
2	2.4689	0.2667	2.0751	0.2944
3	1.9527	0.3306	1.7240	0.3611
4	1.7228	0.3667	1.5683	0.4833
5	1.5914	0.3903	1.4686	0.4222
6	1.4474	0.4778	1.4232	0.4111
7	1.4210	0.4542	1.3507	0.4444
8	1.3495	0.4514	1.2895	0.5222
9	1.3382	0.4875	1.2662	0.5556
10	1.2709	0.5472	1.2244	0.5222
...				
100	0.4096	0.9431	0.5038	0.8833
101	0.4075	0.9389	0.4561	0.9333
102	0.4478	0.9236	0.5238	0.8944
103	0.4091	0.9403	0.5551	0.9000
104	0.3984	0.9514	0.5068	0.9000
105	0.3852	0.9389	0.4735	0.9222
106	0.3701	0.9458	0.4283	0.9167
107	0.3715	0.9472	0.4013	0.9333
108	0.3808	0.9361	0.4693	0.8889
109	0.3732	0.9500	0.4210	0.9222
110	0.3726	0.9375	0.6081	0.8667
...				
240	0.2985	0.9653	0.5482	0.8611
241	0.3076	0.9708	0.6324	0.8667
242	0.3912	0.9431	0.5240	0.8833
243	0.3116	0.9792	0.4588	0.9167
244	0.3458	0.9639	0.4428	0.9000
245	0.3126	0.9736	0.5431	0.8889
246	0.2986	0.9792	0.4234	0.9278
247	0.2949	0.9778	0.5040	0.9000
248	0.2660	0.9819	0.4351	0.8833
249	0.2784	0.9722	0.4891	0.9111

Appendix 6. Training and Validation Metrics in Fold 1 for Database 2

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	2.8984	0.2958	1.7719	0.4667
2	1.4708	0.5382	1.1261	0.6833
3	1.1866	0.6278	1.0257	0.6806
4	1.0534	0.6715	0.8688	0.7944
5	0.9599	0.7049	0.8530	0.7694
6	0.9194	0.7153	0.8062	0.7972
7	0.8678	0.7347	0.7873	0.7722
8	0.8211	0.7514	0.7890	0.7528
9	0.7981	0.7597	0.6749	0.8528
10	0.7419	0.7833	0.6764	0.8278
...				
100	0.3107	0.9444	0.4265	0.9194
101	0.3150	0.9438	0.4872	0.9056
102	0.3058	0.9493	0.4600	0.9000
103	0.2633	0.9715	0.4552	0.9111
104	0.2806	0.9597	0.4573	0.9028
105	0.2644	0.9625	0.5035	0.9056
106	0.2664	0.9604	0.4886	0.9111
107	0.2684	0.9611	0.4471	0.9139
108	0.2440	0.9646	0.4367	0.9139
109	0.2623	0.9632	0.4626	0.9250
110	0.2574	0.9583	0.4231	0.9250
...				
240	0.2090	0.9722	0.5176	0.9028
241	0.1913	0.9785	0.4939	0.9111
242	0.1937	0.9778	0.5403	0.9056
243	0.2058	0.9708	0.5293	0.8944
244	0.1921	0.9764	0.5319	0.9056
245	0.1937	0.9743	0.4936	0.9028
246	0.1877	0.9778	0.5164	0.9028
247	0.1980	0.9806	0.5288	0.8917
248	0.1922	0.9785	0.5315	0.9028
249	0.1833	0.9854	0.6263	0.8972

Appendix 7. Training and Validation Metrics in Fold 2 for Database 2

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	3.0915	0.2486	2.0171	0.3278
2	1.7374	0.4736	1.2997	0.6972
3	1.3051	0.6222	1.1137	0.6917
4	1.1629	0.6465	0.9283	0.7722
5	1.0102	0.7021	0.8215	0.7806
6	0.9436	0.7201	0.7927	0.8056
7	0.8864	0.7521	0.7552	0.7833
8	0.8201	0.7486	0.7041	0.8083
9	0.7967	0.7743	0.6771	0.8222
10	0.7985	0.7611	0.7050	0.8333
...				
100	0.2936	0.9528	0.3805	0.9139
101	0.2565	0.9597	0.3979	0.9222
102	0.2657	0.9590	0.3609	0.9306
103	0.2982	0.9472	0.4035	0.9083
104	0.2879	0.9507	0.4648	0.9111
105	0.2802	0.9563	0.3913	0.8972
106	0.2753	0.9632	0.3722	0.9167
107	0.2850	0.9569	0.3254	0.9333
108	0.2545	0.9556	0.3612	0.9222
109	0.2655	0.9604	0.3888	0.9139
110	0.2506	0.9618	0.4258	0.9056
...				
240	0.2191	0.9694	0.3688	0.9083
241	0.2051	0.9812	0.2945	0.9278
242	0.1866	0.9840	0.3482	0.9333
243	0.1817	0.9854	0.3305	0.9250
244	0.1766	0.9833	0.3502	0.9250
245	0.1775	0.9854	0.3017	0.9361
246	0.1607	0.9847	0.3102	0.9306
247	0.1706	0.9819	0.4037	0.9000
248	0.1618	0.9847	0.3212	0.9389
249	0.1746	0.9799	0.3084	0.9306

Appendix 8. Training and Validation Metrics in Fold 3 for Database 2

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	3.0935	0.3083	1.9949	0.5278
2	1.6893	0.5542	1.3135	0.6806
3	1.2612	0.6465	0.9971	0.7389
4	1.1073	0.6917	0.8756	0.7833
5	1.0074	0.7118	0.8313	0.8000
6	0.9260	0.7437	0.7471	0.8306
7	0.8849	0.7486	0.7579	0.8139
8	0.8316	0.7833	0.7171	0.8139
9	0.8331	0.7667	0.6890	0.8500
10	0.7809	0.7861	0.6229	0.8528
...				
100	0.2756	0.9611	0.3868	0.9167
101	0.2846	0.9611	0.4480	0.9194
102	0.2728	0.9611	0.4171	0.9167
103	0.3103	0.9396	0.4121	0.9028
104	0.2904	0.9590	0.4311	0.9083
105	0.2804	0.9583	0.4062	0.9222
106	0.2685	0.9618	0.4059	0.9278
107	0.2690	0.9611	0.4284	0.9250
108	0.2662	0.9708	0.4066	0.9333
109	0.2869	0.9556	0.3997	0.9250
110	0.2766	0.9646	0.4017	0.9306
...				
240	0.2025	0.9819	0.4536	0.9222
241	0.1931	0.9799	0.5174	0.9056
242	0.1866	0.9799	0.5201	0.9111
243	0.1835	0.9819	0.5246	0.9000
244	0.1812	0.9861	0.4652	0.9167
245	0.1807	0.9819	0.4477	0.9222
246	0.1851	0.9812	0.5069	0.9139
247	0.2073	0.9736	0.4387	0.9083
248	0.2288	0.9750	0.4273	0.9250
249	0.2356	0.9715	0.5040	0.9028

Appendix 9. Training and Validation Metrics in Fold 4 for Database 2

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	2.9723	0.2188	1.9106	0.3722
2	1.5766	0.4799	1.1649	0.6778
3	1.2341	0.6042	1.0544	0.6639
4	1.1329	0.6361	0.9226	0.7500
5	1.0270	0.6750	0.8438	0.7667
6	0.9296	0.7285	0.7983	0.7278
7	0.8955	0.7361	0.7815	0.7917
8	0.8644	0.7514	0.7118	0.7972
9	0.8448	0.7576	0.7085	0.8333
10	0.8010	0.7688	0.6719	0.7917
...				
100	0.2917	0.9542	0.3296	0.9389
101	0.2961	0.9507	0.3739	0.9278
102	0.3007	0.9451	0.3685	0.9194
103	0.3246	0.9417	0.3506	0.9306
104	0.3233	0.9465	0.3551	0.9222
105	0.3156	0.9507	0.3565	0.9361
106	0.2801	0.9500	0.3600	0.9278
107	0.3197	0.9347	0.3869	0.9194
108	0.3121	0.9514	0.3428	0.9333
109	0.2972	0.9535	0.3574	0.9361
110	0.2964	0.9563	0.3605	0.9139
...				
240	0.2441	0.9611	0.4429	0.9278
241	0.2249	0.9750	0.4427	0.9306
242	0.2151	0.9764	0.4036	0.9222
243	0.2017	0.9729	0.3974	0.9250
244	0.2050	0.9757	0.3967	0.9333
245	0.2055	0.9736	0.3553	0.9444
246	0.2025	0.9701	0.3349	0.9250
247	0.1828	0.9785	0.3867	0.9278
248	0.1896	0.9764	0.4598	0.9222
249	0.1832	0.9778	0.4098	0.9222

Appendix 10. Training and Validation Metrics in Fold 5 for Database 2

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	2.9930	0.2389	1.9142	0.3722
2	1.5321	0.5368	1.0885	0.6861
3	1.1326	0.6493	1.0231	0.7222
4	1.0728	0.6701	0.9019	0.7528
5	0.9486	0.7153	0.8440	0.7722
6	0.8841	0.7250	0.7733	0.7889
7	0.8619	0.7472	0.7426	0.8056
8	0.8202	0.7556	0.7041	0.8083
9	0.8133	0.7472	0.6973	0.8222
10	0.7901	0.7715	0.7146	0.7944
...				
100	0.3230	0.9451	0.3270	0.9444
101	0.2977	0.9500	0.3254	0.9417
102	0.2927	0.9569	0.3157	0.9361
103	0.2795	0.9549	0.3287	0.9389
104	0.3022	0.9431	0.3585	0.9333
105	0.3241	0.9438	0.3199	0.9417
106	0.3059	0.9403	0.3536	0.9389
107	0.3152	0.9389	0.3488	0.9389
108	0.3060	0.9514	0.3626	0.9361
109	0.2977	0.9458	0.3471	0.9306
110	0.2933	0.9472	0.3276	0.9444
...				
240	0.1762	0.9785	0.3594	0.9444
241	0.1642	0.9819	0.3281	0.9444
242	0.1979	0.9701	0.3686	0.9333
243	0.2346	0.9660	0.4369	0.9250
244	0.2525	0.9590	0.4521	0.9083
245	0.2495	0.9667	0.5564	0.9083
246	0.2883	0.9528	0.4415	0.9333
247	0.2625	0.9618	0.4427	0.9361
248	0.2563	0.9569	0.3667	0.9306
249	0.2289	0.9653	0.4095	0.9139

Appendix 11. Model Evaluation Metrics for Database 1

Fold	Metric	CNN	SVM	KNN
1/5	Accuracy	0.9278	0.9333	0.9556
	Precision	0.9317	0.9367	0.9599
	Recall	0.9278	0.9333	0.9556
	F1-score	0.9272	0.9336	0.9551
2/5	Accuracy	0.8944	0.9000	0.9111
	Precision	0.8992	0.9039	0.9156
	Recall	0.8944	0.9000	0.9111
	F1-score	0.8941	0.9004	0.9114
3/5	Accuracy	0.8500	0.8611	0.8611
	Precision	0.8752	0.8666	0.8666
	Recall	0.8500	0.8611	0.8611
	F1-score	0.8498	0.8613	0.8612
4/5	Accuracy	0.9167	0.9222	0.9278
	Precision	0.9206	0.9242	0.9302
	Recall	0.9167	0.9222	0.9278
	F1-score	0.9172	0.9228	0.9284
5/5	Accuracy	0.8556	0.9222	0.9444
	Precision	0.8604	0.9239	0.9488
	Recall	0.8556	0.9222	0.9444
	F1-score	0.8553	0.9226	0.9450

Appendix 12. Model Evaluation Metrics for Database 2

Fold	Metric	CNN	SVM	KNN
1/5	Accuracy	0.8972	0.9167	0.9111
	Precision	0.9015	0.9196	0.9155
	Recall	0.8972	0.9167	0.9111
	F1-score	0.8982	0.9170	0.9117
2/5	Accuracy	0.9111	0.9306	0.9417
	Precision	0.9145	0.9326	0.9457
	Recall	0.9111	0.9306	0.9417
	F1-score	0.9119	0.9309	0.9416
3/5	Accuracy	0.8972	0.9167	0.9083
	Precision	0.8976	0.9157	0.9080
	Recall	0.8972	0.9167	0.9083
	F1-score	0.8966	0.9156	0.9072
4/5	Accuracy	0.9250	0.9333	0.9472
	Precision	0.9261	0.9349	0.9480
	Recall	0.9250	0.9333	0.9472
	F1-score	0.9252	0.9331	0.9473
5/5	Accuracy	0.9222	0.9278	0.9250
	Precision	0.9233	0.9283	0.9260
	Recall	0.9222	0.9278	0.9250
	F1-score	0.9224	0.9279	0.9253